UNCUYO
UNIVERSIDAD NACIONAL DE CUYO

FACULTAD DE INGENIERÍA

Semáforos Inteligentes

Un Enfoque con Aprendizaje por Refuerzo Multiagente

Tesina

Licenciatura en Ciencias de la Computación

Autor

Juan Martín Morales

Directora

Dra. Ana Carolina Olivera

Ciudad de Mendoza, Argentina

2026

Resumen

Este trabajo aborda la congestión vehicular urbana mediante el control semafórico con aprendizaje por refuerzo multiagente. Cada intersección se representa por un agente que selecciona fases a partir de una observación local del tránsito. Se consideran la optimización de política proximal multiagente (*Multi-Agent Proximal Policy Optimization*, MAPPO) y con agentes heterogéneos (*Heterogeneous-Agent Proximal Policy Optimization*, HAPPO) con entrenamiento centralizado y ejecución descentralizada, y dos métodos independientes: optimización de política proximal (*Independent Proximal Policy Optimization*, IPPO) y red Q profunda (*Independent Deep Q-Network*, IDQN); además, se describe una adaptación de HAPPO con actualización secuencial de los actores. La espera acumulada por carril se codifica mediante una transformación logarítmico-lineal acotada y discretizada, con mayor resolución en valores bajos. La búsqueda de hiperparámetros se organiza en dos fases, una amplia y otra refinada, para los cuatro algoritmos en tres redes sintéticas; su extensión al microcentro de Mendoza requiere adaptar y verificar el protocolo urbano anterior. La comparación local y vecinal de MAPPO se circunscribe a las redes sintéticas. El escenario urbano de trabajo combina información cartográfica y conteos parciales con una demanda nominal de 220.000 viajes diarios y 73 programas fijos de 90 segundos. En doce horas, sus tasas representan 152.168,75 solicitudes esperadas. Seis semillas completaron dos ventanas de comprobación operacional, de dos y doce horas, bajo los criterios definidos. En la ventana larga quedaron entre 25.984 y 26.529 vehículos pendientes de ingreso. Esta comprobación caracteriza la operación del escenario y no evalúa políticas aprendidas ni valida integralmente la demanda frente a datos de campo.

Abstract

This work addresses urban traffic congestion through multi-agent reinforcement learning for traffic-signal control. Each intersection is represented by an agent that selects phases from a local observation of traffic. The algorithms considered are Multi-Agent Proximal Policy Optimization (MAPPO) and Heterogeneous-Agent Proximal Policy Optimization (HAPPO) with centralized training and decentralized execution, and Independent Proximal Policy Optimization (IPPO) and Independent Deep Q-Network (IDQN) with independent learning; the method also describes an adaptation of HAPPO with sequential actor updates. Accumulated lane waiting is represented by a bounded, discretized log-linear transformation with finer resolution at lower values. Hyperparameter search is organized in two phases, a broad exploration and a refined search, for the four algorithms across three synthetic networks; extending it to downtown Mendoza requires adapting and verifying the previous urban protocol. The MAPPO comparison of local and neighborhood variants is limited to the synthetic networks. The urban working scenario combines cartographic information and partial traffic counts with a nominal daily demand of 220,000 trips and 73 fixed signal programs with 90-second cycles. Its rates represent 152,168.75 expected requests over twelve hours. Six seeds completed two operational check windows, lasting two and twelve hours, under the specified criteria. Between 25,984 and 26,529 vehicles remained waiting to enter at the end of the longer window. These checks characterize scenario operation; they do not evaluate learned policies or establish comprehensive field validation of demand.

Agradecimientos

Página de agradecimientos

Índice general

1. Introducción **1**
1.1. Motivación . . . 3
1.2. Organización del documento . . . 4

2. Marco teórico **5**
2.1. Ingeniería de tránsito y simulación . . . 5
2.1.1. Terminología de tránsito . . . 6
2.1.2. Fundamentos de la dinámica vehicular . . . 8
2.1.3. Sistemas de control de tránsito . . . 14
2.1.4. Simulación de tránsito . . . 16
2.2. Aprendizaje por refuerzo (RL) . . . 21
2.2.1. Procesos de decisión de Markov (MDP) . . . 23
2.2.2. Métodos basados en valor y basados en política . . . 24
2.2.3. Aprendizaje profundo y redes neuronales . . . 28
2.2.4. Aprendizaje por refuerzo profundo (DRL) . . . 31
2.3. Aprendizaje por refuerzo multiagente (MARL) . . . 38
2.3.1. Fundamentos teóricos: juegos estocásticos, equilibrio de Nash y Dec-POMDP . . . 39
2.3.2. Desafíos del aprendizaje multiagente . . . 40
2.3.3. Paradigmas de entrenamiento y ejecución . . . 42
2.3.4. MADRL y líneas de base . . . 43
2.3.5. MAPPO . . . 44
2.3.6. HAPPO . . . 46
2.3.7. Antecedentes recientes del control semafórico multiagente . . . 52

3. Metodología **57**
3.1. Diseño experimental . . . 58
3.2. Escenarios de simulación . . . 59
3.2.1. Preparación de las redes de tráfico . . . 59
3.2.2. Redes de tráfico sintéticas . . . 60
3.2.3. Red de tráfico de la Ciudad de Mendoza . . . 63

3.3. Sistema de control multiagente 67
3.3.1. Entorno de simulación 68
3.3.2. Observación, acciones y recompensas 68
3.3.3. Agentes y algoritmos de aprendizaje 71
3.4. Entrenamiento y búsqueda de hiperparámetros 73
3.4.1. Condiciones de simulación y entrenamiento 73
3.4.2. Función objetivo de la búsqueda 75
3.4.3. Fase 1: búsqueda amplia 76
3.4.4. Fase 2: búsqueda con dominios reducidos 76
3.4.5. Criterios de selección de configuraciones 77
3.5. Comparación local y vecinal de MAPPO 78
3.5.1. Prueba preliminar de formulaciones 78
3.5.2. Entrenamiento de las variantes en las tres redes sintéticas 79
3.6. Evaluación y análisis de políticas 80
3.6.1. Métricas de desempeño y vehículos evaluados 80
3.6.2. Evaluación de las formulaciones local y vecinal de MAPPO 83
3.6.3. Análisis por pares y criterio de decisión 83
3.6.4. Entrenamiento y evaluación de los modelos finales 84
3.7. Reproducibilidad y alcance 85
3.7.1. Entorno de cómputo y envío de trabajos 85
3.7.2. Alcance y limitaciones 85

4. Análisis y discusión de resultados **87**
4.1. Resultados del ajuste de hiperparámetros 87
4.1.1. Estado de la consolidación y de las búsquedas 88
4.1.2. Resultados del ajuste y análisis de sensibilidad 88
4.1.3. Selección de configuraciones de HPO 88
4.2. Evaluación de políticas y comprobación de escenarios 88
4.2.1. Criterios de evaluación y métricas de comparación 88
4.2.2. Desempeño en escenarios sintéticos: 3×1, 3×3, 4×4 heterogéneo 88
4.2.3. Comprobación operacional del escenario de la Ciudad de Mendoza . 88
4.3. Discusión e interpretación de resultados 89

5. Conclusiones **91**

A. Arquitectura de software y especificaciones experimentales **92**
A.1. Diseño orientado a objetos del entorno de simulación 92
A.2. Jerarquía de funciones de recompensa y patrón Strategy 93
A.3. Patrones de diseño de la arquitectura 93
A.4. Parámetros de SUMO y programas semafóricos de referencia 95

A.4.1. Mendoza: caracterización de la red y antecedentes de partida 95
A.4.2. Construcción de los programas semafóricos fijos 97
A.4.3. Transformaciones de la demanda y selección del nominal 100
A.4.4. Entradas, ejecución y evidencia final 101
A.5. Especificaciones para reproducir los experimentos 103
A.5.1. Infraestructura y almacenamiento de las ejecuciones 103
A.5.2. Comprobaciones preliminares 103
A.5.3. Detalles de las redes sintéticas 104
A.5.4. Espacio de búsqueda inicial de fase 1 104
A.5.5. Semillas de fase 1 en las redes sintéticas 108
A.5.6. Arquitecturas y semillas de la búsqueda refinada sintética 109
A.5.7. Dominios declarados y significado de los parámetros 111
A.5.8. Antecedente del protocolo de aprendizaje urbano 115
A.5.9. Métricas y análisis por pares de A/C 117
A.6. Detalles del aprendizaje y de la codificación 118
A.6.1. Desarrollo algebraico del codificador 119
A.6.2. Aplicación de la máscara de acciones 120
A.6.3. Configuración fija de la comparación multiescenario 121
A.6.4. Calibraciones y semillas de cooperación 122
A.6.5. Preparación de las figuras de escenarios 122
A.6.6. Vistas documentales de los escenarios 123

Índice de figuras

1.1. Evolución de la flota vehicular circulante estimada en Argentina entre 1990 y 2025, expresada en millones de vehículos. La línea y los marcadores representan los valores anuales disponibles; el área sombreada facilita la lectura de su magnitud. Elaboración propia a partir de reportes de AFAC. 1

2.1. Movimientos vehiculares, fases y cruces peatonales 7
2.2. Representación esquemática de movimientos vehiculares y puntos de conflicto en una intersección semaforizada. 8
2.3. Relaciones temporales y espaciales entre vehículos consecutivos de una corriente de tránsito: intervalo (h) y espaciamiento (s). 11
2.4. Comparación esquemática de distribuciones de intervalos temporales entre vehículos en flujo libre y en una corriente con formación de agrupamientos vehiculares. Las curvas representan situaciones conceptuales y no una distribución empírica particular. 13
2.5. Distribución temporal de la tasa de descarga y pérdidas asociadas durante un intervalo de verde en una aproximación semaforizada. 15
2.6. Correspondencia entre la circulación real y su representación en SUMO . . 17
2.7. Interfaz sumo-gui durante una simulación microscópica 18
2.8. Esquema conceptual de carriles, línea de detención y conexiones de una intersección semaforizada bajo circulación por la derecha. La fase representada habilita movimientos directos opuestos del eje horizontal y retiene los movimientos conflictivos. 19
2.9. Interacción agente-entorno en un proceso de decisión de Markov [SB18]. . . 23
2.10. Esquema de un método basado en valor 25
2.11. Esquema de un método basado en política 26
2.12. Arquitectura actor-crítico aplicada al control semafórico 28
2.13. Operaciones de una unidad neuronal artificial: suma ponderada de las entradas, incorporación del sesgo y aplicación de una función de activación. . 29
2.14. Arquitectura conceptual de una red neuronal *feedforward* con dos capas ocultas. 30

2.15. Flujo conceptual de DQN con repetición de experiencias y red objetivo. La red en línea selecciona acciones y recibe las actualizaciones; la red objetivo proporciona el término de *bootstrapping* del valor objetivo diferencia temporal (*Temporal-Difference*, TD). Elaboración propia a partir del algoritmo descrito por Mnih et al. [MKS+15]. 33
2.16. Red de política para un espacio discreto de acciones 35
2.17. Interacción agente-entorno en MARL cooperativo 40
2.18. Evolución de las políticas de dos agentes bajo WoLF-PHC en Piedra, Papel o Tijera 42
2.19. Paradigma de entrenamiento centralizado con ejecución descentralizada (CTDE). Durante el entrenamiento, el crítico utiliza información global privilegiada para estimar valores o ventajas que orientan la actualización de los actores locales; durante la ejecución, los actores solo requieren observaciones locales. 43
2.20. Esquema de actualización secuencial de HAPPO. A diferencia de PPO independiente, la actualización de cada agente está condicionada por el factor de ventaja reponderada M_{i_m}, que incorpora los cambios efectuados por los agentes que lo precedieron en la secuencia. 49

3.1. Etapas y dependencias funcionales del diseño experimental: (a) ajuste y comparación de algoritmos; (b) contraste de formulaciones local y vecinal en MAPPO. 58
3.2. Redes sintéticas: (a) corredor 3×1, (b) cuadrícula 3×3 y (c) red heterogénea $4 \times 4_H$. 62
3.3. Dominio urbano de Mendoza, nodos semaforizados (puntos rojos) e ingreso oriental asociado a la zona TAZ 28. 64
3.4. Solicitudes esperadas por hora y horizontes de evaluación en el escenario de Mendoza: corto (06:30–08:30, 2 h) y largo (06:00–18:00, 12 h). 66
3.5. Intercambios de información entre simulación, entorno, muestreo y aprendizaje en `marlTLC`. 67
3.6. Curva de cuantización del codificador local de espera para $I = 22$ niveles y $M_{\text{lane}} = 1.136$ veh·s. 70
3.7. Esquema de actualización de actores con tres agentes ilustrativos: (a) actualización paralela en MAPPO; (b) actualización secuencial con propagación del factor posoptimización en HAPPO. 73
3.8. Línea temporal de transición de fase y bloqueo por máscara de acciones con decisiones cada $\Delta t = 5$ s. 75
3.9. Derivación de dominios reducidos y configuración inicial en la transición de la fase 1 a la fase 2 de búsqueda. 77

3.10. Estructura de la observación del actor y de la señal de recompensa en MAPPO: (a) formulación local A; (b) formulación vecinal C. 80
3.11. Criterio de inclusión en el conjunto de vehículos evaluados ($\mathcal{V}_{\text{ins}}$) según el instante efectivo de inserción. 81

4.1. Vehículos pendientes de ingreso durante las seis ejecuciones de doce horas. El rayado señala la hora en que se redistribuyeron las solicitudes. 89

A.1. Diagrama de clases del entorno de simulación y de sus relaciones principales. 94
A.2. Especificación UML del módulo de recompensas. 94
A.3. Ejemplo de codificación de observaciones. 120
A.4. Distribución de los intervalos de verde, amarillo y todo rojo en los ciclos de referencia de 90 segundos. 123
A.5. Zonas de análisis de tránsito y sus tramos asociados en la red de Mendoza. 124

Índice de tablas

2.1. Variables de tránsito macroscópicas y microscópicas 10
2.2. Archivos de entrada y configuración empleados en una simulación de SUMO. 21
2.3. Comparación sintética de familias algorítmicas PPO en entornos multiagente cooperativos. 51

3.1. Duraciones de los programas semafóricos de referencia en las redes de tráfico sintéticas (segundos). 60
3.2. Perfiles de vehículos de las redes de tráfico sintéticas. 60
3.3. Topología, control y demanda de las redes sintéticas. Se excluyen los tramos y carriles internos de los cruces. 63
3.4. Observaciones y recompensas de las búsquedas sintéticas y de la comparación A/C. 71
3.5. Calentamiento y duración del control por procedimiento. 74
3.6. Variantes examinadas en la prueba preliminar sobre la red $4 \times 4_H$. . . . 79

4.1. Comprobación del escenario urbano con programas fijos: síntesis por ventana. 88

A.1. Herramientas y responsabilidades en el sistema experimental. 93
A.2. Duraciones nominales de la red mixta preliminar de Mendoza, en segundos. 96
A.3. Comprobaciones históricas sobre la red mixta preliminar (`v14`). 96
A.4. Desfases modificados entre el plan fijo inicial (`T0`) y el plan fijo coordinado final (`P2`). 98
A.5. Programas fijos del plan `P2`: controladores 1–25. 98
A.6. Programas fijos del plan `P2`: controladores 26–50. 99
A.7. Programas fijos del plan `P2`: controladores 51–73. 99
A.8. Rutas del catálogo histórico asociadas con el movimiento recto analizado en Tiburcio Benegas. 100
A.9. Tasas de arribo de los cuatro pares que intervienen en el intercambio final `O2` durante el intervalo $[10.800, 14.400)$ s. 100
A.10. Síntesis de los tratamientos evaluados para la elección del volumen nominal diario de Mendoza. 101

A.11.Opciones efectivas de SUMO en las corridas de comprobación operacional de Mendoza. 102
A.12.Corridas de aceptación con programas fijos (O2/P2): horizonte de dos horas. 102
A.13.Corridas de aceptación con programas fijos (O2/P2): horizonte de doce horas. 102
A.14.Espacio inicial de fase 1 para MAPPO, IPPO y HAPPO. Las excepciones por algoritmo se indican en las filas. 105
A.15.Espacio inicial de fase 1 para IDQN. Los contadores se expresan en pasos según la interfaz del algoritmo. 106
A.16.Factores de estudio y valores fijos documentados para la fase 1 sintética. . 107
A.17.Correspondencia entre los parámetros de búsqueda y sus claves de configuración. 108
A.18.Semillas por escenario sintético en la fase 1. 108
A.19.Arquitecturas y semillas de la búsqueda refinada para las doce combinaciones sintéticas. La semilla de evaluación es común a los algoritmos del mismo escenario; las semillas de TPE no son réplicas de entrenamiento. . . 109
A.20.Dominios declarados para la búsqueda `phase2_refined` de HAPPO por escenario y arquitectura. 112
A.21.Dominios declarados para la búsqueda `phase2_refined` de IDQN por escenario y arquitectura. 112
A.22.Dominios declarados para la búsqueda refinada de IPPO por escenario y arquitectura. 113
A.23.Dominios declarados para la búsqueda refinada de MAPPO por escenario y arquitectura. 113
A.24.Hiperparámetros fijos de aprendizaje de MAPPO. 121

Siglas

A2C Actor–crítico con ventaja (*Advantage Actor-Critic*).

AFAC Asociación de Fábricas Argentinas de Componentes.

API Interfaz de programación de aplicaciones (*Application Programming Interface*).

ASHA Algoritmo asíncrono de reducción sucesiva a la mitad (*Asynchronous Successive Halving Algorithm*).

BFS Búsqueda en anchura (*Breadth-First Search*).

CCAD Centro de Cómputo de Alto Desempeño.

CCAR Centro de Cómputo de Alto Rendimiento.

CI Tasa de completitud de vehículos insertados (*Completion of Inserted Vehicles*).

CLIP Objetivo sustituto recortado (*Clipped surrogate objective*).

CO Monóxido de carbono.

CO2 Dióxido de carbono.

CONICET Consejo Nacional de Investigaciones Científicas y Técnicas.

CPU Unidad central de procesamiento (*Central Processing Unit*).

CR Tasa de completitud (*Completion Rate*).

CTDE Entrenamiento centralizado con ejecución descentralizada (*Centralized Training with Decentralized Execution*).

CVaR Valor en riesgo condicional (*Conditional Value at Risk*).

DAG Grafo acíclico dirigido (*Directed Acyclic Graph*).

DDPG Gradiente de política determinista profundo (*Deep Deterministic Policy Gradient*).

Dec-POMDP Proceso de decisión de Markov parcialmente observable descentralizado (*Decentralized Partially Observable Markov Decision Process*).

DLR Centro Aeroespacial Alemán (*Deutsches Zentrum für Luft- und Raumfahrt*).

DQN Red Q profunda (*Deep Q-Network*).

DRL Aprendizaje por refuerzo profundo (*Deep Reinforcement Learning*).

E/S Entrada/salida (*Input/Output*).

FCEFN Facultad de Ciencias Exactas, Físicas y Naturales.

FHWA Administración Federal de Carreteras (*Federal Highway Administration*).

FLIR Infrarrojo de barrido frontal (*Forward Looking Infrared*).

GAE Estimación generalizada de ventaja (*Generalized Advantage Estimation*).

GPU Unidad de procesamiento gráfico (*Graphics Processing Unit*).

GUI Interfaz gráfica de usuario (*Graphical User Interface*).

HAML Aprendizaje espejo con agentes heterogéneos (*Heterogeneous-Agent Mirror Learning*).

HAPPO Optimización de política proximal con agentes heterogéneos (*Heterogeneous-Agent Proximal Policy Optimization*).

HARL Aprendizaje por refuerzo con agentes heterogéneos (*Heterogeneous-Agent Reinforcement Learning*).

HATRPO Optimización de política por región de confianza con agentes heterogéneos (*Heterogeneous-Agent Trust Region Policy Optimization*).

HBEFA Manual de factores de emisión para transporte por carretera (*Handbook Emission Factors for Road Transport*).

HC Hidrocarburos no combustionados.

HPC Computación de alto rendimiento (*High-Performance Computing*).

HPO Optimización de hiperparámetros (*Hyperparameter Optimization*).

IA Inteligencia artificial (*Artificial Intelligence*).

IDQN Red Q profunda independiente (*Independent Deep Q-Network*).

IL Aprendizaje independiente (*Independent Learning*).

IPPO Optimización de política proximal independiente (*Independent Proximal Policy Optimization*).

IQM Media intercuartílica (*Interquartile Mean*).

IR Tasa de inserción (*Insertion Rate*).

ITE Instituto de Ingenieros de Transporte (*Institute of Transportation Engineers*).

KL Divergencia de Kullback–Leibler.

MADDPG Gradiente de política determinista profundo multiagente (*Multi-Agent Deep Deterministic Policy Gradient*).

MADRL Aprendizaje por refuerzo profundo multiagente (*Multi-Agent Deep Reinforcement Learning*).

MAPPO Optimización de política proximal multiagente (*Multi-Agent Proximal Policy Optimization*).

MARL Aprendizaje por refuerzo multiagente (*Multi-Agent Reinforcement Learning*).

MDP Proceso de decisión de Markov (*Markov Decision Process*).

ML Aprendizaje automático (*Machine Learning*).

MLP Perceptrón multicapa (*Multi-Layer Perceptron*).

NFS Sistema de archivos de red (*Network File System*).

NOx Óxidos de nitrógeno.

OD Origen-destino (*Origin-Destination*).

OOD Fuera de distribución (*Out-of-Distribution*).

OSM OpenStreetMap.

PHEMlight Modelo de emisiones para vehículos livianos y pesados (*Passenger car and Heavy duty Emission Model light*).

POMDP Proceso de decisión de Markov parcialmente observable (*Partially Observable Markov Decision Process*).

POSIX Interfaz portable de sistemas operativos (*Portable Operating System Interface*).

PPO Optimización de política proximal (*Proximal Policy Optimization*).

QMIX Red de mezcla de valores Q (*Q-Mixing Network*).

ReLU Unidad lineal rectificada (*Rectified Linear Unit*).

RL Aprendizaje por refuerzo (*Reinforcement Learning*).

SGD Descenso de gradiente estocástico (*Stochastic Gradient Descent*).

SL Aprendizaje supervisado (*Supervised Learning*).

SLURM Herramienta simple de Linux para gestión de recursos (*Simple Linux Utility for Resource Management*).

SUMO Simulación de movilidad urbana (*Simulation of Urban MObility*).

TAZ Zona de análisis de tránsito (*Traffic Analysis Zone*).

TCP Protocolo de control de transmisión (*Transmission Control Protocol*).

TD Diferencia temporal (*Temporal-Difference*).

TPE Estimador de Parzen estructurado en árbol (*Tree-structured Parzen Estimator*).

TraCI Interfaz de control del tránsito (*Traffic Control Interface*).

TRPO Optimización de política por región de confianza (*Trust Region Policy Optimization*).

UL Aprendizaje no supervisado (*Unsupervised Learning*).

UML Lenguaje unificado de modelado (*Unified Modeling Language*).

UTM Sistema de coordenadas universal transversal de Mercator (*Universal Transverse Mercator*).

VDN Redes de descomposición de valor (*Value-Decomposition Networks*).

VF Función de valor (*Value Function*).

WGS84 Sistema Geodésico Mundial 1984 (*World Geodetic System 1984*).

WoLF-PHC Ascenso de colina en la política con ganar o aprender rápido (*Win or Learn Fast Policy Hill-Climbing*).

XML Lenguaje de marcado extensible (*Extensible Markup Language*).

Glosario

G_t **(retorno)** Suma acumulada, usualmente descontada, de las recompensas futuras a partir del instante t.

$G_{\mathbf{wait}}$ **(coeficiente de Gini de esperas)** Índice adimensional entre 0 y 1 que cuantifica la desigualdad entre las medias temporales de espera por controlador semafórico; no describe la distribución entre vehículos.

K_i **(número de acciones)** Número de fases semafóricas seleccionables por el agente i. Excluye las fases con amarillo y el estado final de todo rojo; las restricciones temporales pueden reducir el subconjunto de acciones habilitado en un instante.

$M_{i_{1:m}}$ **(factor de reponderación HAPPO)** Factor acumulativo de razones de probabilidad que repondera la ventaja conjunta en la actualización secuencial de HAPPO para reflejar los cambios de política de los agentes predecesores.

$Q(s, a)$ **(valor de acción)** Retorno esperado de ejecutar la acción a en el estado s y continuar posteriormente de acuerdo con una política determinada.

$R_t^{\mathbf{target}}$ **(retorno objetivo)** Estimación de retorno utilizada como referencia para ajustar la predicción de valor del crítico en el instante t.

$S[\pi_\theta]$ **(entropía de la política)** Medida de la dispersión de la distribución de acciones de la política; su inclusión en el objetivo favorece la exploración.

$V(s)$ **(valor de estado)** Retorno esperado al comenzar en el estado s y seguir una política determinada, promediando las acciones que dicha política selecciona.

$W_i^{(t)}$ **(espera atribuida a la intersección)** Suma de las esperas atribuidas a los carriles controlados por el agente i en el paso de decisión t, en veh·s. Se utiliza en la observación vecinal y en las diferencias de espera de la recompensa; depende de la memoria temporal finita de SUMO y no equivale a la espera total por viaje de TripInfo.

$W_{\mathbf{net}}$ **(demora acumulada global)** Métrica agregada que suma los segundos de espera de los vehículos detenidos en la red durante la ventana de evaluación.

α **(tasa de aprendizaje)** Paso de actualización que regula la magnitud del ajuste de los parámetros o valores en cada iteración de aprendizaje. En arquitecturas actor–crítico se especializa mediante subíndices, tales como α_θ para la red del actor y α_ϕ (o η) para el crítico.

δ_t **(error de diferencia temporal)** Diferencia entre la recompensa más el valor descontado del estado siguiente y la estimación de valor del estado actual.

ϵ_{clip} **(margen de recorte)** Hiperparámetro de PPO que delimita el intervalo utilizado para recortar la razón entre la probabilidad nueva y la anterior de una acción.

ϵ_{exp} **(exploración)** Probabilidad o coeficiente que controla la selección de acciones exploratorias frente a las acciones favorecidas por la política o la función de valor.

η **(tasa del optimizador)** Tamaño de paso empleado por el optimizador en la actualización de parámetros mediante descenso de gradiente.

γ **(factor de descuento)** Parámetro en el intervalo $[0, 1)$ que pondera la contribución de las recompensas futuras al retorno esperado.

$\hat{A}_t$ **(ventaja estimada)** Estimación del valor de la función de ventaja en el paso temporal t (habitualmente calculada mediante GAE), que cuantifica cuánto supera o desmejora el rendimiento de la acción ejecutada respecto del valor basal esperado del estado.

λ **(suavizado de la ventaja)** Hiperparámetro que pondera errores de diferencia temporal de distintos horizontes al estimar la ventaja y equilibra sesgo y varianza.

$\mathcal{L}(\theta)$ **(función de pérdida)** Función escalar que cuantifica el error del modelo y que se minimiza para ajustar los parámetros θ.

ϕ **(parámetros del crítico)** Vector de parámetros entrenables de la red que estima la función de valor del crítico.

π_θ **(política parametrizada)** Distribución de probabilidad sobre las acciones condicionada por el estado u observación y controlada por el vector de parámetros entrenables θ.

ρ_{coop} **(ponderación cooperativa)** Coeficiente en $[0, 1)$ que pondera la combinación entre la recompensa local de una intersección y la señal promedio de sus vecinos topológicos; los dominios de búsqueda pueden restringirlo a un intervalo menor.

θ **(parámetros de la política)** Vector de parámetros entrenables que determina la distribución de acciones de la política del actor.

a **(aceleración vehicular)** Parámetro de aumento de velocidad del tipo de vehículo en SUMO, expresado en metros por segundo cuadrado. En las ecuaciones de aprendizaje, la letra a puede representar una acción; son usos distintos que se distinguen por el contexto.

a_i, A_t **(acción)** Decisión ejecutada por un agente. En formulaciones temporales, A_t denota la variable aleatoria de acción en el paso t; en entornos multiagente, a_i representa la acción local del agente i y $\mathbf{a} = (a_1, \ldots, a_n)$ la acción conjunta. En el control semafórico corresponde a la fase asignada a la intersección.

c_1 **(coeficiente de valor)** Peso asignado al término de pérdida de la función de valor en el objetivo combinado de PPO.

c_2 **(coeficiente de entropía)** Peso asignado al término de entropía que incentiva la exploración en el objetivo combinado de PPO.

d **(desaceleración vehicular)** Parámetro de frenado del tipo de vehículo en SUMO, expresado como una magnitud positiva en metros por segundo cuadrado.

m_i **(carriles controlados)** Número de carriles entrantes controlados por el agente i. En el entorno utilizado, la observación local incorpora una medida de espera codificada por cada uno de estos carriles.

o_i **(observación local)** Información disponible para el agente i durante la ejecución descentralizada. Se obtiene a partir de los sensores o variables observables de la intersección que controla.

r_t **(recompensa)** Señal escalar recibida en el paso temporal t después de ejecutar una acción. Se utiliza para orientar la actualización de la política o de la función de valor.

s **(estado global)** Representación conjunta de las variables relevantes del entorno en un instante determinado. En el modelo multiagente puede incluir la información de todas las intersecciones y sus corrientes vehiculares.

$v_{\text{máx}}$ **(velocidad máxima vehicular)** Velocidad máxima configurada para un tipo de vehículo, expresada en metros por segundo; el límite de la vía también restringe la velocidad permitida.

Binding Mecanismo que conecta una biblioteca escrita en un lenguaje de programación con otro lenguaje para poder utilizar sus funciones.

Bootstrap Procedimiento de remuestreo con reemplazo utilizado para aproximar la incertidumbre de un estimador a partir de las observaciones disponibles.

Buffer Estructura que almacena temporalmente datos para su posterior procesamiento o remuestreo en algoritmos de aprendizaje.

***Epoch* (época)** Recorrido completo de optimización sobre el conjunto de datos recolectado para una iteración; puede reutilizarse durante varios *epochs*.

Logit Puntuación real sin normalizar que una red de política produce para una acción antes de convertirla en probabilidad.

***Minibatch* (minilote)** Subconjunto reducido de muestras extraído de un lote utilizado para estimar el gradiente y actualizar los parámetros de un modelo durante el entrenamiento.

Replay buffer Estructura que almacena experiencias previas para remuestrearlas durante el entrenamiento de un método basado en valor.

Rollout Secuencia ordenada de transiciones generada mientras una política interactúa con el entorno.

Softmax Función que transforma un conjunto de puntuaciones reales en probabilidades no negativas cuya suma es uno.

checkpoint Archivo que guarda los parámetros del modelo y, si está implementado, el estado del entrenamiento en una iteración intermedia o final. Puede utilizarse para reanudar o evaluar desde ese punto; su existencia no demuestra que el ensayo esté completo ni que sus resultados sean admisibles.

Actor (Aprendizaje por Refuerzo) Componente de la arquitectura del modelo de aprendizaje responsable de seleccionar y ejecutar la acción en el entorno basándose en el estado actual y la política que está aprendiendo.

Agente Entidad computacional que observa el estado de su entorno y ejecuta acciones según una política determinada con el objetivo de maximizar su recompensa a largo plazo.

Ciclo (Ingeniería de Tránsito) Secuencia completa y repetitiva de todas las fases semafóricas programadas en una intersección particular.

Cola vehicular Fila de vehículos que se detienen o avanzan muy lentamente debido a una restricción en la capacidad de la vía, tal como la luz roja de un semáforo.

Crítico (Aprendizaje por Refuerzo) Componente encargado de estimar la función de valor, prediciendo la recompensa futura esperada para evaluar la calidad de las acciones tomadas por el Actor y así guiar su actualización.

Densidad vehicular Número de vehículos presentes en un tramo o longitud específica de un carril en un instante dado.

desfase semafórico Relación temporal entre un punto de referencia de un programa semafórico local y el reloj común de coordinación.

Distribución loguniforme Distribución uniforme en el logaritmo de una variable positiva. Asigna la misma probabilidad a intervalos con igual razón entre sus extremos y permite explorar distintas escalas de magnitud.

Distribución uniforme continua Distribución que asigna la misma probabilidad a intervalos de igual longitud dentro de un rango. En la búsqueda de hiperparámetros, delimita propuestas para parámetros continuos.

Dominio de búsqueda Conjunto de rangos u opciones permitidos para los hiperparámetros en una búsqueda de configuraciones.

Ensayo de búsqueda de hiperparámetros Evaluación de una configuración concreta de hiperparámetros mediante entrenamiento y registro del objetivo y sus métricas. Puede terminar antes del presupuesto previsto y no equivale a un episodio ni a una iteración de entrenamiento.

Entorno El sistema dinámico con el que interactúa el agente. Responde a las acciones del agente produciendo transiciones de estado y emitiendo señales de recompensa.

Episodio Secuencia de interacción que comienza al reiniciar el entorno y termina cuando se alcanza una condición terminal o el horizonte máximo definido.

Estado Representación cuantitativa o cualitativa de la situación actual del entorno en un instante de tiempo específico. Sirve como base para que el agente seleccione su próxima acción.

Estudio de búsqueda de hiperparámetros Búsqueda que agrupa ensayos y su historial bajo una función objetivo y un espacio de búsqueda. En el protocolo de este proyecto, cada estudio se asocia con un algoritmo, un escenario y una arquitectura.

Fase (Ingeniería de Tránsito) Estado temporal de un controlador semafórico que habilita el derecho de paso a uno o más movimientos o corrientes vehiculares compatibles entre sí para que crucen una intersección de forma segura.

Flujo vehicular Cantidad de vehículos que pasan por una sección transversal específica de un carril o calzada durante un intervalo de tiempo determinado.

Función de Valor Estimación matemática de la cantidad total de recompensa que un agente puede esperar acumular en el futuro, comenzando desde un estado particular y siguiendo una política específica.

Grado de vecindad Cantidad de agentes vecinos directos de un agente. En el entorno utilizado, una conexión vial entre carriles controlados establece una relación de vecindad registrada para ambos agentes, incluso si la vía es de sentido único.

Hiperparámetro Valor de configuración que se fija antes de un entrenamiento, como la tasa de aprendizaje o el tamaño de las capas ocultas. Se distingue de los parámetros de la red, que se ajustan durante el aprendizaje.

Intervalo de confianza Rango que expresa la incertidumbre de una estimación. Su nivel nominal indica la proporción de estudios repetidos en la que el procedimiento cubriría el valor verdadero bajo sus supuestos; no asigna una probabilidad a ese valor una vez calculado el intervalo.

Iteración de entrenamiento Ciclo completo en el que se recolectan datos con una política y se realizan una o más actualizaciones de sus parámetros.

Lote (*batch*) Conjunto de muestras o transiciones procesadas conjuntamente durante una operación de entrenamiento..

Memoria temporal de espera Intervalo finito sobre el cual SUMO acumula la espera consultada mediante TraCI; se distingue de la espera total registrada por viaje en TripInfo.

Máscara de acciones Representación de las acciones habilitadas en una decisión; excluye del proceso de selección las acciones no admisibles. En el control semafórico, la admisibilidad depende de las fases, las transiciones y los tiempos definidos por el control; la máscara no constituye por sí sola una garantía de seguridad vial en campo.

Observación Subconjunto de información del estado completo del entorno que es efectivamente visible o medible por un agente individual, especialmente relevante en escenarios de observabilidad parcial.

Par de políticas Unidad experimental compuesta por dos políticas entrenadas con la misma semilla y evaluadas con el mismo conjunto de semillas de simulación.

Percentil Valor que deja una proporción determinada de las observaciones por debajo o a su altura. El percentil 99 corresponde aproximadamente al 99 % de las observaciones.

Política (Aprendizaje por Refuerzo) Estrategia que define el comportamiento del agente en un momento dado, mapeando los estados observados a probabilidades de selección de las acciones disponibles.

Proceso de Decisión de Markov Marco matemático utilizado para modelar la toma de decisiones en situaciones donde los resultados son en parte aleatorios y en parte bajo el control de un agente tomador de decisiones.

Recompensa Señal escalar devuelta por el entorno que evalúa cuán buena o mala fue la acción ejecutada por el agente en un estado particular. El objetivo del agente es maximizar la suma total de estas señales.

Semilla Número que inicializa un generador de números pseudoaleatorios. Permite repetir su secuencia cuando se conservan el procedimiento y la configuración; usar la misma semilla no garantiza resultados idénticos si se modifican las decisiones del agente o el entorno.

Serialización Conversión de un objeto o mensaje a una representación que puede almacenarse o transmitirse y luego reconstruirse.

Telemetría Conjunto de mediciones que el simulador registra durante la ejecución, como velocidades, posiciones, detenciones y tiempos de espera.

Teletransporte de SUMO Mecanismo de SUMO, denominado originalmente *teleport*, que retira temporalmente un vehículo bloqueado y lo reinserta para evitar que la simulación se detenga.

Transición (Aprendizaje por Refuerzo) Registro de un paso de interacción que contiene el estado u observación actual, la acción ejecutada, la recompensa recibida, el estado siguiente y, cuando corresponde, un indicador de terminación.

vehículos pendientes de ingreso Vehículos cuya salida programada ya corresponde, pero que todavía no han podido insertarse en la red vial; se distinguen de los vehículos retenidos dentro de ella.

Ventaja (Aprendizaje por Refuerzo) Métrica que cuantifica cuánto mejor o peor resulta tomar una acción particular en un estado dado, en comparación con el desempeño promedio esperado según la política actual.

1

Introducción

En las últimas décadas, el crecimiento del parque automotor y la urbanización sostenida han convertido a la congestión vehicular en uno de los principales desafíos para la movilidad urbana. Según los reportes de flota vehicular de Argentina elaborados por la Asociación de Fábricas Argentinas de Componentes (AFAC), y tomando 2012 como línea base, el volumen de vehículos aumentó un 21,7 % en 2017. Esta tendencia continuó durante la última década y alcanzó un incremento acumulado del 44,4 % en 2025 (véase la Figura 1.1) [Aso18, Aso26].

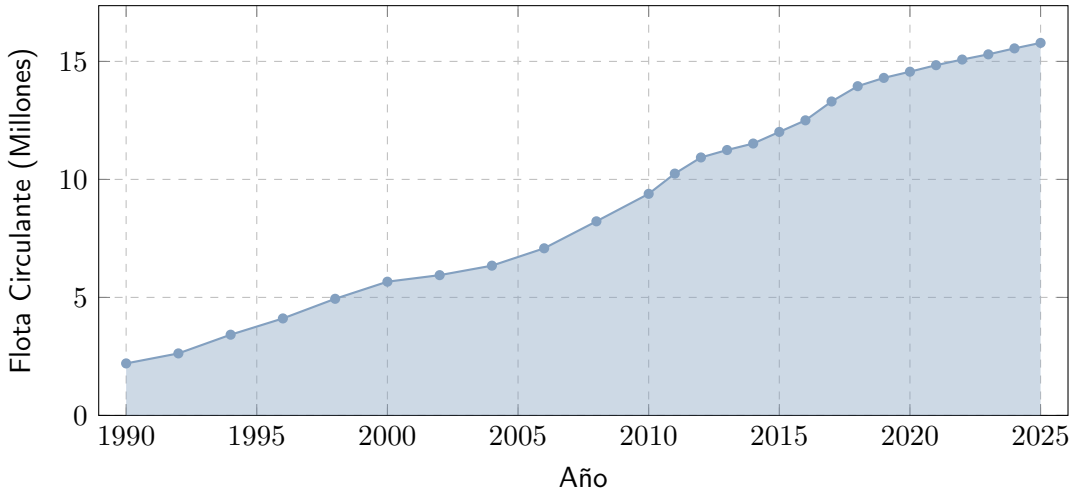


Figura 1.1: Evolución de la flota vehicular circulante estimada en Argentina entre 1990 y 2025, expresada en millones de vehículos. La línea y los marcadores representan los valores anuales disponibles; el área sombreada facilita la lectura de su magnitud. Elaboración propia a partir de reportes de AFAC.

Como refleja la Figura 1.1, la flota vehicular activa en Argentina experimentó una expansión sostenida entre 1990 y 2025, pasando de aproximadamente 2,2 millones a 15,78 millones de unidades operativas [Aso18, Aso26]. En los reportes de AFAC, la flota circulante comprende automóviles y vehículos comerciales livianos y pesados en condiciones de circular, excluyendo acoplados y remolques; representa una estimación del parque activo bajo esa definición, no el total histórico de unidades patentadas ni la cantidad de viajes realizados. Este incremento acumulado ejerce una presión creciente sobre redes viales consolidadas que poseen una capacidad física fija, agudizando la formación de colas y demoras en las intersecciones urbanas críticas.

La congestión deteriora las condiciones de operación de la red e incrementa las demoras y las detenciones de los vehículos [CyMR18]. Este trabajo aborda decisiones de fases semafóricas en tres escenarios sintéticos y prepara también una red del microcentro de la Ciudad de Mendoza con información parcial de tránsito. La comparación local y vecinal de MAPPO se circunscribe a las redes sintéticas; la aplicación del aprendizaje a Mendoza queda pendiente de adaptación. El escenario urbano representa 220.000 viajes nominales diarios y utiliza 73 programas semafóricos fijos de 90 segundos. Su comprobación operacional se presenta por separado. La geometría y las condiciones de construcción se especifican en la Sección 3.2.

Frente a la congestión, una alternativa que no requiere modificaciones importantes de la infraestructura consiste en ajustar la operación de las intersecciones mediante sistemas semafóricos adaptativos. Los enfoques tradicionales pueden presentar limitaciones para responder a variaciones en el flujo vehicular [PW16]. En este contexto, los avances en inteligencia artificial (*Artificial Intelligence*, IA) y en particular el aprendizaje por refuerzo (*Reinforcement Learning*, RL) permiten modelar los semáforos como agentes que aprenden políticas de control a partir de la interacción con el entorno. Las acciones corresponden a cambios de fase y la recompensa puede definirse, por ejemplo, como el negativo del tiempo de espera acumulado. Diversos estudios han mostrado la viabilidad de este enfoque, desde algoritmos tabulares aplicados a una sola intersección [Wie00] hasta modelos de redes coordinadas basados en redes neuronales profundas y control multiagente [W$^+$19].

Sin embargo, la aplicación simultánea de este enfoque en múltiples intersecciones introduce desafíos adicionales. Un esquema centralizado puede resultar poco escalable, pues el espacio de acciones conjunto crece multiplicativamente con el número de intersecciones [ACS24]. En una formulación de aprendizaje por refuerzo multiagente (*Multi-Agent Reinforcement Learning*, MARL) con aprendizaje independiente (*Independent Learning*, IL), cada semáforo optimiza su propia intersección sin considerar explícitamente el efecto de sus decisiones sobre el resto de la red [AS22]. Además, como todos los agentes aprenden y modifican sus políticas al mismo tiempo, cada uno observa un entorno no estacionario. Esto dificulta la convergencia de los algoritmos de agente único [FNF$^+$17].

Por esta razón, este trabajo estudia el control multiagente con un objetivo de coor-

dinación vial. Cada agente recibe una recompensa individual. Las búsquedas informadas emplean una señal local, mientras que una línea experimental separada compara MAPPO local con una variante que agrega observaciones y recompensa vecinales. En MAPPO y HAPPO, un crítico centralizado incorpora información conjunta durante el entrenamiento. La observación vecinal, la recompensa mixta y la centralización del crítico son mecanismos distintos cuyo uso depende del protocolo.

El objetivo general es desarrollar y evaluar procedimientos de control semafórico mediante aprendizaje por refuerzo multiagente. Se consideran MAPPO y HAPPO con entrenamiento centralizado y ejecución descentralizada, e IPPO e IDQN con aprendizaje independiente. La búsqueda organiza una fase amplia y otra refinada para estos cuatro algoritmos en las tres redes sintéticas. La extensión urbana requiere adaptar y verificar el procedimiento para la red, la demanda y los programas fijos de Mendoza. En las redes sintéticas también se compara una configuración de cooperación vecinal de MAPPO con su variante local mediante pares con semillas compartidas. La comprobación operacional de Mendoza utiliza los programas fijos y no evalúa políticas aprendidas.

La propuesta incorpora una codificación de la espera acumulada por carril mediante una transformación logarítmico-lineal en I niveles. La resolución no uniforme distingue con mayor detalle valores bajos de esa medida y acota los valores representados a partir de un umbral. La elección de cómo representar el tránsito influye en el problema de aprendizaje y cuenta con antecedentes en control semafórico [GR18]. En este trabajo, la codificación se utiliza en todos los agentes.

También se estudia una adaptación de HAPPO para intersecciones con distintas geometrías, fases disponibles y demandas. A diferencia de la actualización de actores utilizada en MAPPO, esta adaptación los actualiza secuencialmente y repondera las ventajas individuales según los cambios de política precedentes [KCW+22]. El crítico centralizado incorpora información conjunta durante el entrenamiento. La comparación con MAPPO, IPPO y IDQN examina el desempeño de procedimientos completos en esos escenarios, sin atribuir de antemano una ventaja a HAPPO ni aislar el efecto de su actualización secuencial.

1.1. Motivación

La motivación de este trabajo se centra en estudiar cómo coordinar agentes que aprenden en una misma red vial. El control adaptativo basado en aprendizaje por refuerzo permite ajustar decisiones a las condiciones observadas [MMLK25]. En una red de múltiples intersecciones, interesa examinar cómo esa adaptación puede incorporar los efectos de las decisiones locales sobre los cruces vecinos.

Las diferencias de demanda, geometría y fases disponibles entre intersecciones motivan, además, el estudio de políticas que puedan adaptarse a esas condiciones. La comparación de MAPPO y HAPPO con los esquemas independientes IPPO e IDQN permite examinar

distintas formas de aprendizaje y coordinación bajo una misma formulación del problema.

1.2. Organización del documento

Para dar respuesta a los objetivos planteados y estructurar el desarrollo de la investigación, el presente documento se organiza en cinco capítulos y un apéndice técnico.

Este capítulo contextualiza la congestión vehicular urbana y presenta el alcance y los objetivos del trabajo. El capítulo 2 desarrolla los fundamentos de ingeniería de tránsito, procesos de decisión de Markov, aprendizaje profundo y aprendizaje por refuerzo multiagente necesarios para seguir el diseño experimental.

El capítulo 3 describe los escenarios, la representación de observaciones, acciones y recompensas, y los protocolos de búsqueda y evaluación. Comprende las tres redes sintéticas y el escenario urbano de Mendoza, construido con conteos parciales, una demanda nominal de 220.000 viajes diarios y programas fijos. La comprobación operacional se distingue de la aplicación del aprendizaje urbano, que requiere trabajo adicional. El capítulo 4 organiza la evidencia de búsqueda, la comparación entre las variantes local y vecinal de MAPPO y la comprobación operacional de Mendoza, que no evalúa políticas aprendidas.

Finalmente, el capítulo 5 sintetiza las conclusiones principales a la luz de los objetivos propuestos, delimita el alcance y las limitaciones del estudio, y proyecta líneas de trabajo futuro. Como complemento técnico, el apéndice A describe el diseño de software del paquete `marlTLC` mediante diagramas de clases en lenguaje unificado de modelado (*Unified Modeling Language*, UML) y reúne los dominios, las semillas y las especificaciones necesarias para reproducir los experimentos.

2

Marco teórico

Este capítulo presenta los fundamentos teóricos del trabajo y vincula la ingeniería de tránsito con el aprendizaje por refuerzo para la toma de decisiones secuenciales en redes viales. Estos conceptos permiten modelar una red semafórica como un entorno estocástico y formular políticas de control adaptativo con agentes homogéneos y heterogéneos.

Primero se introducen los conceptos de ingeniería de tránsito y simulación (sección 2.1). Luego se desarrollan los fundamentos del aprendizaje por refuerzo y su integración con redes neuronales (sección 2.2), seguidos de su extensión al caso multiagente y de los algoritmos MAPPO y HAPPO (sección 2.3). Finalmente, se revisan los antecedentes recientes de control semafórico multiagente según sus esquemas de coordinación, representación del estado y tratamiento de la heterogeneidad (sección 2.3.7).

2.1. Ingeniería de tránsito y simulación

La ingeniería de tránsito es una rama de la ingeniería de transporte, tradicionalmente vinculada con la ingeniería civil. Se ocupa de la planificación, el diseño geométrico y la operación segura y eficiente del tránsito en calles, carreteras y redes viales, de acuerdo con manuales especializados como el *Traffic Engineering Handbook* [PW16] y con la literatura clásica del área [CyMR18]. En este trabajo, sus conceptos se abordan desde la perspectiva del modelado computacional y la gestión adaptativa. Permiten describir el funcionamiento físico de la red, caracterizar la oferta y la demanda de circulación e identificar las variables sobre las que puede intervenir el sistema de control.

Históricamente, la práctica de la ingeniería de tránsito puso un fuerte énfasis en aumentar la capacidad física vial mediante la construcción de nuevas obras o el ensanche de

calzadas [PW16]. Sin embargo, el crecimiento sostenido del parque automotor evidenció que la ampliación de la infraestructura no constituye una solución definitiva al problema de la congestión. En áreas urbanas consolidadas, además, modificar el trazado vial resulta costoso y, en algunos casos, difícil de realizar [CyMR18]. Por ello, la práctica contemporánea complementa la infraestructura física con estrategias de control dinámico del tránsito, priorizando la optimización de los recursos existentes mediante dispositivos de control semafórico.

La operación de una red vial puede analizarse a partir de la relación entre la demanda vehicular y la capacidad disponible. La demanda vehicular (D) corresponde a la cantidad de vehículos que requieren desplazarse por un determinado sistema vial en un lapso dado, mientras que la oferta vial o capacidad (c) representa la cantidad máxima de vehículos que pueden circular por dicho espacio físico bajo determinadas condiciones de operación [CyMR18]. Cuando la demanda se aproxima a la capacidad, el tránsito se vuelve inestable; cuando la supera ($D > c$), se presentan condiciones de sobresaturación o flujo forzado, caracterizadas por detenciones frecuentes y grandes demoras. En este sentido, la congestión no se entiende solo como una percepción general de mal funcionamiento, sino como un fenómeno con manifestaciones físicas medibles: demoras, colas y detenciones [FA11].

Estas magnitudes cumplen una función doble en el modelo computacional: pueden formar parte de las observaciones y de la señal de recompensa de los agentes, y también se utilizan para evaluar el desempeño del agente. Su definición permite vincular la descripción física del tránsito con la formulación posterior del problema de aprendizaje.

2.1.1. Terminología de tránsito

Antes de formular el problema de control, es necesario distinguir algunos términos que se utilizarán a lo largo del capítulo. En particular, se diferencian los movimientos que realizan los vehículos, las fases que habilitan esos movimientos y los intervalos durante los cuales se mantienen las indicaciones semafóricas [Fed08].

En este contexto, se emplearán las siguientes nociones:

- **Intersección (*intersection*), acceso (*approach*) y carril (*lane*):** una intersección es el área física y operacional en la que confluyen dos o más corrientes de tránsito. Cada una de las calzadas de aproximación que ingresan a ella constituye un acceso [PW16], organizándose internamente en uno o más carriles destinados a canalizar los flujos vehiculares [CyMR18].

- **Movimiento vehicular y movimientos en conflicto:** un movimiento vehicular representa la trayectoria específica que describe un vehículo al cruzar la intersección hacia una salida determinada, por ejemplo, un giro o un movimiento directo [W$^+$19]. Dos movimientos pueden generar puntos de conflicto cuando sus trayectorias se intersectan (véase la Figura 2.2). Cuando la geometría y la señalización no permiten

proteger su operación simultánea, esos movimientos deben programarse en intervalos distintos por razones de seguridad [FA11].

- **Fase (*phase*), ciclo (*cycle*) e intervalo (*interval*):** una fase es una configuración de señales que habilita simultáneamente un conjunto de movimientos compatibles [PW16]. El ciclo corresponde a la secuencia completa de fases necesaria para atender los movimientos de la intersección. Un intervalo es el período durante el cual las indicaciones semafóricas permanecen constantes [CyMR18].

La Figura 2.1 ilustra estos conceptos en una intersección de cuatro accesos. Los números del 1 al 8 identifican movimientos vehiculares individuales y las etiquetas $P2$, $P4$, $P6$ y $P8$ identifican cruces peatonales del esquema. Esta numeración sigue de manera esquemática una convención habitual de los manuales de temporización semafórica; no representa una configuración experimental ni una asignación obligatoria de fases para el caso de estudio. Las flechas, los accesos y las etiquetas de calles permiten distinguir trayectorias y sentidos, mientras que los cruces peatonales muestran otros usuarios que también deben considerarse al ordenar los movimientos. En este trabajo, la Figura se utiliza para separar la noción geométrica de movimiento de la regla semafórica que habilita un conjunto de movimientos compatibles [Fed08]. Por ejemplo, dos movimientos que no comparten un punto de conflicto pueden incluirse en una misma fase, mientras que los que sí lo comparten deben atenderse en fases compatibles o sucesivas.

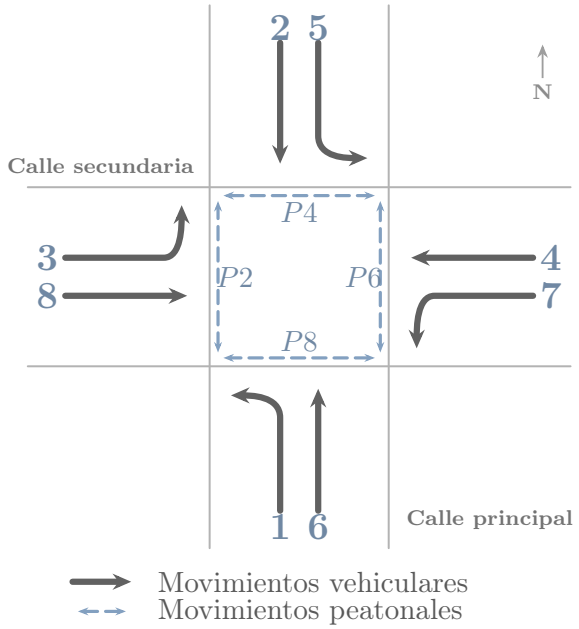


Figura 2.1: Convención esquemática de movimientos vehiculares, fases y cruces peatonales concurrentes en una intersección de cuatro accesos, adaptada conceptualmente del manual de temporización semafórica de Administración Federal de Carreteras (*Federal Highway Administration*, FHWA) [Fed08].

La Figura 2.2 presenta tres formas esquemáticas de relación entre trayectorias. Hay

cruce cuando dos trayectorias se intersectan en un punto; *convergencia* cuando trayectorias distintas se unen en una misma corriente; y *divergencia* cuando una corriente se separa en trayectorias diferentes. Los puntos coloreados marcan el lugar de interacción que se analiza en cada caso. Cada relación puede originar un punto de conflicto según la geometría y la simultaneidad de los movimientos: por eso, la incompatibilidad no depende solo de que las líneas se relacionen, sino de que su operación conjunta comprometa la seguridad. En el modelo semafórico, estos casos se utilizan para determinar qué movimientos pueden habilitarse en una misma fase y cuáles deben retenerse; la Figura muestra relaciones geométricas, no una programación temporal concreta.

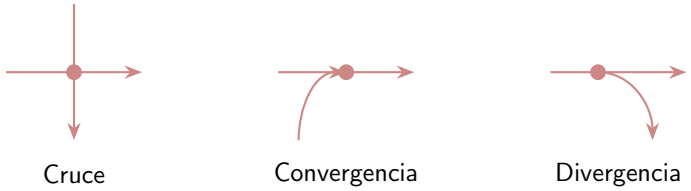


Figura 2.2: Representación esquemática de movimientos vehiculares y puntos de conflicto en una intersección semaforizada.

En arterias urbanas, el tránsito se desarrolla bajo condiciones de flujo interrumpido: los vehículos no avanzan únicamente por su interacción con otros vehículos, sino también por la presencia de intersecciones, señales y dispositivos de control de tránsito [PW16]. Las intersecciones son puntos críticos de la red porque allí confluyen distintos accesos y movimientos vehiculares de cruce, convergencia y divergencia que pueden resultar incompatibles entre sí, como resume la Figura 2.2. Por ello, su operación requiere ordenar estos movimientos y asignar la prioridad de paso de manera segura y eficiente.

El semáforo es uno de los dispositivos de control de tránsito utilizados para regular la circulación en intersecciones. Su función es asignar la prioridad de paso en forma alternada entre corrientes vehiculares incompatibles, evitando que distintos movimientos intenten ocupar simultáneamente el mismo espacio de cruce [FA11]. Desde el punto de vista del modelado algorítmico, esta capacidad de modificar la prioridad de paso convierte al semáforo en el actuador mediante el cual un sistema de control puede intervenir sobre la operación de la intersección y afectar, directa o indirectamente, las demoras y colas observadas en la red.

2.1.2. Fundamentos de la dinámica vehicular

Para modelar y optimizar la operación de una red vial, es necesario caracterizar físicamente el movimiento de los vehículos. La ingeniería de tránsito estudia este fenómeno a partir de las características fundamentales de la corriente de tránsito, que permiten describir las condiciones de operación de una vía o intersección mediante magnitudes observables como el flujo, la velocidad y la densidad [PW16]. Cal y Mayor distingue estas magnitudes entre variables principales, que describen el comportamiento agregado del flu-

jo vehicular, y variables asociadas, que expresan relaciones temporales y espaciales entre vehículos consecutivos [CyMR18].

A nivel macroscópico, la corriente de tránsito se representa mediante variables agregadas como la tasa de flujo, la densidad y los promedios de velocidad. A nivel microscópico, se analizan magnitudes propias de vehículos individuales e interacciones entre pares consecutivos, como la velocidad puntual, el intervalo y el espaciamiento [CyMR18]. Esta separación entre escalas es fundamental para el modelado computacional: las variables macroscópicas resumen el estado operacional de los accesos de la red, mientras que las variables microscópicas describen el comportamiento de cada vehículo dentro del simulador [FA11]. La correspondencia y las relaciones de conversión entre las variables de ambas escalas se resumen en la Tabla 2.1.

En el nivel macroscópico, las variables principales de la corriente de tránsito son [CyMR18]:

- **Flujo o tasa de flujo (q):** representa la cantidad de vehículos que atraviesan una sección transversal de la vía por unidad de tiempo. Si el período de observación es menor a una hora, se proyecta como una tasa horaria equivalente en vehículos por hora, calculada como $q = N/T$, donde N es el número de vehículos observados y T es la duración del intervalo de medición expresado en horas.

- **Densidad o concentración (k):** indica el número de vehículos que ocupan simultáneamente una longitud específica de vía, expresada en vehículos por kilómetro, calculada como $k = N/d$, donde d representa la longitud del tramo de vía analizado. En la práctica, se suele estimar indirectamente a través del porcentaje de ocupación temporal registrado por detectores inductivos [PW16].

- **Velocidad media espacial (v_s):** es la velocidad media de los vehículos que ocupan un tramo de vía en un instante determinado. Corresponde al promedio armónico de las velocidades puntuales de los vehículos individuales y representa la magnitud de velocidad consistente con la dinámica del flujo espacial.

- **Velocidad media temporal (v_t):** representa el promedio aritmético de las velocidades de los vehículos que cruzan una sección transversal fija de la vía durante un período de tiempo determinado.

En el nivel microscópico, las variables asociadas son [CyMR18]:

- **Velocidad puntual (v_i):** es la velocidad instantánea a la que un vehículo individual (i) atraviesa una sección transversal específica de la vía.

- **Intervalo temporal (h):** es el tiempo transcurrido entre el paso de dos vehículos consecutivos por una sección fija de la calzada, medido respecto a puntos homólogos,

por ejemplo, los paragolpes delanteros. El intervalo promedio en segundos (h_{prom}) se relaciona con la tasa de flujo como $h_{\text{prom}} = 3600/q$.

- **Espaciamiento simple (s):** es la distancia física entre puntos homólogos de dos vehículos consecutivos en un instante de tiempo. El espaciamiento promedio en metros (s_{prom}) se relaciona de forma inversa con la densidad de la corriente de tránsito como $s_{\text{prom}} = 1000/k$.

La Figura 2.3 complementa estas definiciones con las magnitudes que se miden en el espacio y en el tiempo. El largo del vehículo l y la separación libre e son distancias, expresadas en metros; el espaciamiento s es la distancia entre puntos homólogos de vehículos consecutivos, también en metros. El avance temporal β es el tiempo asociado al recorrido del largo del vehículo a la velocidad v, la brecha temporal g es el tiempo entre el paso del vehículo precedente y el siguiente respecto de puntos de referencia distintos, y el intervalo temporal h es el tiempo entre pasos de puntos homólogos; β, g y h se expresan en segundos. Las cotas inferiores representan relaciones espaciales y las superiores relaciones temporales; no son dos vehículos distintos ni dos mediciones independientes. En este trabajo, estas magnitudes sirven para vincular la descripción individual de los vehículos con las variables agregadas del flujo.

Tabla 2.1: Comparación y correspondencia de variables de tránsito en escalas macroscópica y microscópica.

Escala	**Variable Principal**	**Relación / Conversión**
Macroscópica	Flujo (q, vehículos por hora)	$q = 3600/h_{\text{prom}}$
	Densidad (k, vehículos por kilómetro)	$k = 1000/s_{\text{prom}}$
	Velocidad media espacial (v_s, kilómetros por hora)	$q = k \cdot v_s$
	Velocidad media temporal (v_t, kilómetros por hora)	$v_t = v_s + \frac{\sigma_s^2}{v_s}$ (Relación de Wardrop)
Microscópica	Velocidad puntual (v_i, kilómetros por hora)	Variable individual de cada vehículo
	Intervalo temporal (h, segundos)	Tiempo entre pasos de vehículos sucesivos
	Espaciamiento simple (s, metros)	Distancia espacial entre paragolpes homólogos

La Figura 2.3 presenta de manera conjunta estas distancias y tiempos entre vehículos, y permite interpretar cómo las magnitudes microscópicas describen la separación y el avance de una corriente.

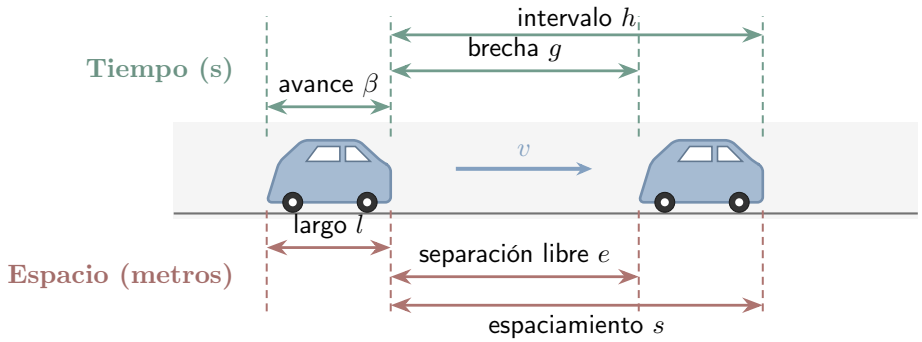


Figura 2.3: Relaciones temporales y espaciales entre vehículos consecutivos de una corriente de tránsito: intervalo (h) y espaciamiento (s).

Estas variables macroscópicas se relacionan de manera directa a través de la ecuación fundamental del flujo de tránsito:

$$q = k \cdot v_s \tag{2.1}$$

donde q es la tasa de flujo, k es la densidad y v_s es la velocidad media espacial. La ecuación (2.1) utiliza la velocidad media espacial, ya que relaciona el flujo y la cantidad de vehículos presentes en un tramo. La velocidad media temporal (v_t) es la media aritmética de las velocidades observadas en un punto durante un intervalo, mientras que v_s, asociada al promedio de las velocidades de una muestra espacial tomada sobre un tramo, se estima en la práctica como media armónica de las velocidades puntuales medidas en una sección durante un intervalo. Cuando existe variabilidad entre las velocidades, ambas magnitudes pueden diferir. La relación entre ellas se expresa mediante la relación de Wardrop [FA11]:

$$v_t = v_s + \frac{\sigma_s^2}{v_s} \tag{2.2}$$

donde σ_s^2 representa la varianza de la distribución de las velocidades de los vehículos en el espacio. La ecuación (2.2) muestra que, dado que la varianza es no negativa ($\sigma_s^2 \geq 0$), se cumple $v_t \geq v_s$, con igualdad únicamente cuando todos los vehículos circulan a la misma velocidad. En una intersección semaforizada, una indicación roja puede reducir localmente la velocidad media y aumentar la acumulación de vehículos en los accesos. Estos efectos se manifiestan en la formación de colas y en el incremento de las demoras [CyMR18].

Métricas de ineficiencia operativa

La interrupción del flujo causada por semáforos o saturación genera efectos adversos que los sistemas de control buscan mitigar. Siguiendo la terminología de la ingeniería de tránsito, las principales manifestaciones medibles de esta ineficiencia son las demoras, las colas y las detenciones [FA11]:

- **Demoras:** representan el tiempo adicional que un vehículo debe invertir para atravesar una intersección respecto del tiempo que le tomaría circular a velocidad de flujo libre. En intersecciones semaforizadas, una parte de esa demora aparece por la espera normal durante la luz roja. Otras demoras se producen cuando las llegadas de vehículos varían, cuando la demanda se acerca o supera la capacidad de la intersección, o cuando quedan colas sin disiparse al finalizar el verde, de modo que algunos vehículos deben esperar más de un ciclo para cruzar.

- **Colas:** corresponden al número de vehículos acumulados detrás de una línea de detención a la espera de traspasarla. Esta medida espacial es especialmente relevante porque, cuando la cola supera la longitud del arco de aproximación, puede bloquear la intersección precedente y propagar la congestión al resto de la red.

- **Detenciones:** cuantifican la cantidad de veces que un vehículo debe reducir su velocidad hasta cero durante el recorrido. Una alta frecuencia de detenciones deteriora la continuidad operativa del flujo y suele asociarse con mayores procesos de aceleración y desaceleración.

Estas magnitudes permiten describir el desempeño operacional de una intersección y de la red. En la formulación computacional, también pueden utilizarse para construir observaciones, señales de recompensa o criterios de evaluación del agente.

Descripción probabilística del flujo y agrupamientos vehiculares

En el modelado de tránsito, las llegadas vehiculares no siempre pueden representarse adecuadamente como una secuencia determinista y constante. Para flujos vehiculares bajos y medios, y cuando las llegadas no están fuertemente condicionadas por semáforos previos, es habitual aproximar el número de vehículos que llega a una sección durante un intervalo de tiempo mediante un proceso de Poisson [CyMR18].

Esta aproximación supone independencia estadística entre llegadas y permite describir la variabilidad observada en la demanda de acceso a una intersección [CyMR18]. La probabilidad discreta $P(X)$ de que lleguen exactamente X vehículos en un intervalo de tiempo se define como:

$$P(X) = \frac{m^X \cdot e^{-m}}{X!} \tag{2.3}$$

donde $X \in \mathbb{N}_0$ es la variable aleatoria discreta que representa el número de arribos vehiculares, $m > 0$ representa el valor esperado o media de llegadas en dicho intervalo, y e es la base del logaritmo natural. La ecuación (2.3) formaliza esta probabilidad discreta. Bajo el mismo supuesto, los intervalos de tiempo continuos (h) medidos entre vehículos sucesivos se describen mediante una distribución de probabilidad exponencial negativa [CyMR18]:

$$P(h \geq t) = e^{-\frac{q}{3600} \cdot t} \tag{2.4}$$

donde h es el intervalo temporal en segundos, q es el flujo en vehículos por hora, y t es el umbral de tiempo analizado. La ecuación (2.4) expresa la probabilidad de que el intervalo supere dicho umbral.

Esta representación probabilística supone independencia entre arribos sucesivos y resulta adecuada para flujos bajos o moderados en condiciones de flujo libre, sin influencia de semáforos cercanos en intersecciones precedentes [CyMR18]. El supuesto pierde validez en condiciones de congestión o en redes urbanas semaforizadas. En estos casos, los semáforos interrumpen el flujo agrupando los vehículos, lo que introduce dependencia espacio-temporal entre arribos consecutivos. La Figura 2.4 compara conceptualmente los intervalos temporales resultantes bajo ambas condiciones. En ella, $h_{\text{mín}}$ representa un umbral mínimo conceptual para el intervalo mostrado en la curva con agrupamientos vehiculares; no es una estimación obtenida de datos de este trabajo. Las curvas y ese umbral tienen, por lo tanto, función ilustrativa y no constituyen una distribución empírica particular.

En este trabajo, la limitación de esos modelos analíticos motiva el uso de simulación microscópica para analizar la operación semafórica bajo condiciones variables. En el simulador *Simulation of Urban Mobility* (SUMO), la formación, dispersión e interacción de los agrupamientos vehiculares emergen de la dinámica simulada de los vehículos y de sus cambios de carril. Los procesos aleatorios del simulador se inicializan con una semilla (*seed*), un número que permite repetir su secuencia pseudoaleatoria bajo la misma configuración y versión. Cambiar la semilla permite estudiar variaciones de la simulación. Repetirla no hace iguales los resultados si también cambia el agente o sus decisiones.

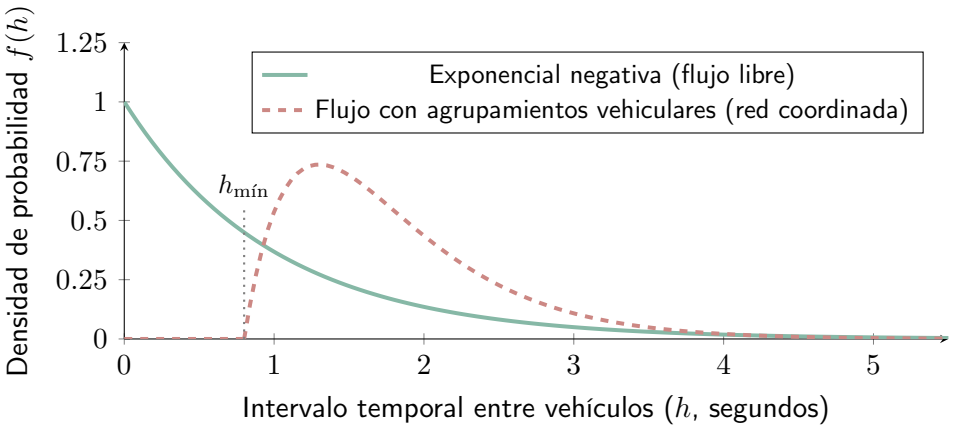


Figura 2.4: Comparación esquemática de distribuciones de intervalos temporales entre vehículos en flujo libre y en una corriente con formación de agrupamientos vehiculares. Las curvas representan situaciones conceptuales y no una distribución empírica particular.

La Figura 2.4 se interpreta comparando la forma de ambas curvas. El eje horizontal representa el intervalo h entre vehículos y el eje vertical la densidad de probabilidad $f(h)$, es decir, la concentración relativa de intervalos alrededor de cada valor. En el modelo exponencial, la densidad disminuye desde el origen y asigna mayor frecuencia relativa

a los intervalos breves. En la representación con agrupamientos vehiculares, la densidad comienza después de $h_{\text{mín}}$ y alcanza un máximo alejado del origen, lo que ilustra que el arribo de un vehículo queda condicionado por la separación y la coordinación con los que lo preceden. La comparación permite reconocer por qué el supuesto de arribos independientes puede resultar insuficiente en una red semaforizada.

2.1.3. Sistemas de control de tránsito

Para ordenar los movimientos en conflicto y regular el paso en las intersecciones urbanas se utilizan sistemas de control de tránsito [PW16]. El semáforo asigna el derecho de paso de manera alternada mediante indicaciones luminosas rojas, amarillas y verdes [CyMR18].

En el modelo computacional, el semáforo actúa como el mecanismo mediante el cual el controlador interviene sobre la red. La selección de fases, la duración del verde efectivo y la coordinación entre intersecciones modifican la prioridad de paso y afectan la evolución de las colas y las demoras.

Sobre esta base terminológica, la programación temporal de un semáforo se describe mediante la duración de las fases, la longitud de ciclo y la definición de los intervalos de transición y despeje. Estos parámetros determinan cuánto tiempo recibe prioridad cada conjunto de movimientos compatibles y condicionan directamente la capacidad de descarga, las demoras y la seguridad operacional de la intersección [PW16].

En relación con los giros izquierdos, los manuales de temporización distinguen entre operación permisiva, protegida y protegida-permisiva. En una fase permisiva, el giro se ejecuta aprovechando brechas en la corriente opuesta; en una fase protegida, el giro recibe una indicación exclusiva y los movimientos conflictivos permanecen detenidos; en una fase protegida-permisiva, ambas condiciones se combinan durante distintos intervalos del ciclo [Fed08]. En este análisis, la distinción resulta útil porque muestra que una fase no es solo un color del semáforo, sino una regla operacional que habilita ciertos movimientos, impone cesión de paso en otros y restringe los movimientos en conflicto.

Para garantizar la seguridad vial, las transiciones entre fases no pueden ejecutarse de manera instantánea, sino que deben respetar intervalos de cambio y despeje ($t_{\text{amarillo}} + t_{\text{rojo_todo}}$) [PW16]. En esta investigación, esos tiempos se consideran restricciones operativas de seguridad previamente definidas en la configuración del simulador y del sistema semafórico. En consecuencia, su determinación no forma parte del proceso de decisión del agente, cuyo control se limita a la selección o extensión de fases dentro de un esquema temporal compatible con dichas restricciones.

Modelado de la capacidad y el verde efectivo

Desde el punto de vista operativo, la capacidad de descarga de un acceso semaforizado depende del tiempo efectivo de servicio que recibe cada fase y de la tasa con la que la fila puede disiparse cuando el movimiento tiene prioridad [FA11]. En la Figura 2.5, $q_{\text{máx}}$

representa la tasa de descarga de saturación, g el intervalo de verde efectivo, y l_1 y l_2 las pérdidas inicial y final de servicio, respectivamente; el tiempo se expresa en segundos y la tasa de flujo en vehículos por hora. La barra inferior distingue los intervalos de rojo, verde y amarillo, mientras que la curva superior muestra cómo la descarga alcanza y abandona el régimen de saturación. La descarga observada no depende solo de la duración nominal del verde, sino también de pérdidas de arranque, transiciones y otras restricciones temporales del control. La Figura es conceptual: estos parámetros no se estiman en este trabajo. En su lugar, los efectos se representan mediante la dinámica microscópica del simulador y la configuración temporal del sistema semafórico, ya que esas interacciones determinan las colas, demoras y detenciones que el controlador busca mejorar.

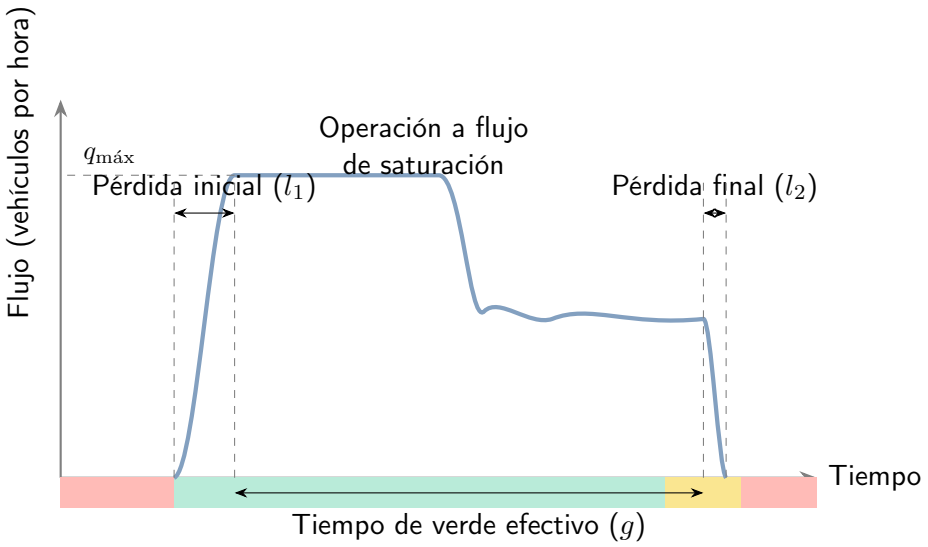


Figura 2.5: Distribución temporal de la tasa de descarga y pérdidas asociadas durante un intervalo de verde en una aproximación semaforizada.

La Figura 2.5 debe leerse como una relación temporal: las pérdidas l_1 y l_2 reducen el intervalo en que la fase puede descargar vehículos a la tasa $q_{\text{máx}}$. Por tanto, el verde efectivo g no equivale necesariamente al verde nominal programado en el controlador.

Clasificación de los controladores de semáforos

De acuerdo con su mecanismo operativo y su grado de respuesta a la demanda vehicular, los controladores de semáforos se clasifican clásicamente en control de tiempo fijo y control accionado por el tránsito [PW16]. En el primero, los planes de señales se repiten de manera preprogramada a partir de datos históricos y no cambian en respuesta a la demanda instantánea. En el segundo, la intersección utiliza detectores en los accesos para variar dinámicamente el inicio o la duración de las fases según la demanda observada en tiempo real.

Incluso en los esquemas accionados, la operación permanece sujeta a parámetros límite de seguridad y funcionamiento, como el verde mínimo ($g_{\text{mín}}$) y el verde máximo ($g_{\text{máx}}$)

[PW16]. Estos parámetros restringen cuánto puede sostenerse o finalizarse una fase y, en este trabajo, forman parte de las condiciones operativas dentro de las cuales el controlador toma decisiones.

2.1.4. Simulación de tránsito

Debido a la densidad y complejidad de las redes urbanas, la sincronización y optimización de los tiempos semafóricos requiere representar la evolución del tránsito bajo condiciones variables. Los modelos analíticos y agregados resultan útiles bajo determinados supuestos, pero pueden ser limitados para describir la formación de colas y las interacciones locales durante la operación dinámica de una red [PW16]. De acuerdo con la taxonomía clásica [FA11], los modelos de tránsito pueden organizarse en tres niveles de resolución. Los modelos macroscópicos describen el tránsito de manera agregada mediante variables como flujo, densidad y velocidad; los mesoscópicos representan entidades intermedias, como agrupamientos vehiculares; y los microscópicos modelan individualmente los vehículos, sus rutas y su evolución dentro de la red. Para esta investigación resulta especialmente relevante este último nivel, pues permite examinar cómo las decisiones semafóricas inciden en la formación de colas, las detenciones y las interacciones locales entre vehículos.

La simulación de tránsito constituye una representación abstracta del fenómeno real. Su objetivo no es reproducir visualmente una escena particular, sino modelar los procesos que determinan la evolución de la circulación mediante reglas de comportamiento y variables observables. La calidad de los resultados depende de la representación de la red, la demanda, la flota, los modelos de comportamiento y las reglas de control. Por ello, la simulación debe entenderse como un entorno experimental sujeto a supuestos y parámetros, cuyos resultados requieren una evaluación operacional acorde con el objetivo del estudio. La correspondencia entre el fenómeno y su representación se resume en la Figura 2.6: el tránsito real constituye la referencia y el simulador representa sus vehículos, red y reglas dinámicas para permitir experimentos controlados.

La Figura 2.6 muestra, de forma sintética, la correspondencia entre una escena vial y su representación microscópica en SUMO. Su función es ilustrar cómo los vehículos y la infraestructura se transforman en entidades computacionales; no constituye una validación del simulador ni una representación del escenario utilizado en este trabajo.

SUMO pertenece a la categoría microscópica descrita por Fernández [FA11]. Es una *suite* libre y de código abierto iniciada en el Instituto de Sistemas de Transporte del Centro Aeroespacial Alemán (*Deutsches Zentrum für Luft- und Raumfahrt*, DLR), que permite modelar sistemas de tránsito intermodales, incluidos vehículos, transporte público y peatones. También ofrece herramientas para importar redes, calcular rutas, visualizar y evaluar simulaciones, además de interfaces para controlar su ejecución [DLR26].

En SUMO, la evolución vehicular se actualiza en pasos temporales discretos sobre una



Figura 2.6: Correspondencia entre la circulación real y su representación microscópica en SUMO. Fuente: DLR – Institute of Transportation Systems, *SUMO User Documentation* [DLR26].

representación espacial continua. El entorno permite modelar calles con múltiples carriles, cambios de carril, distintos tipos de vehículos, rutas individuales, semáforos y salidas de simulación a nivel de vehículo, carril, arco o detector [DLR26].

La interfaz gráfica del programa `sumo-gui` permite inspeccionar visualmente la red, los vehículos y algunos valores de la simulación durante la ejecución. Esta visualización cumple una función de apoyo para la inspección del escenario y la depuración del modelo; las métricas utilizadas para la evaluación se obtienen de las salidas de simulación y de las interfaces de control [DLR26]. La interfaz y su función de inspección se muestran en la Figura 2.7; la interfaz gráfica de usuario (*Graphical User Interface*, GUI) no constituye la fuente de las métricas ni reemplaza su cálculo operacional.

La Figura 2.7 reúne cuatro zonas de inspección. A la izquierda se observa la red completa, útil para localizar la parte del escenario que se está examinando. A la derecha se muestra una vista ampliada de la calzada y de un vehículo seleccionado, mientras que el gráfico superpuesto en la parte inferior representa la evolución temporal de su velocidad, expresada en metros por segundo. La franja inferior contiene el registro de mensajes y advertencias producidos durante la ejecución, y la barra superior agrupa los controles de avance, pausa, tiempo y escala de visualización. En conjunto, estos elementos permiten verificar visualmente la presencia de vehículos, su movimiento y posibles inconsistencias de configuración. En este trabajo la interfaz se usa con fines de inspección y depuración; la evaluación cuantitativa se realiza con las salidas de simulación y las interfaces de control [DLR26].

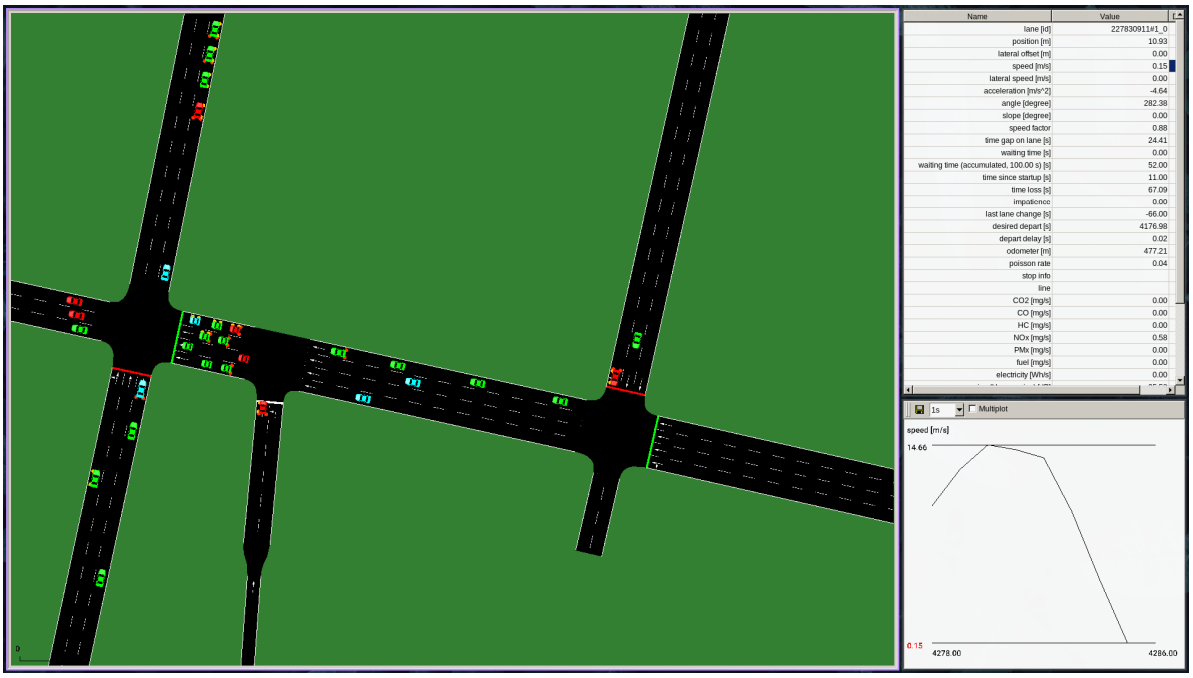


Figura 2.7: Interfaz del programa `sumo-gui` durante una simulación microscópica. El panel izquierdo presenta una vista general de la red; el derecho muestra información sobre un vehículo particular; el recuadro inferior grafica su velocidad y el superior muestra los valores de sus variables relacionadas. Fuente: DLR – Institute of Transportation Systems, *SUMO User Documentation* [DLR26].

Esta granularidad vuelve a la simulación microscópica especialmente adecuada para estudiar control semafórico mediante aprendizaje por refuerzo. Las decisiones sobre fases y tiempos de servicio pueden evaluarse mediante sus consecuencias sobre la dinámica simulada: colas, tiempos de espera, detenciones, ocupación de carriles, completitud de viajes y propagación de congestión. En la formulación computacional, las variables de tránsito constituyen observaciones, las fases admisibles constituyen acciones y la evolución simulada constituye la transición del entorno. La definición operacional de estas variables y su procedimiento de cálculo se presentan en la metodología.

Representación microscópica de intersecciones en SUMO

La naturaleza microscópica de la simulación no se reduce a representar visualmente vehículos. Cada unidad modifica su velocidad y posición de acuerdo con modelos de seguimiento vehicular y cambio de carril, los parámetros de su tipo de vehículo y las restricciones de la red. De este modo, variables como la aceleración, la separación entre vehículos, la velocidad deseada y la disponibilidad de carriles intervienen en la formación y dispersión de colas. Los valores concretos utilizados para la flota y para estos parámetros pertenecen a la configuración experimental y se detallan en la metodología [DLR26].

En este contexto, el simulador de tránsito constituye el entorno operativo sobre el cual se formaliza el problema de control semafórico. La Figura 2.8 esquematiza la representación microscópica de una intersección con circulación por la derecha [DLR26], en la que

se distinguen carriles, líneas de detención y conexiones:

- **Carriles entrantes (L_{in}) y salientes (L_{out}):** los carriles entrantes conducen el flujo vehicular hacia la intersección y los salientes reciben la descarga tras cruzar el nodo, indicados mediante flechas que señalan el sentido de circulación.
- **Línea de detención (*stop line*):** líneas blancas continuas próximas al final de cada carril entrante, delante de las cuales se detienen los vehículos cuando la fase no habilita su movimiento.
- **Conexiones (*links*):** relaciones que conectan un carril entrante con uno saliente a través del área de cruce. En SUMO, cada enlace controlado se asocia con un índice y con un estado de señal (G para verde prioritario, g para verde con cesión de paso, y para amarillo y r para rojo). En la fase conceptual ilustrada, las flechas verdes corresponden a los movimientos directos opuestos habilitados del eje horizontal, mientras que las flechas rojas señalan las conexiones retenidas.

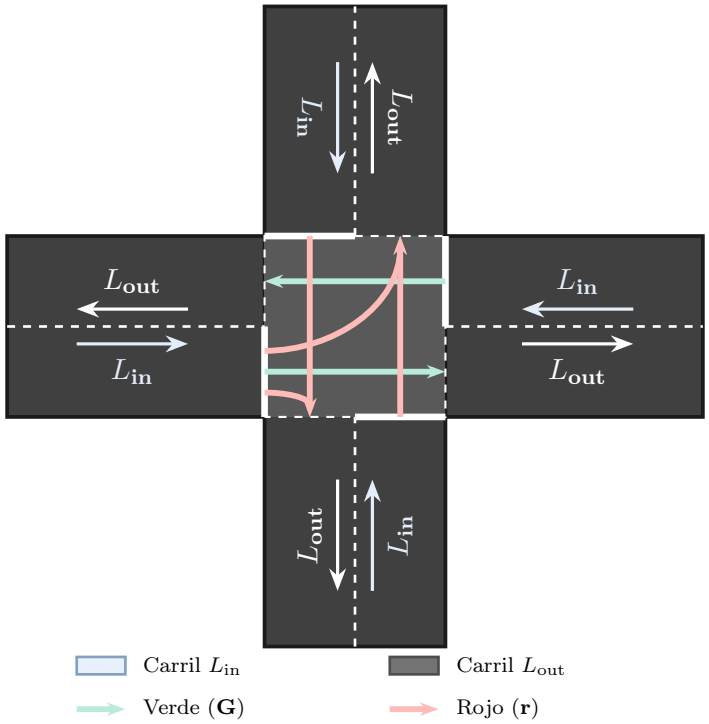


Figura 2.8: Esquema conceptual de carriles, línea de detención y conexiones de una intersección semaforizada bajo circulación por la derecha. La fase representada habilita movimientos directos opuestos del eje horizontal y retiene los movimientos conflictivos.

Por consiguiente, una fase no modifica la geometría ni el sentido de los carriles, sino que determina qué conexiones pueden utilizarse durante cada intervalo de control. Esta representación resulta directamente compatible con la formulación algorítmica del problema, donde una acción válida debe seleccionar una fase compatible con la geometría

de la intersección y respetar las transiciones y restricciones de seguridad definidas por el entorno.

Componentes y configuración de SUMO

La *suite* SUMO reúne aplicaciones para construir redes, generar demandas, ejecutar simulaciones, visualizar sus resultados y controlar la ejecución mediante interfaces externas [DLR26]:

- **netconvert:** aplicación que importa redes desde distintas fuentes y compila su representación en el formato nativo de SUMO. También permite reconstruir o ajustar elementos de la red, como las conexiones y los programas semafóricos.
- **netedit:** editor gráfico para crear y modificar de forma interactiva nodos, calles, carriles, conexiones, restricciones de giro y elementos de control semafórico.
- **osmWebWizard:** herramienta que automatiza la generación de un escenario a partir de un área seleccionada de OpenStreetMap (OSM). Se considera una alternativa de importación y no reemplaza el procesamiento específico realizado con `netconvert`.
- **interfaz de control del tránsito (*Traffic Control Interface*, TraCI):** interfaz que permite consultar el estado de una simulación en ejecución y modificarlo durante el avance paso a paso, incluyendo variables de vehículos, carriles, detectores y semáforos [DLR26].
- **libsumo:** implementación embebida que ofrece la interfaz de programación de SUMO sin la comunicación mediante conexiones de red (*sockets*) de la modalidad TraCI estándar. Esta alternativa resulta útil para integrar el simulador con controladores externos y reducir la sobrecarga de comunicación.

Desde la perspectiva del modelado computacional, una simulación de SUMO se construye a partir de archivos lenguaje de marcado extensible (*Extensible Markup Language*, XML) de red, demanda, elementos adicionales y configuración [DLR26]. Los archivos de zonas de análisis de tránsito (*Traffic Analysis Zone*, TAZ) pueden utilizarse durante la estructuración espacial y la generación de demanda, mientras que las salidas de la simulación se solicitan mediante opciones de configuración específicas. Sus funciones generales se resumen en la Tabla 2.2:

Métricas de evaluación operacional

La simulación produce medidas para evaluar el control semafórico. En este trabajo se consideran las siguientes categorías de indicadores:

- **Demora y tiempo de espera:** cuantifican el tiempo adicional o el tiempo detenido que experimentan los vehículos durante su recorrido. Pueden analizarse de forma

Tabla 2.2: Archivos de entrada y configuración empleados en una simulación de SUMO.

Archivo	**Extensión**	**Función en el Modelo Computacional**
Red Vial	`.net.xml`	Topología física, carriles, conexiones y lógicas semafóricas.
Rutas y Demanda	`.rou.xml`	Tipos de vehículos, viajes, flujos e itinerarios.
Zonas TAZ	`.taz.xml`	Zonas para estructurar espacialmente la demanda.
Elementos Adicionales	`.add.xml`	Detectores, tipos de vehículo y programas auxiliares.
Configuración	`.sumocfg`	Archivos de entrada, opciones de ejecución y salidas.

agregada en la red y desagregarse por intersección, acceso o agente, siempre que se mantenga una definición operacional consistente [CyMR18, DLR26].

- **Completitud y flujo servido:** la cantidad de vehículos que completa su itinerario y su relación con la demanda generada permiten distinguir una reducción genuina de la demora de una disminución del flujo atendido.
- **Equidad espacial:** la distribución de las demoras entre intersecciones permite identificar si una política mejora el desempeño global a costa de concentrar la espera en accesos o agentes particulares.
- **Indicadores de validez de la simulación:** el mecanismo de teletransporte de SUMO, denominado originalmente *teleport*, retira temporalmente vehículos bloqueados y los reinserta para evitar que la simulación se detenga. Sus activaciones, los vehículos pendientes y otras incidencias permiten detectar bloqueos o condiciones que comprometen la comparabilidad de una ejecución.

Las definiciones operacionales, fórmulas, unidades, ventanas de evaluación y procedimientos de agregación empleados en este trabajo se presentan en la metodología. Las recompensas y las pérdidas de optimización corresponden al entrenamiento de los algoritmos de aprendizaje y se desarrollan en las secciones dedicadas a RL y MARL.

En este marco, SUMO permite observar el estado de la red y modificar las fases semafóricas durante la ejecución mediante interfaces de control. Esta capacidad convierte el control semafórico en un problema de decisión secuencial, lo que motiva la introducción del aprendizaje por refuerzo en la siguiente sección [DLR26].

2.2. Aprendizaje por refuerzo (RL)

El aprendizaje automático (*Machine Learning*, ML) suele dividirse en tres paradigmas. En el aprendizaje supervisado (*Supervised Learning*, SL), el modelo se entrena con ejemplos etiquetados que indican la salida esperada. En el aprendizaje no supervisado (*Unsupervised Learning*, UL), se busca descubrir estructura en datos no etiquetados, por

ejemplo mediante agrupamiento o reducción de dimensionalidad. El aprendizaje por refuerzo (RL) aborda un problema distinto: una entidad aprende mediante interacción con un entorno qué acciones elegir para maximizar una recompensa acumulada [SB18].

Esta diferencia es central. En RL no existe, en general, un supervisor que indique cuál era la acción correcta en cada situación. El agente aprende por experiencia, ensayando acciones, observando sus consecuencias y ajustando su comportamiento a partir de las recompensas obtenidas. Además, las consecuencias relevantes de una acción no siempre se manifiestan de forma inmediata: una decisión puede producir una recompensa baja en el corto plazo, pero conducir a estados favorables en el futuro. Por este motivo, el aprendizaje por refuerzo se ocupa explícitamente del problema de la recompensa diferida y de la asignación de crédito a decisiones pasadas [SB18].

La formulación básica distingue entre el agente (*agent*), que aprende y toma decisiones, y el entorno (*environment*), que representa todo aquello externo al agente con lo cual este interactúa. En cada paso, el agente observa una situación, selecciona una acción y recibe una nueva observación junto con una recompensa: este ciclo constituye la interacción agente–entorno. El comportamiento está determinado por una política (*policy*), denotada por π, que especifica cómo elegir acciones a partir de los estados u observaciones disponibles. La señal de recompensa (*reward signal*) define el objetivo inmediato, mientras que la función de valor estima la conveniencia de estados o acciones en términos de recompensa futura. Sutton y Barto también distinguen el modelo del entorno, entendido como una descripción de sus transiciones y recompensas; cuando no se conoce o no se utiliza explícitamente, se habla de métodos libres de modelo (*model-free*).

En este trabajo, se utiliza controlador para el sistema general que regula las señales y agente para la entidad que aprende y selecciona acciones. Cada agente regula una intersección y la red vial simulada en SUMO constituye su entorno. La selección de una fase constituye la acción, mientras que la evolución posterior del tránsito produce nuevas observaciones y una señal de recompensa que orienta el aprendizaje [DLR26].

Para fijar ideas, considérese un par de intersecciones vecinas. El agente observa localmente las colas y demoras de su intersección, selecciona una fase y modifica la corriente de vehículos que llegará a la siguiente. Esa consecuencia aguas abajo puede cambiar sus arribos y colas, y se refleja en una recompensa asociada al desempeño de la red. El retorno permite acumular esos efectos a lo largo de varias decisiones, en lugar de evaluar una fase solo por su consecuencia inmediata.

Un aspecto característico de RL es el dilema entre exploración y explotación. El agente debe explotar acciones que, según su experiencia, producen buenos resultados, pero también debe explorar alternativas menos conocidas que podrían conducir a políticas superiores. Si explora demasiado, desperdicia oportunidades de obtener recompensa; si explota demasiado pronto, puede converger a comportamientos subóptimos. Este equilibrio es especialmente relevante en problemas de control, donde las acciones modifican la

experiencia futura disponible para el aprendizaje [SB18].

2.2.1. Procesos de decisión de Markov (MDP)

El marco matemático estándar para estudiar problemas de aprendizaje por refuerzo de un solo agente es el proceso de decisión de Markov (*Markov Decision Process*, MDP). En un MDP finito, el agente y el entorno interactúan en pasos temporales discretos $t = 0, 1, 2, \ldots$: en cada instante, el agente recibe $S_t \in \mathcal{S}$, selecciona $A_t \in \mathcal{A}(S_t)$ y el entorno emite $R_{t+1} \in \mathcal{R}$ y transita a S_{t+1} [LML+25]. La notación se ilustra en la Figura 2.9.

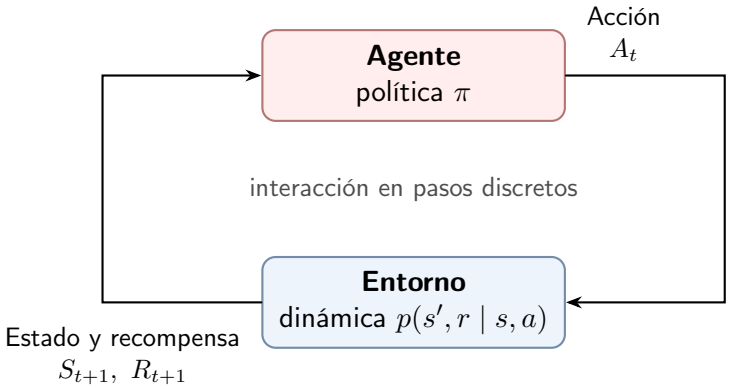


Figura 2.9: Interacción agente-entorno en un proceso de decisión de Markov [SB18].

De forma compacta, un MDP finito queda caracterizado por la tupla $\langle \mathcal{S}, \mathcal{A}, P, R, \gamma \rangle$, donde $\mathcal{S}$ es el espacio de estados, $\mathcal{A}$ el espacio de acciones, P la dinámica de transición probabilística, R la función de recompensa y $\gamma \in [0, 1)$ el factor de descuento.

La dinámica del entorno queda definida por la función $p(s', r \mid s, a)$, que especifica la probabilidad condicional conjunta de observar el próximo estado s' y la recompensa r al ejecutar la acción a en el estado s:

$$p(s', r \mid s, a) \doteq \Pr\{S_{t+1} = s', R_{t+1} = r \mid S_t = s, A_t = a\} \tag{2.5}$$

En esta formulación, $S_t \in \mathcal{S}$ y $A_t \in \mathcal{A}$ denotan el estado y la acción en el instante t, mientras que $S_{t+1} \in \mathcal{S}$ y $R_{t+1} \in \mathbb{R}$ representan el estado sucesor y la recompensa escalar devueltos por el entorno. Si bien conceptualmente el estado comprende toda la información necesaria para determinar la dinámica, en el control semafórico el agente suele disponer de observaciones locales de tránsito que constituyen una representación parcial de dicho estado.

El proceso satisface la propiedad de Markov si la distribución condicional del próximo estado y recompensa depende exclusivamente del estado y la acción presentes, resultando condicionalmente independiente del historial de transiciones previas [SB18].

Una suposición habitual del MDP es que el agente observa un estado suficientemente informativo del entorno. Sin embargo, en muchas aplicaciones reales el agente solo recibe observaciones parciales o ruidosas. En estos casos, el marco se generaliza a un proceso

de decisión de Markov parcialmente observable (*Partially Observable Markov Decision Process*, POMDP), donde el historial de observaciones puede ser necesario para inferir el estado subyacente [ACS24].

Para formalizar la meta de maximizar la recompensa a largo plazo se define el retorno descontado G_t, que incorpora un factor $\gamma \in [0, 1)$ para ponderar la importancia de las recompensas futuras [SB18]:

$$G_t \doteq R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \tag{2.6}$$

En esta formulación, el retorno G_t acumula las recompensas futuras ponderadas por el factor de descuento $\gamma \in [0, 1)$. Un valor de γ cercano a cero privilegia la optimización a muy corto plazo, mientras que valores próximos a uno confieren mayor peso a las consecuencias remotas [ACS24]. En el control semafórico, esta acumulación temporal resulta indispensable para que el agente aprenda a prevenir condiciones de bloqueo o saturación en ciclos futuros, en lugar de limitarse a disipar la cola inmediata a expensas de la red.

2.2.2. Métodos basados en valor y basados en política

Los algoritmos de RL pueden agruparse, a grandes rasgos, en métodos basados en valor y métodos basados en política. Esta distinción no agota la taxonomía del campo, pero permite introducir las familias de algoritmos más relevantes para este trabajo.

Esta clasificación distingue dos formas de obtener una regla de decisión. Los métodos basados en política aprenden directamente una función que relaciona cada estado con una acción o con una distribución de probabilidad sobre las acciones. En cambio, los métodos basados en valor aprenden indirectamente la regla de decisión: estiman la conveniencia de los estados o de los pares estado–acción y, a partir de esas estimaciones, determinan qué alternativa conviene ejecutar [SB18]. En el control semafórico, las alternativas corresponden a las distintas fases admisibles de una intersección.

La Figura 2.10 representa esta segunda alternativa. A partir del estado observado, la función de valor estima la conveniencia esperada de las acciones disponibles y la decisión se obtiene comparando esas estimaciones. El esquema corresponde directamente al caso de una función de valor de acción $Q(s, a)$; si se utiliza una función de valor de estado $V(s)$, la elección de la acción requiere valorar los estados sucesores que produciría cada alternativa. Por ello, $V(s)$ y $Q(s, a)$ cumplen funciones relacionadas, pero no intercambiables en la evaluación de los estados y en la selección de acciones.

En los métodos basados en valor, el objetivo es determinar la conveniencia de encontrarse en un estado o de ejecutar una acción concreta. Formalmente, esto se expresa mediante la función de valor de estado $v_\pi(s) \doteq \mathbb{E}_\pi[G_t \mid S_t = s]$ y la función de valor de acción $q_\pi(s, a) \doteq \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a]$, las cuales satisfacen las relaciones recursivas de las ecuaciones de Bellman [SB18]. En la notación empleada en la Figura 2.10, estas

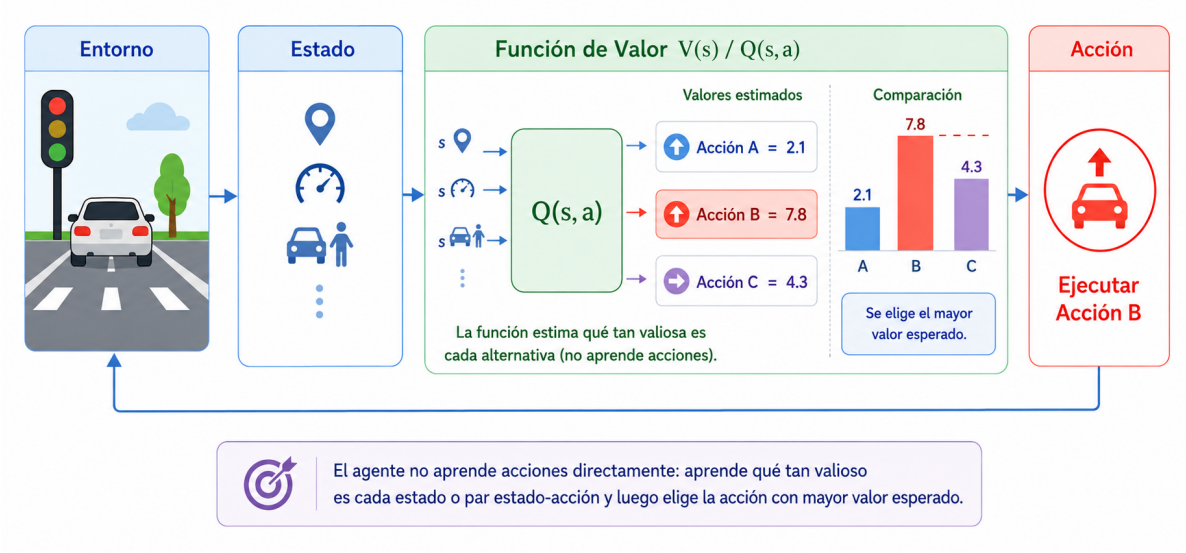


Figura 2.10: Esquema de un método basado en valor. A partir del estado observado s, la función de valor estima el retorno esperado de los pares estado–acción $Q(s, a)$. Las estimaciones para las acciones disponibles se comparan y se selecciona la alternativa con mayor valor esperado para ejecutarla en el entorno. La exploración puede incorporarse mediante estrategias como la selección ε-greedy [SB18].

funciones se escriben habitualmente como $V(s)$ y $Q(s, a)$ cuando no es necesario hacer explícita la política respecto de la cual se calcula el valor.

Cuando el propósito es identificar la política óptima en lugar de evaluar una fija, se emplean las ecuaciones de optimalidad de Bellman:

$$q_*(s, a) = \mathbb{E}\left[R_{t+1} + \gamma \max_{a'} q_*(S_{t+1}, a') \mid S_t = s, A_t = a\right]. \tag{2.7}$$

En la ecuación (2.7), la esperanza $\mathbb{E}$ promedia las transiciones estocásticas del entorno y la maximización sobre las fases admisibles a' formaliza el principio de optimalidad. En el ámbito semafórico, esta relación expresa el compromiso entre la recompensa inmediata obtenida por la fase activa y la calidad del estado vehicular resultante para las decisiones subsiguientes [SB18].

En ausencia de un modelo analítico de transición, los métodos de TD aproximan estas funciones a partir de la experiencia mediante *bootstrapping*: utilizan una estimación del valor del estado siguiente para actualizar el valor actual. Un ejemplo es Q-learning, cuya regla de actualización converge hacia la función óptima $q_*(s, a)$ en el caso tabular, bajo el supuesto de que todos los pares estado–acción continúen explorándose y que la tasa de aprendizaje decrezca adecuadamente a lo largo del tiempo [SB18]:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\left[R_{t+1} + \gamma \max_{a} Q(S_{t+1}, a) - Q(S_t, A_t)\right] \tag{2.8}$$

donde $\alpha \in (0, 1]$ es la tasa de aprendizaje (*learning rate*) que pondera la magnitud del

ajuste, R_{t+1} es la recompensa inmediata observada, y el término $\max_a Q(S_{t+1}, a)$ aproxima el retorno óptimo alcanzable desde el estado sucesor S_{t+1}.

Q-learning es un algoritmo *off-policy*: la política que genera los datos de interacción puede diferir de la política objetivo cuya función de valor se optimiza. Por el contrario, los métodos de gradiente de política operan generalmente de forma *on-policy*, utilizando transiciones producidas por la política en curso de optimización. Esta diferencia resulta decisiva al extender los algoritmos al entorno multiagente, donde las políticas de las intersecciones vecinas evolucionan simultáneamente.

En los métodos basados en política, la regla de decisión se parametriza directamente como $\pi_\theta(a \mid s) = \Pr\{A_t = a \mid S_t = s, \theta\}$, donde $\theta \in \mathbb{R}^d$ representa el vector de parámetros ajustables de la política. La política puede ser determinista, si asigna siempre la misma acción a un estado, o estocástica, si produce una distribución de probabilidad sobre las acciones posibles [SB18]. La Figura 2.11 ilustra este último caso: para el estado observado, la política asigna probabilidades a cada fase admisible y la selección se efectúa mediante muestreo o según la alternativa más probable, de acuerdo con la regla de ejecución adoptada.

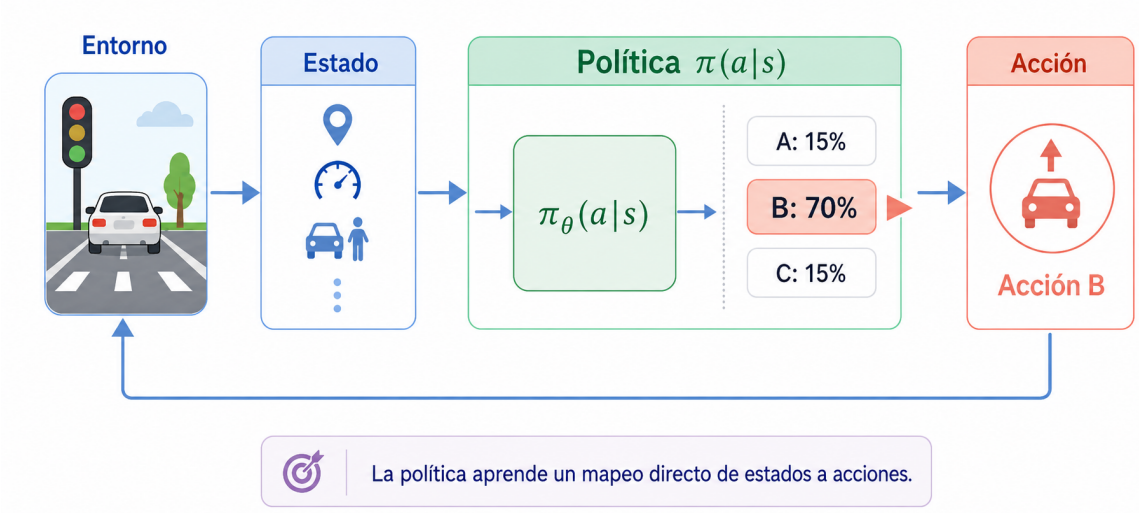


Figura 2.11: Esquema de un método basado en política. A partir del estado observado s, la política parametrizada $\pi_\theta(a \mid s)$ genera directamente una distribución de probabilidad sobre las acciones disponibles. La regla de ejecución puede seleccionar la alternativa de mayor probabilidad o muestrear una acción a partir de dicha distribución [SB18].

La optimización persigue maximizar el rendimiento esperado $J(\theta)$ mediante ascenso de gradiente:

$$\theta_{t+1} = \theta_t + \alpha \nabla J(\theta_t) \tag{2.9}$$

donde $\nabla J(\theta_t)$ es el gradiente del rendimiento respecto de los parámetros y $\alpha > 0$ determina la magnitud del paso de actualización. El Teorema del Gradiente de Política (*Policy Gradient Theorem*) proporciona el sustento teórico para estimar analíticamente $\nabla J(\theta_t)$

a partir de trayectorias muestreadas, ajustando la política para elevar la probabilidad de aquellas fases que conducen a menores demoras en la red [SB18].

Los métodos actor-crítico combinan una política parametrizada con una estimación de valor y actualizan ambos componentes de manera concurrente [KT03, SB18]. El actor representa la política $\pi_\theta(a \mid s)$ y determina la distribución utilizada para seleccionar la acción. El crítico no controla el entorno: estima, por ejemplo, el valor de estado $V_\phi(s)$ y produce una señal que indica si el resultado observado fue mejor o peor que lo esperado bajo la política vigente. Los parámetros θ y ϕ pueden optimizarse de forma independiente o compartir una representación común de las observaciones de tránsito.

En una formulación actor-crítico de un paso, después de ejecutar A_t en S_t, el entorno devuelve la recompensa R_{t+1}, el estado S_{t+1} y el indicador terminal d_{t+1}. El crítico calcula el error de diferencia temporal mediante:

$$\delta_t = R_{t+1} + \gamma(1 - d_{t+1})V_\phi(S_{t+1}) - V_\phi(S_t), \tag{2.10}$$

En la ecuación (2.10), el factor $(1 - d_{t+1})$ anula la contribución del estado siguiente ante transiciones terminales. Cuando V_ϕ aproxima adecuadamente el valor real V^π, δ_t actúa como un estimador de un paso de la ventaja: un valor positivo refleja que la fase ejecutada reportó un desempeño superior al esperado en ese estado de tránsito, mientras que un valor negativo indica lo contrario. El crítico ajusta sus parámetros ϕ minimizando el error temporal, mientras que el actor utiliza esta misma señal δ_t para modular la actualización de la política:

$$\theta \leftarrow \theta + \alpha_\theta \, \delta_t \, \nabla_\theta \log \pi_\theta(A_t \mid S_t). \tag{2.11}$$

En la regla (2.11), $\alpha_\theta > 0$ es la tasa de aprendizaje del actor y $\nabla_\theta \log \pi_\theta(A_t \mid S_t)$ indica cómo cambiar sus parámetros para aumentar la probabilidad de la acción elegida. Un error δ_t positivo favorece esa acción, mientras que uno negativo reduce su probabilidad. Los métodos posteriores emplean estimadores de ventaja de varios pasos, como estimación generalizada de ventaja (*Generalized Advantage Estimation*, GAE), y procesan conjuntos de transiciones llamados lote (*batch*). Esta regla permite comprender cómo el crítico orienta la actualización del actor.

La Figura 2.12 sintetiza el flujo operativo entre actor y entorno, así como el circuito de evaluación mediante el cual el crítico genera la señal de retroalimentación compartida.

En síntesis, el estado resume las condiciones de tránsito disponibles para el agente, la acción corresponde a la selección de una fase, la recompensa refleja la consecuencia operativa inmediata y el valor estima su impacto a largo plazo. Cuando el espacio de estados es continuo o de gran escala, como ocurre en las redes viales urbanas, las funciones de valor y las políticas requieren aproximadores de alta capacidad representacional. A continuación se introducen las redes neuronales artificiales como el modelo adoptado para este fin, preservando intactos los fundamentos de RL expuestos hasta aquí.

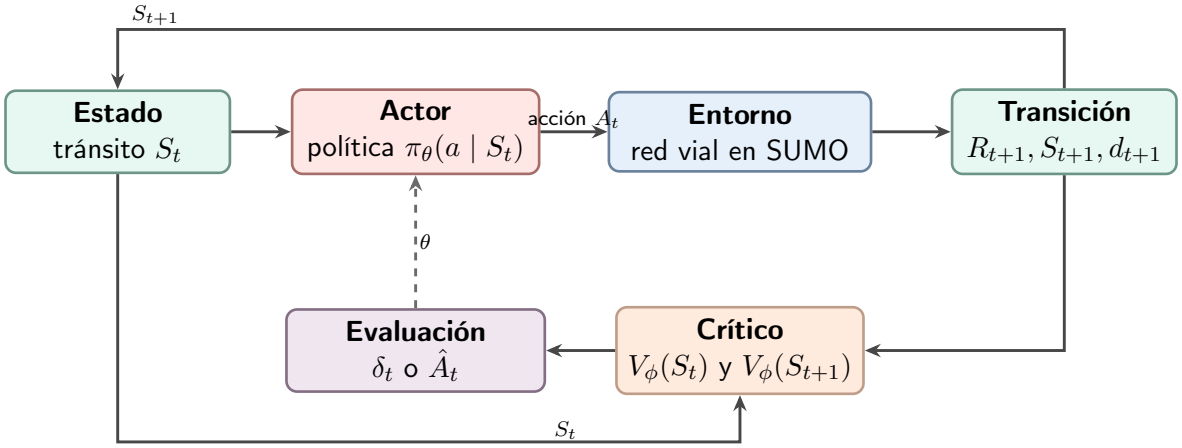


Figura 2.12: Arquitectura actor-crítico aplicada al control semafórico en la formulación de un MDP. El actor selecciona una acción y el crítico evalúa la transición para construir la señal que actualiza ambos componentes. Elaboración propia a partir de [KT03, SB18].

2.2.3. Aprendizaje profundo y redes neuronales

Para estructurar este desarrollo, se expone en primer término la unidad neuronal básica y su composición en redes multicapa; luego se aborda su optimización mediante retropropagación y descenso de gradiente; finalmente, se analiza su integración en RL a través de DQN y PPO. Cabe destacar que la red neuronal actúa como un aproximador de funciones de alta capacidad, mientras que la dirección del aprendizaje proviene exclusivamente de las señales de recompensa generadas por la interacción con el entorno [SB18].

El aprendizaje profundo (*Deep Learning*) se enfoca en el entrenamiento de arquitecturas con múltiples capas de parámetros ajustables capaces de extraer representaciones jerárquicas y relaciones no lineales directamente de los datos [BB24, GBC16]. En este trabajo, esta capacidad se aprovecha para aproximar funciones de valor y políticas de control semafórico donde una formulación tabular resulta impracticable debido a la continuidad y dimensionalidad del estado del tránsito.

Aunque la denominación "neuronal" proviene de una analogía simplificada con modelos biológicos iniciales, una red artificial constituye un modelo matemático determinista cuyos parámetros se optimizan a partir de datos y gradientes de pérdida [GBC16].

La unidad neuronal y redes *feedforward*

En problemas de control con espacios de estados continuos o de gran escala, los métodos tabulares resultan impracticables [SB18]. La aproximación de funciones resuelve esta limitación mediante representaciones parametrizadas, lo que permite generalizar lo aprendido a situaciones similares no observadas. Para este fin se emplean redes neuronales artificiales [BB24].

La unidad neuronal artificial constituye el bloque computacional básico de estas arquitecturas [BB24, GBC16]. Esta procesa un vector de entradas ponderándolo mediante un vector de pesos, añade un término de sesgo y aplica una función de activación no lineal

para producir una respuesta escalar. En el ámbito semafórico, las entradas corresponden típicamente a variables observables del tránsito (como tiempos de espera acumulados o longitudes de cola por carril), mientras que los pesos modulan la contribución relativa de cada acceso hacia la decisión final.

La Figura 2.13 ilustra este procesamiento: la transformación afín $z = \mathbf{w}^\top \mathbf{x} + b$ integra las entradas $\mathbf{x} = (x_1, \ldots, x_n)$ con los pesos $\mathbf{w}$ y el sesgo b, antes de que la activación $y = g(z)$ genere la salida. Sin la introducción de estas no linealidades $g(\cdot)$, una red multicapa se reduciría a una transformación afín simple, imposibilitando la captura de patrones de congestión complejos.

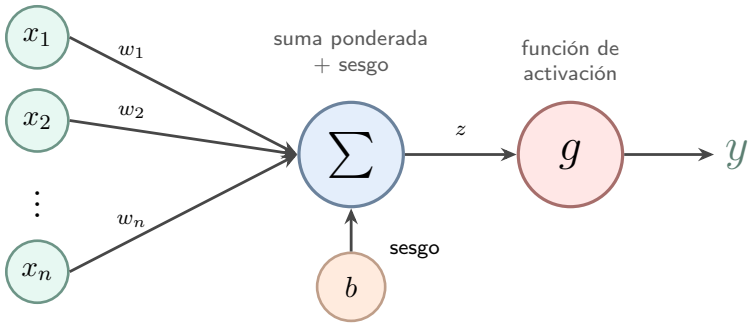


Figura 2.13: Operaciones de una unidad neuronal artificial: suma ponderada de las entradas, incorporación del sesgo y aplicación de una función de activación.

Como ejemplo conceptual, la red puede recibir las métricas de demora de una intersección y emitir estimaciones de calidad o probabilidades sobre las fases admisibles, orientando la asignación del verde hacia las aproximaciones más críticas.

En una red multicapa, la operación para una unidad u dentro de la capa k se formaliza mediante la ecuación (2.12) [BB24, GBC16]:

$$a_{k,u} = \mathbf{w}_{k,u}^\top \mathbf{x}_{k-1} + b_{k,u}, \qquad y_{k,u} = g_k(a_{k,u}) \tag{2.12}$$

donde $\mathbf{x}_{k-1}$ es el vector de activaciones proveniente de la capa anterior, $\mathbf{w}_{k,u}$ es el vector de pesos, $b_{k,u}$ es el sesgo y g_k es la función de activación. Entre las funciones más empleadas en capas intermedias destacan:

- **unidad lineal rectificada (*Rectified Linear Unit*, ReLU):** $g(z) = \max\{0, z\}$, que propaga linealmente las activaciones positivas y bloquea las negativas, con pendiente uno para valores positivos durante la optimización.
- **Leaky ReLU:** $g(z) = \max\{0, z\} + \alpha \min\{0, z\}$, con $\alpha \ll 1$, que conserva una pendiente pequeña en el semieje negativo para mitigar la inactivación de unidades.
- **Tanh:** $g(z) = \tanh(z)$, que ofrece una respuesta no lineal diferenciable acotada en $(-1, 1)$.

Las redes *feedforward* encadenan estas transformaciones en capas sucesivas sin conexiones recurrentes sin retroalimentación temporal. Para una arquitectura con L capas parametrizadas, la propagación completa se describe como:

$$\mathbf{h}_k = g_k\left(\mathbf{W}_k^\top \mathbf{h}_{k-1} + \mathbf{b}_k\right), \qquad k = 1, \ldots, L, \qquad \mathbf{h}_0 = \mathbf{x}, \qquad \mathbf{y} = \mathbf{h}_L \tag{2.13}$$

donde $\mathbf{W}_k$ y $\mathbf{b}_k$ denotan la matriz de pesos y el vector de sesgos de la capa k, respectivamente, y g_k se aplica componente a componente. En el agente semafórico, el vector de entrada $\mathbf{h}_0 = \mathbf{x}$ contiene las esperas acumuladas por carril y la salida $\mathbf{y} = \mathbf{h}_L$ produce las valoraciones Q o probabilidades de selección de fase.

La Figura 2.14 esquematiza esta composición con dos capas ocultas conectadas en cascada hacia la salida.

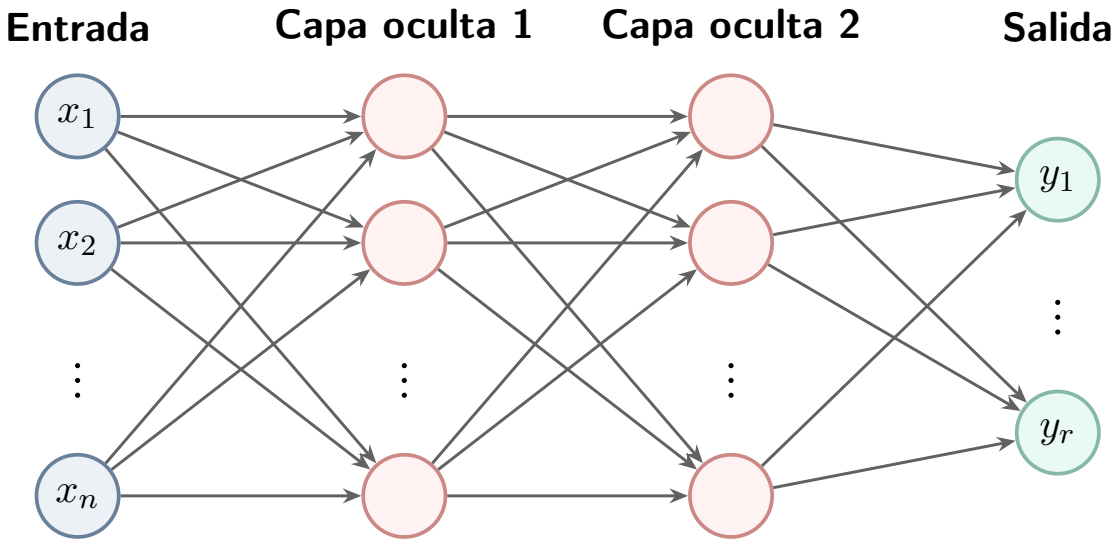


Figura 2.14: Arquitectura conceptual de una red neuronal *feedforward* con dos capas ocultas.

El Teorema de Aproximación Universal es un resultado de existencia: bajo condiciones apropiadas, una red de dos capas parametrizadas, con una capa oculta suficientemente ancha, puede aproximar una función continua sobre un dominio compacto con una precisión arbitraria. El teorema no proporciona por sí mismo un procedimiento para encontrar los parámetros, no garantiza un entrenamiento eficiente, una buena generalización ni un buen desempeño práctico sobre datos no observados. La profundidad puede ofrecer representaciones jerárquicas y una mayor eficiencia expresiva para ciertas familias de funciones, pero agregar capas o neuronas no garantiza por sí solo un menor error de prueba [BB24].

El proceso de optimización

Durante el entrenamiento, la red procesa las observaciones mediante propagación hacia adelante y contrasta su respuesta frente a un objetivo mediante una función de pérdida escalar diferenciable $\mathcal{L}(\theta)$. El algoritmo de retropropagación (*backpropagation*) calcula

analíticamente las derivadas parciales de dicha pérdida respecto de cada peso y sesgo aplicando de forma sistemática la regla de la cadena [BB24].

Los pesos y sesgos son parámetros que se ajustan durante el aprendizaje; los hiperparámetros, como la tasa de aprendizaje y el tamaño de las capas ocultas, son valores de configuración que se fijan antes de entrenar y pueden seleccionarse mediante una búsqueda.

Con estos gradientes, los parámetros se actualizan iterativamente en sentido opuesto a la pendiente del error:

$$\theta_{t+1} = \theta_t - \eta \nabla_\theta \mathcal{L}(\theta_t), \tag{2.14}$$

donde θ reúne los parámetros entrenables de la red, $\nabla_\theta \mathcal{L}(\theta_t)$ es el gradiente de la función de pérdida y $\eta > 0$ es la tasa de aprendizaje. Calcular el gradiente con todos los datos puede resultar costoso. Por ello, se utilizan subconjuntos de muestras, denominados *minibatches*, para realizar cada actualización. Adam adapta el tamaño del paso para cada parámetro a partir de promedios móviles del gradiente y de su cuadrado [BB24].

A diferencia del aprendizaje supervisado convencional —donde la pérdida compara la salida con una etiqueta externa fija—, en RL el objetivo se deriva de las recompensas y transiciones producidas por la interacción con el simulador, y cambia a medida que la política mejora su comportamiento.

2.2.4. Aprendizaje por refuerzo profundo (DRL)

El DRL combina algoritmos de RL con aproximadores neuronales. En el control semafórico, las observaciones del tránsito, como colas, ocupación o velocidades, pueden alimentar la red; sus salidas pueden representar valores de acción o parámetros de una política; y las recompensas asociadas con demoras, detenciones y colas orientan la función de pérdida del algoritmo de aprendizaje. Esta integración resulta útil cuando el espacio de estados es continuo o de alta dimensionalidad, aunque exige mecanismos de estabilización y una evaluación cuidadosa de la eficiencia de las muestras [ACS24].

Factores de inestabilidad del aprendizaje

La integración de aproximadores no lineales con aprendizaje por refuerzo puede inducir inestabilidad numérica o divergencia en las estimaciones si no se controlan las condiciones de actualización. Sutton y Barto [SB18] describen este fenómeno mediante la denominada tríada mortal (*deadly triad*), originada por la concurrencia de tres elementos:

- **Aproximación de funciones:** el uso de un modelo parametrizado, como una red neuronal, para estimar valores en lugar de almacenarlos individualmente en una tabla.
- ***Bootstrapping*:** la actualización de estimaciones de valor apoyándose en otras estimaciones aprendidas previamente (métodos TD), sin esperar a disponer del retorno

completo.

- **Entrenamiento *off-policy*:** el aprendizaje sustentado en una distribución de datos generada por una política de comportamiento distinta de la política objetivo.

Aunque prescindir del *bootstrapping* o de la aproximación de funciones comprometería la eficiencia muestral o la escalabilidad ante redes viales complejas, los algoritmos profundos modernos introducen mecanismos arquitectónicos específicos para amortiguar esta inestabilidad [SB18].

Deep Q-Network (DQN): aproximación neuronal de valores de acción

En los métodos basados en valor, DQN sustituye la tabla de estados y acciones por una red neuronal $Q(s, a; \theta)$ parametrizada por θ [MKS+15]. Ante una observación de tránsito, la red genera una estimación del valor Q para cada fase admisible. La función de acción-valor óptima $q_*(s, a)$ representa el máximo retorno esperado al seleccionar la mejor secuencia de decisiones futuras, objetivo que DQN persigue aproximar a partir de las recompensas del entorno.

Para mitigar la inestabilidad de la tríada mortal —en particular el problema del valor objetivo cambiante, donde actualizar el valor de un estado altera impredeciblemente las estimaciones de estados adyacentes—, DQN incorpora dos componentes estructurales [MKS+15]:

- **Redes objetivo (*Target Networks*):** una red secundaria idéntica cuyos parámetros θ^- se congelan temporalmente. Esta red se utiliza exclusivamente para calcular el término de *bootstrapping* del valor objetivo TD, desacoplando la estimación en curso del objetivo de aprendizaje y amortiguando oscilaciones.

- **Memoria de repetición de experiencias (*replay buffer*):** almacena transiciones históricas $(s_t, a_t, r_{t+1}, s_{t+1}, d_{t+1})$ en un *buffer* $\mathcal{D}$. La optimización extrae *minibatches* aleatorios de esta memoria, lo cual mitiga la correlación temporal propia del flujo continuo de vehículos en el simulador y aproxima los datos a la hipótesis de muestras independientes.

Durante la interacción con SUMO, la red en línea evalúa la observación de tránsito y una regla ϵ_{exp}-greedy selecciona la fase a ejecutar; la transición resultante se almacena en $\mathcal{D}$. Durante el entrenamiento, la red en línea actualiza sus pesos θ a partir de los *minibatches* muestreados del *buffer* contra el valor objetivo fijado por la red objetivo, cuyos parámetros θ^- se sincronizan solo de forma periódica (Figura 2.15).

La función de pérdida minimizada por DQN, representada en el bloque inferior de la

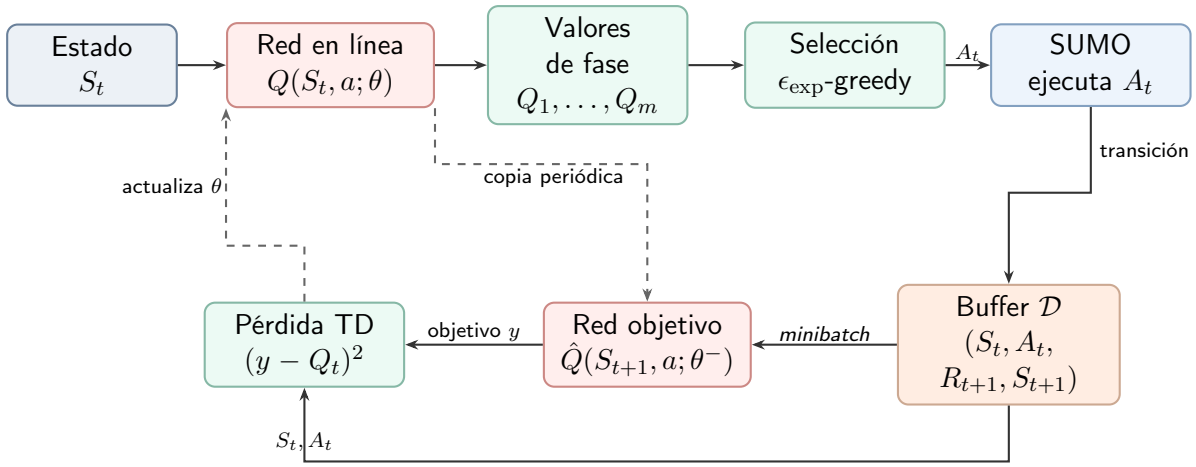


Figura 2.15: Flujo conceptual de DQN con repetición de experiencias y red objetivo. La red en línea selecciona acciones y recibe las actualizaciones; la red objetivo proporciona el término de *bootstrapping* del valor objetivo TD. Elaboración propia a partir del algoritmo descrito por Mnih et al. [MKS+15].

Figura 2.15, es la de la ecuación (2.15):

$$\mathcal{L}(\theta) = \mathbb{E}_{(s_j, a_j, r_{j+1}, s_{j+1}, d_{j+1}) \sim \mathcal{D}} \left[\left(\underbrace{r_{j+1} + \gamma(1 - d_{j+1}) \max_{a'} \hat{Q}(s_{j+1}, a'; \theta^-)}_{\text{Valor objetivo de TD}} - \underbrace{Q(s_j, a_j; \theta)}_{\text{Predicción actual}} \right)^2 \right] \tag{2.15}$$

En la ecuación (2.15), la esperanza se evalúa empíricamente promediando sobre un *mini-batch* de transiciones $(s_j, a_j, r_{j+1}, s_{j+1}, d_{j+1})$ extraídas de $\mathcal{D}$. La función de pérdida minimiza la discrepancia cuadrática entre la estimación actual $Q(s_j, a_j; \theta)$ y el valor objetivo temporal compuesto por la recompensa inmediata observada y la mayor estimación del valor futuro sobre las fases admisibles, calculada por la red objetivo $\hat{Q}(s_{j+1}, a'; \theta^-)$. El factor $(1 - d_{j+1})$ anula la contribución del estado siguiente cuando la transición es terminal $(d_{j+1} = 1)$, mientras que a' recorre las fases semafóricas admisibles.

Los cuatro bloques numerados como algoritmos en este capítulo son pseudocódigos: descripciones ordenadas e independientes de un lenguaje de programación, no fragmentos ejecutables. En ellos, una transición registra lo ocurrido en un paso; un episodio agrupa pasos desde el reinicio hasta la terminación; y un *rollout* es la secuencia de transiciones recolectada con una política. Una iteración de entrenamiento comprende la recolección de datos y su optimización, mientras que una *epoch* es una reutilización completa de esos datos dentro de la misma iteración. Cada actualización puede calcularse sobre un *mini-batch*, es decir, un subconjunto de las muestras. Las líneas "Entrada" y "Salida" indican, respectivamente, qué información necesita el procedimiento y qué componentes entrenados produce; las líneas indentadas pertenecen al ciclo inmediatamente superior. En lo sucesivo se utiliza lote para *batch* y se conservan los términos *minibatch*, *buffer* y *epoch*,

definidos aquí y en el glosario.

El procedimiento de DQN combina la recolección de experiencias, la selección de fases con política ϵ_{exp}-greedy y la optimización de la red neuronal a partir de *minibatches* muestreados del *replay buffer*. El algoritmo 1 especifica este procedimiento:

Algoritmo 1 Deep Q-Network con repetición de experiencias

Entrada: entorno, fases admisibles, capacidad N, horizonte T en pasos de decisión, γ y ϵ_{exp}

Salida: parámetros entrenados θ de la red de acción-valor

Inicializar el *buffer* de experiencias $\mathcal{D}$ con capacidad N
Inicializar la red de acción-valor $Q(s, a; \theta)$ con parámetros aleatorios
Inicializar la red objetivo $\hat{Q}(s, a; \theta^-)$ con $\theta^- \leftarrow \theta$
para episodio $= 1, 2, \ldots$ **hacer**
    Inicializar el estado s_t
    para $t = 1, 2, \ldots, T$ **hacer**
        Seleccionar a_t mediante una política ϵ_{exp}-greedy basada en $Q(s_t, a; \theta)$
        Ejecutar a_t y observar la recompensa r_{t+1}, el estado s_{t+1} y si el episodio terminó
        Almacenar $(s_t, a_t, r_{t+1}, s_{t+1}, d_{t+1})$ en $\mathcal{D}$
        si $\mathcal{D}$ contiene al menos un *minibatch* completo **entonces**
            Extraer un *minibatch* de transiciones de $\mathcal{D}$
            Calcular $y_j = r_{j+1} + \gamma(1 - d_{j+1}) \max_{a'} \hat{Q}(s_{j+1}, a'; \theta^-)$
            Actualizar θ minimizando $(y_j - Q(s_j, a_j; \theta))^2$
        fin si
        Actualizar periódicamente la red objetivo: $\theta^- \leftarrow \theta$
        $s_t \leftarrow s_{t+1}$
        si $d_{t+1} = 1$ **entonces**
            Terminar el ciclo interno y comenzar un nuevo episodio
        fin si
    fin para
fin para

En esta formulación, el agente desacopla la recolección de experiencias en el simulador de la optimización paramétrica de la red. La memoria $\mathcal{D}$ reduce la correlación temporal de las observaciones de tráfico, mientras que la red objetivo estabiliza el valor objetivo de Bellman mediante la sincronización periódica de θ^-. La especificación detallada de los hiperparámetros operacionales (capacidad N, horizonte T, tasa de aprendizaje y cronograma de exploración) se desarrolla en el capítulo 3.

Aproximación de políticas, función de ventaja y *Policy Gradient*

Como alternativa a derivar la política de forma indirecta a partir de una función de valor (como en DQN), DRL permite parametrizar directamente la política del agente mediante una red neuronal $\pi(a|s;\theta)$ [ACS24]. Bajo la formulación MDP, en un problema de acciones discretas la red recibe el estado s, calcula un vector de puntajes continuos $z \in \mathbb{R}^m$ y los proyecta a una distribución de probabilidad mediante la función *softmax* [SB18, BB24]:

$$\pi_\theta(a_i \mid s) = \frac{e^{z_i}}{\sum_{j=1}^{m} e^{z_j}} \tag{2.16}$$

la cual garantiza valores estrictamente positivos que suman la unidad. En este ejemplo, las acciones corresponden a las fases semafóricas admisibles: las variables del estado de tránsito ingresan a la red y sus salidas representan probabilidades sobre dicho conjunto de acciones. Las políticas basadas en observaciones locales se formalizan posteriormente mediante Dec-POMDP. La acción puede muestrearse de esa distribución durante el entrenamiento, preservando la exploración estocástica.

La Figura 2.16 muestra este recorrido para un agente semafórico. A diferencia de DQN, las salidas no son estimaciones de valor: son probabilidades que suman uno. La señal de ventaja no entra a la red para seleccionar la acción; se utiliza durante el entrenamiento para modificar los parámetros en la dirección indicada por el gradiente de la log-probabilidad de la acción observada.

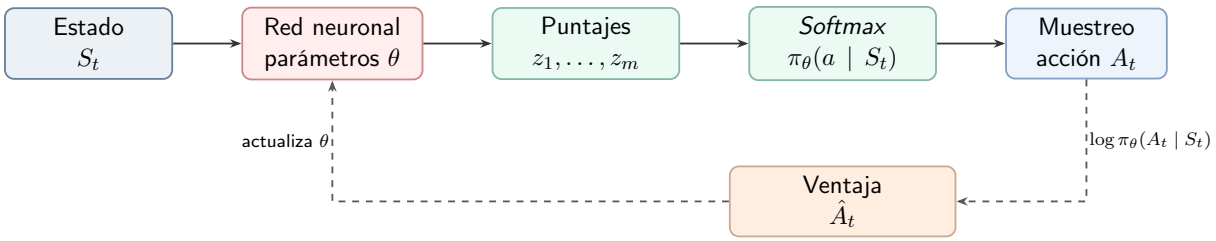


Figura 2.16: Red de política para un espacio discreto de acciones en la formulación de un MDP. El estado determina una distribución sobre las acciones disponibles; durante el aprendizaje, la ventaja pondera el gradiente de la log-probabilidad de la acción seleccionada [SB18, KT03].

La optimización de políticas se fundamenta en aislar la contribución específica de una acción respecto del rendimiento promedio del estado. Formalmente, la función de ventaja (*Advantage Function*) se define como la diferencia entre el valor de la acción y el valor medio del estado [SB18]:

$$A^\pi(s,a) \doteq Q^\pi(s,a) - V^\pi(s) \tag{2.17}$$

donde $Q^\pi(s,a)$ es el retorno esperado al seleccionar la acción a en el estado s bajo la política π, y $V^\pi(s) = \mathbb{E}_{a\sim\pi}[Q^\pi(s,a)]$ representa el valor esperado medio en dicho estado. Un valor $A^\pi(s,a) > 0$ señala que la fase semafórica ejecutada supera el desempeño promedio

de la intersección ante esas condiciones de cola, mientras que $A^\pi(s,a) < 0$ indica una decisión comparativamente deficiente.

Para estimar la ventaja combinando errores temporales de distintos horizontes, Schulman et al. [SML+16] desarrollaron el estimador GAE:

$$\hat{A}_t^{\mathrm{GAE}(\gamma,\lambda)} \doteq \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l}^V \tag{2.18}$$

donde $\delta_t^V = R_{t+1} + \gamma(1 - d_{t+1})V_\phi(S_{t+1}) - V_\phi(S_t)$ es el error temporal del crítico parametrizado por ϕ, d_{t+1} es el indicador terminal, $\gamma \in [0, 1)$ es el factor de descuento y $\lambda \in [0, 1]$ es el parámetro de suavizado exponencial. El hiperparámetro λ interpola estimaciones de ventaja de distintos horizontes. Para $\lambda = 0$, se obtiene el residuo TD de un paso δ_t^V, que suele reducir la varianza a costa de depender de la precisión del crítico y, por tanto, puede introducir sesgo. Para $\lambda = 1$, con el retorno completo, se recupera el estimador Monte Carlo descontado menos el valor del estado; este suele presentar mayor varianza. En trayectorias truncadas, el *bootstrapping* y la aproximación del crítico pueden modificar este compromiso.

El Teorema del Gradiente de Política (*Policy Gradient Theorem*) [SB18] fundamenta analíticamente la optimización directa de los parámetros de la política maximizando el retorno esperado $J(\theta)$. Empleando la función de ventaja como línea de base para mitigar la varianza, el gradiente se aproxima mediante la esperanza muestral sobre un lote de transiciones recolectadas:

$$\nabla_\theta J(\theta) \approx \hat{\mathbb{E}}_t \left[\nabla_\theta \log \pi_\theta(A_t \mid S_t) \hat{A}_t \right] \tag{2.19}$$

donde $\pi_\theta(A_t \mid S_t)$ denota la probabilidad asignada a la fase A_t bajo los parámetros θ, y $\hat{A}_t$ es la ventaja estimada mediante GAE. Como esquematiza la Figura 2.16, la dirección del vector gradiente escala proporcionalmente a $\hat{A}_t$, incrementando la probabilidad de ocurrencia de aquellas fases que superan el desempeño esperado y desincentivando aquellas que provocan demoras excesivas.

Proximal Policy Optimization (PPO) y control de las actualizaciones

Una actualización demasiado grande de los parámetros puede reducir el rendimiento de la política. Los métodos que se describen a continuación buscan limitar ese cambio para conservar un aprendizaje estable.

Para acotar el tamaño de paso en el espacio de distribuciones de probabilidad, Kakade y Langford [KL02] y Schulman et al. formularon el algoritmo de optimización de política por región de confianza (*Trust Region Policy Optimization*, TRPO) [SLA+15] mediante métodos de región de confianza. En el análisis teórico de TRPO, bajo los supuestos del modelo y una actualización que respete la región de confianza basada en la divergencia de

Kullback–Leibler (KL, una medida probabilística que cuantifica la discrepancia estadística entre la nueva política y la previa [GBC16, SLA+15]), la maximización exacta de la cota sustituta produce una política con retorno esperado no decreciente:

$$J(\pi_{k+1}) \geq J(\pi_k) \tag{2.20}$$

El algoritmo práctico de TRPO aproxima este procedimiento mediante ventajas estimadas, una divergencia KL promedio y optimización numérica. Sin embargo, resolver la optimización cuadrática con restricciones de TRPO en redes neuronales profundas resulta costoso en términos de cómputo. Como alternativa de primer orden, PPO [SWD+17] reemplaza la restricción formal por un objetivo sustituto recortado (CLIP) que busca limitar actualizaciones excesivas, sin transferir automáticamente la garantía teórica de TRPO.

Definiendo la razón de probabilidad entre la política actual y la previa:

$$r_t(\theta) \doteq \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\text{old}}}(a_t \mid s_t)} \tag{2.21}$$

donde $\pi_{\theta_{\text{old}}}$ representa la política que recolectó el lote de transiciones, PPO maximiza el objetivo recortado:

$$L^{\text{CLIP}}(\theta) = \hat{\mathbb{E}}_t \left[\text{mín} \left(r_t(\theta)\hat{A}_t,\ \text{clip}(r_t(\theta), 1-\epsilon_{\text{clip}}, 1+\epsilon_{\text{clip}})\hat{A}_t \right) \right] \tag{2.22}$$

En la ecuación (2.22), $\epsilon_{\text{clip}} > 0$ fija el margen de recorte. La función $\text{clip}(x, l, u)$ conserva x si está entre l y u, devuelve l si queda por debajo y u si lo supera. El operador mín elige el menor de los dos términos. Así, una ventaja positiva deja de incentivar aumentos de la razón por encima de $1+\epsilon_{\text{clip}}$, y una ventaja negativa deja de incentivar reducciones por debajo de $1-\epsilon_{\text{clip}}$. Este mecanismo limita el incentivo del objetivo; no impone una restricción estricta al cambio de la política.

En la arquitectura actor-crítico práctica, este objetivo de política se integra de forma conjunta con la regresión del crítico y un término de regularización entrópica:

$$J^{\text{PPO}}(\theta, \phi) = L^{\text{CLIP}}(\theta) - c_1\hat{\mathbb{E}}_t \left[\left(V_\phi(s_t) - R_t^{\text{target}} \right)^2 \right] + c_2\hat{\mathbb{E}}_t \left[S[\pi_\theta](s_t) \right] \tag{2.23}$$

donde ϕ son los parámetros del crítico, R_t^{target} es el retorno objetivo estimado (calculado como $\hat{A}_t + V_\phi(s_t)$), $S[\pi_\theta](s_t) \doteq -\sum_a \pi_\theta(a \mid s_t) \log \pi_\theta(a \mid s_t)$ es la entropía de Shannon de la política en s_t, y $c_1, c_2 > 0$ son coeficientes de ponderación [SWD+17]. La inclusión del término entrópico favorece la exploración activa al penalizar distribuciones excesivamente concentradas, favoreciendo que se consideren distintas fases durante el aprendizaje.

PPO opera de forma predominantemente *on-policy*: recolecta un lote de trayectorias con la política vigente $\pi_{\theta_{\text{old}}}$, ejecuta un número acotado de *epochs* de optimización sobre *minibatches*, y luego descarta el lote. De esta forma, las siguientes actualizaciones utilizan

experiencia obtenida con una política más reciente. El algoritmo 2 describe este ciclo de entrenamiento:

Algoritmo 2 Proximal Policy Optimization (PPO)

Entrada: entorno, horizonte de recolección, *epochs* K, γ, λ_{GAE} y ϵ_{clip}
Salida: parámetros entrenados θ del actor y ϕ del crítico

Inicializar los parámetros del actor θ y del crítico ϕ
Inicializar la política de recolección con $\theta_{\text{old}} \leftarrow \theta$
para iteración $= 1, 2, \ldots$ **hacer**
    Recolectar trayectorias con la política $\pi_{\theta_{\text{old}}}$
    Calcular los retornos objetivo y las ventajas estimadas $\hat{A}_t$
    para *epoch* $= 1$ hasta K **hacer**
        Muestrear *minibatches* del conjunto de trayectorias recolectado
        Calcular $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$
        Actualizar θ maximizando $L^{\text{CLIP}}(\theta)$
        Actualizar ϕ minimizando el error cuadrático del valor
        Incorporar, si corresponde, un término de entropía para favorecer la exploración
    fin para
    $\theta_{\text{old}} \leftarrow \theta$
fin para

En PPO, θ_{old} representa la política recolectora y permanece fijo al calcular las razones de probabilidad. El lote se reutiliza durante un número acotado de *epochs*; el recorte reduce el incentivo para que las razones superen el intervalo $[1 - \epsilon_{\text{clip}}, 1 + \epsilon_{\text{clip}}]$, pero no impone una restricción estricta ni garantiza una mejora monótona [SWD+17]. Después de la optimización, θ_{old} se actualiza con θ y se recolectan nuevas experiencias. Durante la ejecución en la red semafórica, el crítico se desacopla y el agente utiliza únicamente su actor local π_θ.

2.3. Aprendizaje por refuerzo multiagente (MARL)

El MARL generaliza el marco de RL hacia sistemas donde múltiples agentes autónomos interactúan y aprenden en un entorno compartido [ACS24]. En el control de redes semafóricas, la hipótesis de agente aislado deja de sostenerse: la fase habilitada en un cruce regula el flujo vehicular emitido hacia los accesos de las intersecciones adyacentes, acoplando espacialmente las dinámicas de congestión de toda la red. Cada agente influye directamente en las tasas de arribo y longitudes de cola de sus vecinos, lo que demanda esquemas de coordinación capaces de optimizar el flujo global.

Liang et al. [LML+25] categorizan los problemas multiagente según la estructura de sus

recompensas en completamente competitivos, mixtos o completamente cooperativos. Este trabajo persigue un objetivo de coordinación vial, pero sus implementaciones entregan recompensas individuales. Las búsquedas informadas emplean recompensa local; la comparación vecinal de MAPPO agrega un término de vecinos en una de sus variantes (sección 3.3.2). La finalidad cooperativa no hace idénticos los retornos individuales ni convierte automáticamente la implementación en un juego de recompensa escalar compartida.

2.3.1. Fundamentos teóricos: juegos estocásticos, equilibrio de Nash y Dec-POMDP

La extensión matemática del MDP ante múltiples actores decisores se formaliza mediante los Juegos Estocásticos o Juegos Markovianos (*Stochastic Games*), introducidos originalmente por Shapley [Sha53] y adaptados al aprendizaje por refuerzo contemporáneo por Albrecht et al. [ACS24].

Un Juego Estocástico de n agentes se define mediante la tupla:

$$\langle \mathcal{N}, \mathcal{S}, \{\mathcal{A}_i\}_{i=1}^{n}, P, \{R_i\}_{i=1}^{n}, \gamma \rangle \tag{2.24}$$

donde $\mathcal{N} = \{1, \ldots, n\}$ es el conjunto de agentes (intersecciones), $\mathcal{S}$ representa el espacio de estados globales de la red vial, $\mathcal{A}_i$ es el conjunto discreto de fases semafóricas disponibles para el agente i, y $\mathbf{a} = (a_1, \ldots, a_n) \in \boldsymbol{\mathcal{A}} \equiv \mathcal{A}_1 \times \cdots \times \mathcal{A}_n$ denota la acción conjunta. La función de transición $P(s' \mid s, \mathbf{a})$ modela la probabilidad de evolucionar hacia el estado global s' dada la acción conjunta ejecutada en el simulador SUMO, y $R_i(s, \mathbf{a})$ es la función de recompensa asignada al agente i.

En el análisis de estabilidad de estos sistemas, el equilibrio de Nash describe una configuración donde ningún agente puede incrementar unilateralmente su retorno esperado desviándose de su política actual [ACS24]. Formalmente, una política conjunta $\boldsymbol{\pi}^* = (\pi_1^*, \ldots, \pi_n^*)$ constituye un equilibrio de Nash si:

$$J(\pi_i, \boldsymbol{\pi}_{-i}^*) \leq J(\pi_i^*, \boldsymbol{\pi}_{-i}^*), \quad \forall \pi_i \in \Pi_i, \quad \forall i \in \mathcal{N} \tag{2.25}$$

donde $J(\cdot)$ denota el retorno esperado del agente i bajo el perfil de políticas indicado, $\boldsymbol{\pi}_{-i}^*$ representa el perfil de políticas conjuntas de todos los agentes excepto el agente i, y Π_i es el espacio de políticas admisibles. Aunque en sistemas estrictamente cooperativos este equilibrio caracteriza puntos de estabilidad mutua, no garantiza por sí mismo la optimalidad global de Pareto.

En redes urbanas reales, los agentes carecen de información instantánea y global sobre todo el trazado vial. Cada intersección opera sujeta a observabilidad parcial, percibiendo únicamente el estado de sus detectores locales. Una formulación teórica para observabilidad parcial y retorno compartido es el proceso de decisión de Markov parcialmente observable descentralizado (*Decentralized Partially Observable Markov Decision Process*,

Dec-POMDP) [AOS20], definido por la tupla:

$$\langle \mathcal{N}, \mathcal{S}, \{\mathcal{A}_i\}_{i=1}^{n}, P, R, \{\Omega_i\}_{i=1}^{n}, O, \gamma \rangle \tag{2.26}$$

donde Ω_i es el conjunto de observaciones locales accesibles al agente i, y $O(o_1, \ldots, o_n \mid s', \mathbf{a}) = \Pr\{\mathbf{o} = (o_1, \ldots, o_n) \mid s', \mathbf{a}\}$ es la función de emisión estocástica de observaciones [AOS20]. El estado global s es una variable del modelo que no tiene por qué ser accesible a cada agente; en general, una política descentralizada puede depender del historial local de observaciones y acciones τ_i y expresarse como $\pi_i(a_i \mid \tau_i)$. En la tupla, R es una recompensa escalar compartida. La implementación experimental entrega señales individuales R_i y se describe como un juego parcialmente observable, sin identificarla automáticamente con ese caso cooperativo de recompensa compartida (sección 3.3.2). Cada agente adopta una política reactiva basada en la observación actual $\pi_i(a_i \mid o_i)$, sin identificar o_i con s (Figura 2.17).

La Figura 2.17 esquematiza este acoplamiento: cada agente procesa su observación local o_i y emite una fase a_i. La concurrencia de estas decisiones conforma la acción conjunta $\mathbf{a}$, la cual perturba el estado global del simulador de tráfico y devuelve a cada agente su nueva observación y su retroalimentación individual r_i.

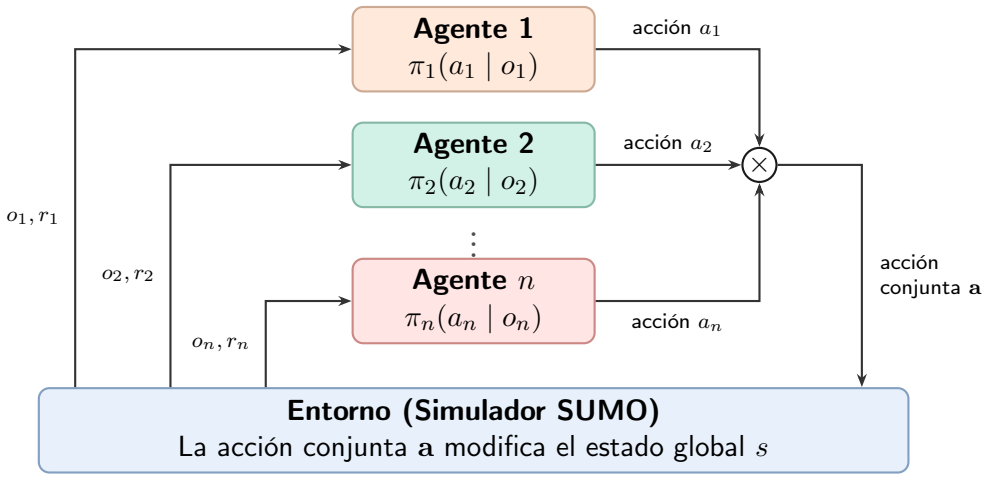


Figura 2.17: Interacción agente-entorno en un esquema MARL cooperativo. Cada semáforo emite una acción local (a_i) formando una acción conjunta ($\mathbf{a}$) que modifica el estado global del entorno de tráfico.

2.3.2. Desafíos del aprendizaje multiagente

La transición desde el aprendizaje individual hacia el dominio multiagente introduce desafíos teóricos y computacionales específicos [ACS24]:

- **Explosión combinatoria del espacio de acciones conjuntas:** si cada agente i dispone de un conjunto de acciones locales $\mathcal{A}_i$, la cardinalidad del espacio de acciones

conjuntas es:

$$|\boldsymbol{\mathcal{A}}| = \prod_{i=1}^{n} |\mathcal{A}_i| \,. \tag{2.27}$$

Cuando todos los agentes poseen A acciones, esta cardinalidad es A^n y, por tanto, crece exponencialmente con el número de agentes. Para una red de escala urbana, evaluar o enumerar exhaustivamente $\boldsymbol{\mathcal{A}}$ deja de ser una alternativa viable, lo que motiva el diseño de enfoques escalables, como políticas de decisión localmente descentralizadas.

- **No estacionariedad y el problema del valor objetivo cambiante:** desde la perspectiva local del agente i, la función de transición ambiental percibida:

$$\mathcal{P}_i(s' \mid s, a_i) = \sum_{\mathbf{a}_{-i}} P(s' \mid s, a_i, \mathbf{a}_{-i}) \boldsymbol{\pi}_{-i}(\mathbf{a}_{-i} \mid s) \tag{2.28}$$

varía de forma continua a medida que los restantes agentes adaptan sus respectivas políticas $\boldsymbol{\pi}_{-i}$. Como formalizaron Bowling y Veloso [BV02], este cambio produce el problema del valor objetivo cambiante (*moving target*): una política óptima frente al comportamiento vigente de los vecinos puede dejar de ser adecuada en cuanto estos ajustan sus parámetros. Aunque el sistema global puede seguir describiéndose mediante un juego markoviano, la dinámica observada por un agente local se vuelve no estacionaria cuando las políticas de los demás agentes cambian y no forman parte de su información. En consecuencia, las condiciones de convergencia de los métodos de agente único basados en MDPs estacionarios dejan de ser suficientes [FNF+17, LML+25].

  En juegos competitivos o de suma cero como Piedra, Papel o Tijera, el aprendizaje independiente con tasas fijas genera trayectorias oscilatorias alrededor del equilibrio. Algoritmos especializados como ascenso de colina en la política con ganar o aprender rápido (*Win or Learn Fast Policy Hill-Climbing*, WoLF-PHC) modulan adaptativamente su tasa de aprendizaje, amortiguando las espirales hacia el equilibrio de Nash ($\pi^*(a) = 1/3$), como ilustra la Figura 2.18. En el control semafórico, la no estacionariedad se manifiesta cuando un cruce extiende su verde para permitir el paso de un grupo de vehículos y modifica las llegadas a los cruces siguientes; si los agentes aprenden sin coordinación, corren el riesgo de generar variaciones de las colas que dificulten el servicio en otros cruces.

- **Observabilidad parcial:** bajo el formalismo Dec-POMDP, cada semáforo basa su inferencia exclusivamente en detectores locales, desconociendo el estado global de la red vial y las decisiones simultáneas de cruces lejanos, lo cual dificulta la estimación del valor de sus decisiones.

- **Asignación de crédito multiagente (*Credit Assignment*):** cuando múltiples agentes actúan en paralelo, determinar la contribución marginal de cada decisión local sobre el desempeño del sistema resulta complejo [ACS24]. Una acción local desfavorable puede quedar enmascarada si el resto de los semáforos operó de forma eficiente, o viceversa. En las variantes vecinales, la recompensa individual combina señales de la intersección y sus vecinos (sección 3.3.2), pero no identifica la contribución causal de cada acción.

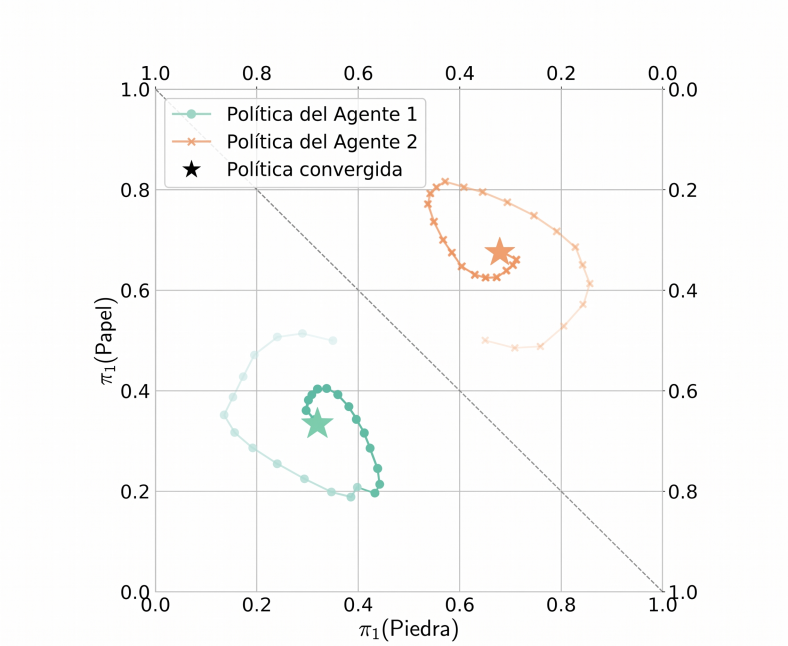


Figura 2.18: Evolución de las políticas de dos agentes con el algoritmo WoLF-PHC en el juego de Piedra, Papel o Tijera. La diagonal discontinua divide el gráfico en dos símplex de probabilidad, uno para cada agente. Cada punto en el símplex representa una distribución sobre las tres acciones disponibles; las trayectorias muestran el avance de las políticas durante el entrenamiento y los marcadores centrales destacan la política convergida en el equilibrio de Nash ($\pi^*(a) = 1/3$ para cada acción). Adaptado de [BV02, ACS24].

2.3.3. Paradigmas de entrenamiento y ejecución

Para mitigar la no estacionariedad respetando la observabilidad parcial de los agentes durante la ejecución, la literatura distingue dos paradigmas fundamentales según la información disponible durante la optimización [ZLYZ23]:

- **Aprendizaje independiente (Independent Learning, IL):** cada agente aprende de forma totalmente descentralizada tratando a los demás agentes como componentes estocásticos del entorno. Aunque computacionalmente simple y altamente escalable, es susceptible a la inestabilidad por no estacionariedad.

- **Entrenamiento centralizado con ejecución descentralizada (Centralized Training with Decentralized Execution, CTDE):** durante la etapa de entrenamiento en el simulador, se aprovecha información global privilegiada (estados conjuntos o concatenaciones de observaciones) para estimar valores y orientar el aprendizaje de las políticas locales; durante la etapa de ejecución, el componente centralizado se desacopla y cada agente opera con su política local. La Figura 2.19 ilustra esta separación entre etapas.

2.3.4. aprendizaje por refuerzo profundo multiagente (*Multi-Agent Deep Reinforcement Learning*, MADRL) y líneas de base

El uso de redes neuronales para aproximar políticas y funciones de valor de varios agentes forma parte del MADRL [LWT+17]. Para evaluar empíricamente el valor agregado del entrenamiento centralizado (MAPPO y HAPPO), esta investigación establece dos líneas de base de aprendizaje independiente (IL):

- **IDQN (Línea de base basada en valor):** extensión de DQN donde cada intersección mantiene su propia red $Q_i(o_i, a_i)$ [TMK+17]. Al operar *off-policy* con un *replay buffer*, almacena transiciones generadas bajo políticas anteriores, por lo que las experiencias pueden reflejar comportamientos distintos de los actuales [FNF+17].

- **IPPO (Línea de base basada en política):** instancia un optimizador PPO independiente por agente [dWGM+20]. Dado su régimen *on-policy*, descarta el lote tras cada iteración, mitigando la acumulación de datos obsoletos. Aunque no elimina el acoplamiento no estacionario, su mecanismo de recorte permite utilizarlo como referencia frente a los métodos coordinados.

La Figura 2.19 resume esta partición operativa entre ambas etapas.

2.3.5. Multi-Agent Proximal Policy Optimization (MAPPO)

Para extender las ventajas de estabilidad de PPO al contexto multiagente bajo el paradigma CTDE, Yu et al. [YVV+22] estudiaron una configuración de Multi-Agent Proximal Policy Optimization (MAPPO) basada en PPO y un crítico centralizado. En lugar de rediseñar el estimador base, MAPPO preserva la simplicidad computacional de PPO mediante la factorización explícita entre políticas descentralizadas y evaluación centralizada:

- **Actores descentralizados ($\pi_{\theta_i}(a_i \mid o_i)$):** cada agente i procesa exclusivamente su vector de observación local o_i para seleccionar su fase semafórica a_i. En intersecciones con geometrías idénticas puede compartirse parámetros entre actores, aunque esto no constituye una restricción conceptual.

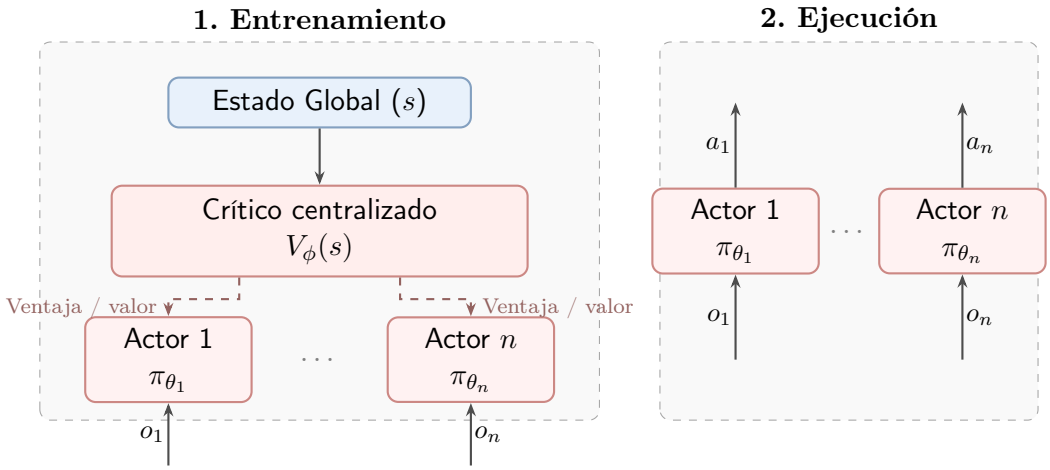


Figura 2.19: Paradigma de entrenamiento centralizado con ejecución descentralizada (CTDE). Durante el entrenamiento, el crítico utiliza información global privilegiada para estimar valores o ventajas que orientan la actualización de los actores locales; durante la ejecución, los actores solo requieren observaciones locales.

- **Crítico centralizado ($V_\phi(s)$):** durante el entrenamiento, el crítico puede utilizar información global que no está disponible para los actores. Una representación global adecuada puede facilitar la estimación del valor. Una estimación más precisa también puede reducir la variabilidad de las actualizaciones, aunque esto depende de la configuración [YVV+22].

El objetivo de optimización de los actores promedia los términos individuales de recorte PPO sobre los n agentes:

$$L^{\text{CLIP}}(\theta) = \hat{\mathbb{E}}_t \left[\frac{1}{n} \sum_{i=1}^{n} \min \left(r_t^i(\theta) \hat{A}_t(s_t, \mathbf{a}_t),\ \text{clip}(r_t^i(\theta), 1-\epsilon_{\text{clip}}, 1+\epsilon_{\text{clip}}) \hat{A}_t(s_t, \mathbf{a}_t) \right) \right] \tag{2.29}$$

donde $r_t^i(\theta) = \frac{\pi_{\theta_i}(a_t^i|o_t^i)}{\pi_{\theta_{i,\text{old}}}(a_t^i|o_t^i)}$ denota la razón de probabilidad local de cada agente y ϵ_{clip} es el margen de recorte. Paralelamente, el crítico centralizado se actualiza minimizando el error cuadrático frente al retorno objetivo R_t^{target}, estimado empíricamente sobre el lote de transiciones mediante la suma de la ventaja GAE y el valor de estado previo $\hat{A}_t(s_t, \mathbf{a}_t) + V_\phi(s_t)$ [YVV+22, SML+16]:

$$\mathcal{L}^{\text{VF}}(\phi) = \hat{\mathbb{E}}_t \left[\left(V_\phi(s_t) - R_t^{\text{target}} \right)^2 \right]. \tag{2.30}$$

Factores críticos de diseño y optimización en MAPPO

Yu et al. [YVV+22] analizaron empíricamente diversos factores de implementación e hiperparámetros que afectan el rendimiento de MAPPO:

1. **Normalización de retornos:** en los experimentos de Yu et al., normalizar los objetivos de valor puede favorecer el ajuste del crítico cuando cambia la escala de

las recompensas.

2. **Estructuración del estado global del crítico:** en ese estudio, la entrada s se evaluó con distintas representaciones: concatenación directa de observaciones (CL : $s = (o_1, \ldots, o_n)$), estado global extraído directamente del simulador (EP), o estados aumentados específicos por agente ($AS_i = EP \cup o_i$), combinados opcionalmente con filtrado de redundancias (FP).

3. **Reutilización de datos:** la cantidad de *epochs* (K) y la división en *minibatches* regulan la reutilización del lote. Yu et al. observaron que una reutilización excesiva puede deteriorar el rendimiento, especialmente en tareas difíciles.

El algoritmo 3 especifica el ciclo computacional de MAPPO bajo el paradigma CTDE.

Algoritmo 3 Algoritmo Multi-Agent Proximal Policy Optimization (MAPPO)

Entrada: entorno multiagente, n agentes, horizonte de recolección, *epochs* K, γ, λ_{GAE} y ϵ_{clip}
Salida: parámetros entrenados de los actores θ y del crítico centralizado ϕ

Inicializar los parámetros de la política (actor) θ y del valor (crítico) ϕ
Inicializar el *buffer* de datos $\mathcal{D} \leftarrow \emptyset$
para iteración $= 1, 2, \ldots$ **hacer**
    Recolectar trayectorias *on-policy* para todos los agentes utilizando la política conjunta $\boldsymbol{\pi}_\theta$
    Calcular las ventajas conjuntas $\hat{A}_t(s_t, \mathbf{a}_t)$ mediante GAE a partir del crítico centralizado $V_\phi(s_t)$
    Calcular los retornos objetivos R_t^{target} normalizados para el crítico
    Almacenar las transiciones $(s_t, \mathbf{o}_t, \mathbf{a}_t, \hat{A}_t, R_t^{\text{target}})$ en $\mathcal{D}$
    para *epoch* $= 1$ hasta K **hacer**
        Muestrear *minibatches* de datos desde $\mathcal{D}$
        Actualizar la política maximizando el objetivo de recorte PPO por agente: $L^{\text{CLIP}}(\theta) = \hat{\mathbb{E}}_t \left[\frac{1}{n} \sum_{i=1}^{n} \text{mín} \left(r_t^i(\theta)\hat{A}_t,\ \text{clip}(r_t^i(\theta), 1 - \epsilon_{\text{clip}}, 1 + \epsilon_{\text{clip}})\hat{A}_t \right) \right]$
        Actualizar el crítico ϕ minimizando el error cuadrático medio del valor: $\mathcal{L}^{\text{VF}}(\phi) = \hat{\mathbb{E}}_t \left[\left(V_\phi(s_t) - R_t^{\text{target}} \right)^2 \right]$
    fin para
    Vaciar el *buffer* de datos $\mathcal{D} \leftarrow \emptyset$
fin para

Durante el entrenamiento, el crítico centralizado $V_\phi(s)$ procesa el estado global de la red vial para calcular ventajas conjuntas mediante GAE, guiando la optimización paralela de los actores locales π_{θ_i} con el objetivo recortado. Al finalizar cada iteración, el *buffer* $\mathcal{D}$ se descarta por completo para preservar el régimen *on-policy* del método. En la etapa de ejecución, el crítico se desacopla y cada agente actúa con autonomía computacional a partir de su vector local o_t^i. Los hiperparámetros de entrenamiento y la arquitectura de las redes se detallan en el capítulo 3.

La formulación anterior presenta la variante teórica con una ventaja conjunta. En la implementación de este trabajo, el crítico centralizado produce una salida por agente y los objetivos de GAE se calculan por agente a partir de la recompensa individual descrita en la sección 3.3.2; por tanto, la ecuación resume el paradigma CTDE y no debe leerse como una identidad exacta del código ejecutado.

2.3.6. Heterogeneous-Agent Proximal Policy Optimization (HAPPO)

En sistemas de control complejos, los agentes pueden presentar asimetrías físicas y operacionales: diferencias en la configuración de actuadores, distintos espacios de acciones o demandas regionales contrapuestas. En entornos homogéneos, compartir parámetros puede mejorar la eficiencia de aprendizaje; en entornos heterogéneos, esa práctica puede limitar la especialización. Además, las actualizaciones independientes no poseen una garantía general de convergencia debido al aprendizaje simultáneo no coordinado [KCW+22].

Kuba et al. y Zhong et al. propusieron algoritmos de aprendizaje por refuerzo multiagente para políticas heterogéneas, sin exigir que todos los agentes compartan parámetros. Esta línea se denomina aprendizaje por refuerzo con agentes heterogéneos (Heterogeneous-Agent Reinforcement Learning, HARL). Cada agente i puede mantener su propia arquitectura y parámetros θ^i, adaptados a su espacio de acciones local $\mathcal{A}_i$. En esta línea, optimización de política por región de confianza con agentes heterogéneos (*Heterogeneous-Agent Trust Region Policy Optimization*, HATRPO) y HAPPO emplean una actualización secuencial de los agentes. HAPPO sustituye la restricción KL de la formulación de región de confianza por un objetivo recortado. Zhong et al. lo encuadran en el marco de aprendizaje espejo con agentes heterogéneos (*Heterogeneous-Agent Mirror Learning*, HAML), que establece resultados de mejora monótona y convergencia bajo sus condiciones, entre ellas una recompensa compartida [KCW+22, ZKF+24].

Compartición de parámetros y políticas heterogéneas

Una distinción central entre MAPPO y HAPPO radica en la homogeneidad de las políticas y en el orden de actualización:

- **Compartición de parámetros:** en MAPPO se utiliza habitualmente cuando los agentes poseen espacios de observación y acción compatibles. Todos los agentes pueden compartir los mismos pesos $\theta_1 = \cdots = \theta_n = \theta$, aunque la formulación de MAPPO también puede implementarse sin esta restricción.
- **Políticas heterogéneas y actualización secuencial:** HAPPO permite que cada agente mantenga una política independiente $\pi_{\theta^i}(a_i \mid o_i)$ y actualiza los actores siguiendo una permutación. El lote de trayectorias de la iteración actual se reutiliza de forma limitada; esto no equivale a un *buffer* histórico de *experience sharing* como el empleado por algoritmos específicos de compartición de experiencia.

Lema de descomposición de la ventaja multiagente

La actualización simultánea de todos los agentes puede generar inestabilidad porque el cambio concurrente de las políticas altera tanto las distribuciones de acción como la distribución conjunta de las trayectorias. HAPPO aborda este problema mediante una actualización secuencial: en cada iteración se optimiza un agente por vez, condicionando cada paso a los cambios ya efectuados por los precedentes. Para sustentar este esquema, la ventaja conjunta de la red debe poder descomponerse en contribuciones marginales ordenadas [KCW+22].

Esta descomposición se formaliza en el lema de descomposición de la ventaja multiagente propuesto por Kuba et al. [KCW+22]. En cualquier juego markoviano cooperativo, para una política conjunta $\boldsymbol{\pi}$ y un orden arbitrario de los agentes $i_{1:n} = (i_1, \ldots, i_n)$, la ventaja conjunta de la red se descompone exactamente como la suma de las ventajas marginales de cada agente, condicionadas por las decisiones de los que lo preceden en la secuencia:

$$A^{\boldsymbol{\pi}}(s, \mathbf{a}) = \sum_{m=1}^{n} A^{\boldsymbol{\pi}}_{i_m}(s, \mathbf{a}_{i_{1:m-1}}, a_{i_m}) \tag{2.31}$$

donde la ventaja marginal del agente i_m se define como:

$$A^{\boldsymbol{\pi}}_{i_m}(s, \mathbf{a}_{i_{1:m-1}}, a_{i_m}) \doteq Q^{\boldsymbol{\pi}}_{i_{1:m}}(s, \mathbf{a}_{i_{1:m-1}}, a_{i_m}) - Q^{\boldsymbol{\pi}}_{i_{1:m-1}}(s, \mathbf{a}_{i_{1:m-1}}). \tag{2.32}$$

En la ecuación (2.32), $Q^{\boldsymbol{\pi}}_{i_{1:m}}(s, \mathbf{a}_{i_{1:m-1}}, a_{i_m})$ evalúa el retorno esperado fijando las acciones de los primeros m agentes de la secuencia, mientras que las decisiones de los agentes posteriores ($\mathbf{a}_{i_{m+1:n}}$) se promedian según sus probabilidades bajo la política vigente.

La deducción matemática original de este lema asume un juego markoviano estrictamente cooperativo con un retorno escalar compartido $R(s, \mathbf{a})$ para toda la red. En sistemas espaciales distribuidos de gran escala, como el control semafórico urbano, asignar una recompensa global idéntica a todas las intersecciones dificulta la asignación de crédito, extremo que Chu et al. respaldan con evidencia empírica en redes extensas [CWCL19], y encarece la estimación de las ventajas [ACS24]. En este trabajo se emplean señales locales y, en protocolos específicos, de vecindario inmediato. Esta decisión de implementación no conserva por sí sola los supuestos del lema original.

Bajo retornos desacoplados, la identidad aditiva de la ecuación (2.31) no se traduce en una equivalencia algebraica directa entre un único valor global y los retornos individuales; no obstante, el principio de optimización secuencial por bloques (tipo Gauss-Seidel) conserva su utilidad práctica para desacoplar el aprendizaje concurrente [ACS24, WZK+23].

La relevancia teórica de este lema radica en que es una identidad analítica exacta para cualquier juego cooperativo sin imponer restricciones de aditividad o de monotonía sobre la función de valor en la red de mezclado (a diferencia de las condiciones de factorización IGM de arquitecturas como redes de descomposición de valor (*Value-Decomposition Networks*,

VDN) [SLG+18] o red de mezcla de valores Q (*Q-Mixing Network*, QMIX) [RSDW+18]).

Esquema de actualización secuencial y factor de razón compuesto M

Para explotar analíticamente esta descomposición teniendo en cuenta los cambios de los agentes ya actualizados, HAPPO actualiza las políticas de los agentes de manera secuencial dentro de cada iteración k. En cada ciclo se sortea una permutación de los agentes $i_{1:n} \in \mathrm{Sym}(n)$ (donde $\mathrm{Sym}(n)$ representa el conjunto de todas las permutaciones posibles de los n agentes) para que ningún agente ocupe siempre la misma posición en la secuencia [KCW+22, WZK+23].

Cuando corresponde el turno del agente i_m, los agentes previos en la secuencia ($i_{1:m-1}$) ya han actualizado sus parámetros a $\theta_{k+1}^{i_j}$, mientras que los agentes posteriores ($i_{m+1:n}$) conservan aún sus parámetros previos $\theta_k^{i_j}$. Para guiar al agente i_m teniendo en cuenta la alteración de la distribución conjunta producida por los agentes ya actualizados, Kuba et al. [KCW+22] demostraron que no se requiere entrenar un crítico separado por cada subgrupo; la ventaja marginal puede computarse reponderando la ventaja conjunta $A^{\boldsymbol{\pi}_k}(s, \mathbf{a})$ mediante la productoria acumulada de las razones de probabilidad de los agentes ya actualizados:

$$M_{i_{1:m}}(s_t, \mathbf{a}_t) \doteq \left(\prod_{j=1}^{m-1} \frac{\pi_{\theta_{k+1}^{i_j}}^{i_j}(a_t^{i_j} \mid o_t^{i_j})}{\pi_{\theta_k^{i_j}}^{i_j}(a_t^{i_j} \mid o_t^{i_j})} \right) A^{\boldsymbol{\pi}_k}(s_t, \mathbf{a}_t) \tag{2.33}$$

donde $\pi_{\theta_{k+1}^{i_j}}^{i_j}$ es la política recién optimizada del agente i_j y $\pi_{\theta_k^{i_j}}^{i_j}$ es su versión previa. Para el primer agente ($m = 1$), la productoria es vacía y $M_{i_1}(s_t, \mathbf{a}_t) \equiv A^{\boldsymbol{\pi}_k}(s_t, \mathbf{a}_t)$. En la formulación algorítmica práctica, el término acumulado $M_{i_{1:m}}$ se mantiene en una variable de memoria M_{i_m} por cada agente, la cual se actualiza recursivamente conforme avanzan las optimizaciones precedentes. El factor $M_{i_{1:m}}$ actúa como un estimador de importancia que ajusta la ventaja conjunta al nuevo perfil de comportamiento sin requerir simulaciones adicionales en el entorno [KCW+22].

Así, la política del agente i_m se optimiza maximizando el objetivo sustituto recortado ponderado por $M_{i_{1:m}}$:

$$\begin{aligned} L^{\mathrm{HAPPO}}(\theta^{i_m}) = \hat{\mathbb{E}}_t \Big[\min \Big(& r_t^{i_m}(\theta^{i_m}) M_{i_{1:m}}(s_t, \mathbf{a}_t), \\ & \mathrm{clip}\big(r_t^{i_m}(\theta^{i_m}), 1 - \epsilon_{\mathrm{clip}}, 1 + \epsilon_{\mathrm{clip}}\big) M_{i_{1:m}}(s_t, \mathbf{a}_t) \Big) \Big] \end{aligned} \tag{2.34}$$

donde $r_t^{i_m}(\theta^{i_m}) = \frac{\pi_{\theta^{i_m}}^{i_m}(a_t^{i_m} | o_t^{i_m})}{\pi_{\theta_k^{i_m}}^{i_m}(a_t^{i_m} | o_t^{i_m})}$ representa la razón de probabilidad individual del agente.

Concluida la optimización de θ^{i_m}, se computa la razón post-optimización:

$$\rho_{\text{post}}^{i_m} \doteq \frac{\pi_{\theta_{k+1}^{i_m}}^{i_m}(a_t^{i_m} \mid o_t^{i_m})}{\pi_{\theta_k^{i_m}}^{i_m}(a_t^{i_m} \mid o_t^{i_m})} \tag{2.35}$$

y se propaga recursivamente hacia los agentes pendientes mediante:

$$M_{i_{1:m+1}}(s_t, \mathbf{a}_t) \leftarrow \rho_{\text{post}}^{i_m} \cdot M_{i_{1:m}}(s_t, \mathbf{a}_t). \tag{2.36}$$

Garantía de mejora monótona

El principio de mejora monótona en aprendizaje por refuerzo formaliza que la secuencia de retornos esperados de la política conjunta no disminuye entre ciclos sucesivos de entrenamiento, comportándose como una función monótonamente no decreciente respecto de la iteración k:

$$J(\boldsymbol{\pi}_{k+1}) \geq J(\boldsymbol{\pi}_k). \tag{2.37}$$

Para políticas heterogéneas, Kuba et al. [KCW+22] demostraron analíticamente esta propiedad para el algoritmo HATRPO bajo los supuestos del juego markoviano cooperativo con recompensa compartida y una actualización restringida por la divergencia KL. Resolver las restricciones de HATRPO en redes neuronales profundas requiere un costo de cómputo elevado. Por ello, HAPPO sustituye la cota formal de divergencia KL por el objetivo sustituto recortado CLIP de primer orden de la ecuación (2.34).

La garantía de HATRPO no se transfiere automáticamente al sustituir su restricción por un recorte. El análisis de HAPPO dentro de HAML establece garantías bajo las condiciones de ese marco [KCW+22, ZKF+24]. Su aplicación a un controlador semafórico requiere comprobar esas condiciones: las recompensas específicas de cada agente no definen por sí solas el retorno compartido del planteo cooperativo. La metodología delimita por ello el alcance de las garantías para la adaptación utilizada y evalúa empíricamente su desempeño. La Figura 2.20 resume este esquema de propagación secuencial.

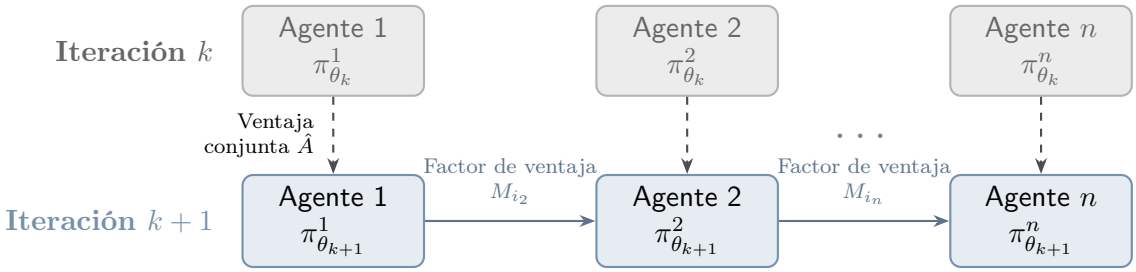


Figura 2.20: Esquema de actualización secuencial de HAPPO. A diferencia de PPO independiente, la actualización de cada agente está condicionada por el factor de ventaja reponderada M_{i_m}, que incorpora los cambios efectuados por los agentes que lo precedieron en la secuencia.

El algoritmo 4 presenta la secuencia completa de optimización de HAPPO.

Algoritmo 4 Algoritmo Heterogeneous-Agent Proximal Policy Optimization (HAPPO)

Entrada: entorno multiagente, conjunto $\mathcal{N}$ de n agentes, horizonte de recolección, γ, λ_{GAE} y ϵ_{clip}

Salida: políticas heterogéneas entrenadas $\{\theta^i\}_{i=1}^n$ y crítico centralizado ϕ

Inicializar los parámetros de las políticas individuales $\{\theta^i, \forall i \in \mathcal{N}\}$ y del crítico centralizado $V_\phi(s)$
para iteración $= 1, 2, \ldots$ **hacer**
    Recolectar un conjunto de trayectorias $\mathcal{D}$ utilizando la política conjunta $\boldsymbol{\pi}_\theta$
    Calcular el estimador de ventaja conjunta $\hat{A}(s, \mathbf{a})$ con GAE a partir del crítico $V_\phi(s)$
    Calcular los retornos objetivo R_t^{target} utilizados para ajustar el crítico
    Sortear una permutación de los agentes $i_{1:n} \in \text{Sym}(n)$
    Inicializar los factores $M_{i_m}(s, \mathbf{a}) \leftarrow \hat{A}(s, \mathbf{a})$ para todo $m \in \{1, \ldots, n\}$
    para $m = 1 \ldots n$ **hacer**
        Obtener el agente actual i_m y su factor modificador $M_{i_m}(s, \mathbf{a})$
        Actualizar θ^{i_m} a $\theta_{k+1}^{i_m}$ maximizando el objetivo recortado de la ecuación (2.34)
        Recalcular la razón post-optimización: $\rho_{\text{post}}^{i_m} \leftarrow \frac{\pi_{\theta_{k+1}^{i_m}}^{i_m}(a_t^{i_m}|o_t^{i_m})}{\pi_{\theta_k^{i_m}}^{i_m}(a_t^{i_m}|o_t^{i_m})}$
        para $j = m + 1 \ldots n$ **hacer**
            Actualizar el factor modificador para agentes posteriores: $M_{i_j}(s_t, \mathbf{a}_t) \leftarrow \rho_{\text{post}}^{i_m} \cdot M_{i_j}(s_t, \mathbf{a}_t)$
        fin para
    fin para
    Actualizar el crítico centralizado ϕ mediante descenso de gradiente: $\mathcal{L}^{\text{VF}}(\phi) = \hat{\mathbb{E}}_t \left[\left(V_\phi(s_t) - R_t^{\text{target}} \right)^2 \right]$
fin para

El procedimiento de HAPPO organiza secuencialmente la actualización de los actores locales mediante la permutación aleatoria $i_{1:n}$ y la propagación recursiva de los factores modificadores M_{i_m}. Tras optimizar sucesivamente a los n agentes sobre el lote de transiciones $\mathcal{D}$, el crítico centralizado V_ϕ ajusta sus parámetros mediante descenso de gradiente sobre los retornos acumulados. Durante la etapa de ejecución descentralizada, cada política local selecciona a_i a partir de o_i, sin requerir evaluar el crítico centralizado ni comunicación entre agentes para seleccionar su acción [KCW+22].

La adaptación para control semafórico de este trabajo conserva la actualización sucesiva de los actores, pero inicializa los factores M_{i_m} con las ventajas individuales $\hat{A}_i(s, \mathbf{a})$ calculadas mediante el crítico centralizado multi-salida a partir de las señales de recompensa individuales (Sección 3.3.2), propagándolos mediante la razón $\rho_{\text{post}}^{i_m}$ tras optimizar a

cada actor. A diferencia del Algoritmo 4, la implementación actualiza el crítico al comienzo de cada época, antes de los actores (Sección 3.3.3).

Esta adaptación utiliza ventajas individuales, mientras que la demostración teórica parte de una recompensa global compartida. Su efecto sobre el aprendizaje debe comprobarse mediante los experimentos; esta adaptación no hereda por sí sola la garantía teórica de mejora.

Para finalizar el estudio comparativo de los algoritmos derivados de PPO en escenarios MARL, la Tabla 2.3 sintetiza sus principales propiedades operativas, arquitectónicas y teóricas.

Tabla 2.3: Comparación sintética de familias algorítmicas PPO en entornos multiagente cooperativos.

Eje de análisis	**PPO de agente único**	**IPPO independiente**	**MAPPO / CTDE**	**HAPPO / HARL**
Paradigma	agente único	IL (descentralizado)	CTDE (centralizado/desc.)	CTDE; familia HARL
Actores	único $\pi_\theta(a \mid s)$	n políticas locales	actores locales; puede compartir parámetros	n políticas heterogéneas
Arquitectura del crítico	único $V_\phi(s)$	n críticos locales $V_{\phi_i}(o_i)$	un crítico centralizado $V_\phi(s)$	un crítico centralizado $V_\phi(s)$
Optimización	PPO estándar	PPO local simultáneo	PPO local con crítico global	actualización secuencial $i_{1:n}$
Señal de ventaja	ventaja local $\hat{A}(s, a)$	ventaja local $\hat{A}_i(o_i, a_i)$	ventaja conjunta $\hat{A}(s, \mathbf{a})$	factor compuesto $M_{i_{1:m}}(s, \mathbf{a})$
Mejora monótona	No estricta para PPO	Sin garantía general	Sin garantía estricta	Bajo las condiciones de HAML
Tipo de agente	No aplica	Depende de la implementación	Pensado para agentes homogéneos; la compartición de parámetros es opcional	Pensado para agentes heterogéneos

2.3.7. Antecedentes recientes del control semafórico multiagente

Para orientar la comparación, los antecedentes seleccionados se organizan según tres ejes: coordinación y entrenamiento, representación de las observaciones, y tratamiento de la heterogeneidad o la equidad entre intersecciones. Se priorizan investigaciones directamente relacionadas con la formulación de este trabajo.

El control coordinado de múltiples intersecciones mediante aprendizaje por refuerzo constituye un área de investigación activa. Entre los enfoques iniciales se encuentran aquellos que asignaban un agente a cada semáforo y utilizaban información local sobre colas, fases o vehículos detenidos. Sin embargo, la optimización aislada de cada intersección no contempla que una acción modifica el flujo recibido por los cruces vecinos. Por este motivo, la literatura evolucionó hacia formulaciones cooperativas que incorporan mecanismos de coordinación espacial, representaciones estructuradas del estado y esquemas de entrenamiento centralizado [ACS24, LML+25].

Coordinación y entrenamiento

En la consolidación del control semafórico coordinado mediante aprendizaje profundo, varios trabajos abordaron de manera temprana la asignación de crédito y la no estacionariedad en redes viales:

- **Control actor-crítico con descuento espacial (MA2C):** Chu et al. [CWCL19] propusieron una arquitectura de ventaja actor-crítico multiagente donde cada intersección incorpora a su función de valor y recompensa la información de sus vecinos inmediatos atenuada por un factor de descuento espacial. Este trabajo fundamentó conceptualmente y evidenció de manera empírica que una recompensa de equipo global en redes extensas induce severos problemas de asignación de crédito, mientras que las recompensas regionalmente estructuradas permiten coordinar corredores sin requerir estados globales inmanejables.

- **Coordinación basada en presión vehicular (PressLight):** Wei et al. [WCZ+19] fundamentaron su función de recompensa en el principio de *Max-Pressure* de la ingeniería de transporte, que en la teoría de control de redes garantiza estabilidad de cola bajo supuestos específicos de un esquema de servicio discreto. PressLight incorpora ese principio mediante una recompensa igual al negativo de la presión de la intersección, calculada a partir de las densidades vehiculares de entrada y salida. Esa incorporación no hereda la garantía formal de estabilidad y coordina de manera implícita las intersecciones adyacentes sin necesidad de comunicación explícita.

- **Atención en grafos para cooperación a nivel de red (CoLight):** Wei et al. [WXZ+19] introdujeron redes de atención en grafos para permitir que los agentes

semafóricos intercambien dinámicamente representaciones de sus estados con cruces vecinos, adaptando la influencia de cada intersección adyacente según las condiciones variables de congestión.

- **Límites del aprendizaje independiente en redes viales:** Ault y Sharon [AS22] analizaron las limitaciones de los enfoques independientes frente a formulaciones cooperativas en redes de múltiples intersecciones, evidenciando un deterioro progresivo en el desempeño y en la fluidez del tránsito a medida que la demanda se incrementa debido a la falta de coordinación y a la no estacionariedad del entorno.

A partir de estas bases fundacionales, diversas investigaciones recientes han explorado mecanismos alternativos de optimización y distribución de cómputo. Wu et al. [WWS+22] estudian el control de redes de gran escala mediante agentes distribuidos de aprendizaje por refuerzo profundo y una arquitectura de cómputo distribuido. Proponen dos variantes, Nash-A2C y Nash-A3C, que incorporan un equilibrio de Nash en el aprendizaje por refuerzo y se despliegan en una arquitectura distribuida para simulación y control. En sus experimentos reportan reducciones del tiempo de congestión y de la demora total de red frente a controladores de referencia. Este antecedente es pertinente como referencia de escalabilidad, pero no es equivalente a la formulación de este trabajo: aquí la ejecución también es local por intersección, mientras que la coordinación se estudia mediante el entrenamiento centralizado y, en las variantes vecinales, la combinación de señales de la intersección y sus vecinos, sin introducir una capa de cómputo fog ni un equilibrio de Nash.

En una formulación cooperativa basada en DQN, Haddad et al. [HHA22] incorporan a la función de aprendizaje de cada intersección información sobre los estados, acciones y recompensas de sus vecinos. El estudio informa mejoras en tiempo de espera, longitud de cola y emisiones de dióxido de carbono frente a controladores de referencia. Esta evidencia respalda la necesidad de considerar el efecto de las decisiones vecinas, pero su mecanismo de coordinación es una transferencia explícita de información hacia la pérdida de DQN. En el protocolo de búsqueda con dominios reducidos de este estudio, IPPO e IDQN implementan el paradigma de aprendizaje independiente y reciben una señal de recompensa estrictamente local ($R_i = L_i$, $\rho_{\text{coop}} = 0$); MAPPO y HAPPO usan también recompensa local en la búsqueda refinada, con crítico centralizado durante el entrenamiento. La mezcla vecinal se evalúa en una línea separada de MAPPO y no se atribuye a todos los algoritmos. Estas alternativas no son equivalentes al mecanismo propuesto por Haddad et al., sino procedimientos de coordinación distintos que se comparan bajo una misma representación y escenario.

Fang et al. [FYX+23] proponen un método de coordinación de red que formula el control semafórico como un problema multiobjetivo: combina eficiencia, seguridad y coordinación espacial entre intersecciones mediante un esquema de entrenamiento centralizado

con ejecución descentralizada. Además de reportar resultados en redes sintéticas y arterias reales, el trabajo destaca que las dependencias espacio-temporales entre agentes deben representarse explícitamente. Su propuesta ofrece un contraste con la señal individual compuesta de las variantes vecinales y la representación interpretable utilizadas en este trabajo, que no incorporan una optimización multiobjetivo ni una representación latente basada en atención.

Representación del estado

En cuanto a la representación agregada y el aprendizaje basado en valor, Kolat et al. [KKBA23] proponen un enfoque DQN multiagente con parámetros compartidos. Los agentes observan cantidades escaladas de vehículos detenidos y reciben una recompensa que penaliza el desequilibrio entre los accesos. Los resultados muestran mejoras en el tiempo de viaje y el consumo de combustible respecto de un controlador de ciclo fijo. Sirve como referencia para contrastar una representación agregada del estado y una coordinación inducida por la recompensa con la evaluación de IDQN y de métodos basados en políticas realizada en esta investigación.

Dentro de la línea centrada en la representación del estado, Yang [Yan23] propone HG-M2I, una arquitectura que combina información espacial y temporal mediante un grafo jerárquico. El modelo integra estados de los agentes y de sus vecinos en representaciones latentes utilizadas por las políticas. Posteriormente, Chen et al. [CYL+24] presentan CoevoMARL, que utiliza atención en grafos y una unidad recurrente para aprender cómo varía en el tiempo la dependencia entre intersecciones. Mientras HG-M2I organiza la información según una jerarquía espacial, CoevoMARL adapta las relaciones entre agentes a la evolución del tránsito. Ambos trabajos muestran la importancia de construir observaciones informativas, aunque recurren a representaciones latentes aprendidas, a diferencia de la transformación explícita e interpretable que se evalúa aquí.

En el caso de una representación basada en celdas, Deng et al. [DLSZ26] introducen una partición de longitud variable. Los accesos se dividen mediante una función que combina componentes logarítmicos y lineales, y cada celda almacena la cantidad de vehículos, la velocidad media y la ocupación espacial. Esta partición asigna mayor detalle a las zonas próximas a la intersección y comprime las regiones más alejadas. Entre los trabajos seleccionados, es el antecedente más cercano al uso de una transformación logarítmica en la representación semafórica; sin embargo, el logaritmo determina la longitud espacial de las celdas y no codifica el tiempo de espera acumulado por carril.

Heterogeneidad y equidad

Ren et al. [RDZ+24] abordan la coordinación bajo demandas heterogéneas mediante un esquema de dos capas. Cada agente combina su ventaja local con la de las intersecciones vecinas a través de un factor de cooperación adaptable, que luego se ajusta según el rendimiento global de la red. El método considera la demora y las emisiones vehiculares, y

coordina los objetivos locales y globales mediante ese factor adaptable. Este antecedente permite contrastar la coordinación adaptativa de las señales de aprendizaje con la heterogeneidad de políticas considerada por HAPPO, aunque no emplea su procedimiento de actualización secuencial.

En la coordinación mediante entrenamiento centralizado, Liu et al. [LZF+25] formulan el control semafórico como un juego parcialmente observable con restricciones y proponen VF-MAPPO. En este método, la equidad se expresa mediante una cota sobre el tiempo máximo de espera de los vehículos, mientras que un crítico centralizado con atención espaciotemporal estima tanto la eficiencia de la red como el cumplimiento de la restricción. Esta formulación resulta especialmente pertinente para esta investigación porque vincula con una variable temporal de operación el paradigma de entrenamiento centralizado con ejecución descentralizada. No obstante, el tiempo de espera se incorpora como restricción de optimización y no mediante una codificación no lineal de la observación.

En el tratamiento de agentes heterogéneos, otro enfoque es HAPS-PPO, presentado por Lu et al. [LFYZ25]. El método completa con relleno las observaciones de distinta dimensión y agrupa a los agentes según sus espacios de acción, asignando cabezales de política específicos a cada grupo. Esto permite coordinar intersecciones con geometrías y conjuntos de fases diferentes.

Posicionamiento del proyecto

La selección analizada sugiere que el desempeño del control semafórico multiagente depende de tres decisiones relacionadas: el algoritmo utilizado para aprender y coordinar las políticas, la representación de las condiciones de operación y el tratamiento de intersecciones con demandas, geometrías o espacios de acción diferentes. Los enfoques basados en DQN proporcionan una referencia para el aprendizaje por valor. Los modelos de grafos construyen representaciones latentes de las relaciones espaciales, mientras que las variantes de MAPPO incorporan información global durante el entrenamiento. Los esquemas de cooperación adaptable o con múltiples cabezales atienden distintas formas de heterogeneidad. La codificación por carril y la discretización de variables de tránsito también cuentan con antecedentes en control semafórico [GR18]; aquí se emplean para representar la espera acumulada por carril mediante la transformación especificada en la metodología.

Entre los antecedentes considerados, HeteroLight aborda la heterogeneidad mediante una arquitectura de parámetros compartidos con módulos que extraen características generales y específicas de cada intersección [ZLFS24], mientras que MATLight se apoya en un marco de aprendizaje especular para agentes heterogéneos (*heterogeneous-agent mirror learning*) y emplea un Transformer para procesar secuencias de estados [ZWL24]. Son mecanismos distintos de la adaptación HAPPO implementada aquí: no se los presenta como equivalentes ni como evidencia de que HAPPO sea superior. Entre las fuentes

consultadas no se localizó una evaluación conjunta de IDQN, IPPO, MAPPO y HAPPO bajo una misma formulación, ni una aplicación directa del HAPPO original al control de fases semafóricas urbanas. Esta observación delimita la búsqueda realizada y no permite concluir que tales estudios no existan.

En particular, este trabajo explora una codificación híbrida logarítmica-lineal del tiempo de espera acumulado por carril. Al asignar valores continuos a un conjunto finito de intervalos, la cuantización agrupa esperas dentro de una misma categoría y limita los valores representables; la saturación también reúne los valores que alcanzan la cota. En aprendizaje por refuerzo, una abstracción de estados puede simplificar el problema y conservar comportamiento cercano al óptimo cuando los estados agrupados son suficientemente similares, bajo las condiciones teóricas correspondientes [AHL16]. Ese resultado no constituye una garantía para esta codificación: aquí se transforman observaciones locales en un problema parcialmente observable y no se demuestra que los valores agrupados sean equivalentes para la política óptima. La resolución no uniforme del codificador asigna más niveles a esperas bajas, lo que permite distinguirlas con mayor detalle; es una hipótesis de diseño, pues la cuantización también puede descartar distinciones relevantes, en particular cerca de los umbrales entre intervalos.

Su efecto se examina como parte del sistema completo, sin un experimento que compare el mismo sistema con y sin esa transformación. Esta propuesta se diferencia de las observaciones agregadas utilizadas por los enfoques DQN, de las proyecciones latentes basadas en grafos y de la partición espacial de Deng et al. [DLSZ26], que divide los accesos y no codifica la espera acumulada por carril.

A su vez, el diseño experimental contempla la búsqueda de configuraciones de IDQN, IPPO, MAPPO y HAPPO sobre una misma representación local de observación. En MAPPO se utiliza un crítico centralizado compartido de múltiples salidas, condicionado por la concatenación de observaciones locales, con valores y ventajas calculados por agente. En HAPPO, las ventajas individuales estimadas mediante GAE se reponderan en las actualizaciones secuenciales de los actores; esta adaptación no aplica directamente el lema original de descomposición de ventajas. Los críticos estiman valores durante el entrenamiento en ambos procedimientos, pero no se atribuye al crítico la diferencia entre ellos ni se postula una actualización secuencial del crítico.

Dado que la comparación se basa en procedimientos completos, no es posible aislar el efecto directo de un solo elemento, como la actualización secuencial o el crítico centralizado. Determinar dicho impacto requeriría una retirada de componentes, evaluando el sistema sin esa pieza específica mientras los demás factores permanecen constantes. De este modo, la solución puede contrastarse con los antecedentes recientes en términos de representación, paradigma de entrenamiento y capacidad para tratar agentes heterogéneos. El capítulo 3 describe cómo se concreta esta propuesta en el diseño experimental.

3

Metodología

El problema consiste en aprender políticas para controlar los semáforos de una red de tráfico simulada. Cada agente recibe observaciones del tránsito en su intersección y elige una fase permitida. Se examina la espera en los carriles que controla cada agente y, a nivel de la red vial, la espera por viaje, la atención a la demanda y la distribución de la espera entre agentes.

El diseño compara implementaciones de MAPPO y una adaptación de HAPPO, con IPPO y IDQN como esquemas de aprendizaje independiente de referencia. La actualización secuencial de los actores de HAPPO motiva su contraste con MAPPO a lo largo de tres redes viales sintéticas de complejidad creciente —un corredor arterial (3×1), una cuadrícula regular homogénea (3×3) y una cuadrícula heterogénea ($4 \times 4_H$)— además de la preparación de una red urbana del microcentro de la Ciudad de Mendoza.

El trabajo se organiza en dos líneas metodológicas complementarias. La primera establece un protocolo común en dos fases para comparar los cuatro algoritmos en las redes sintéticas mediante la búsqueda de hiperparámetros, la selección de configuraciones admisibles y el entrenamiento desde cero de las políticas finales, dejando preparada la base operativa para su futura extensión al escenario de Mendoza. La segunda línea analiza específicamente en MAPPO el efecto de incorporar información vecinal en la observación y en la recompensa dentro de las redes sintéticas.

En las secciones siguientes se describen primero los escenarios de simulación y la formulación del sistema de control multiagente; a continuación se presentan el protocolo de búsqueda y selección de configuraciones, el diseño de comparación entre variantes locales y vecinales, y las métricas e instrumentos de evaluación estadística.

3.1. Diseño experimental

En las tres redes sintéticas, el ajuste de hiperparámetros se estructura en dos fases sucesivas: la fase 1 explora dominios amplios y la fase 2 refina los intervalos de búsqueda a partir de las configuraciones de mejor desempeño en cada escenario. Mientras que la búsqueda genera configuraciones candidatas, una etapa posterior de selección filtra aquellas que cumplen criterios de admisibilidad operacional antes de entrenar desde cero las políticas definitivas. En el escenario urbano de Mendoza se documenta la construcción de la red, de la demanda nominal de 220.000 viajes diarios y del plan de tiempos fijos, junto con su comprobación operacional; la ejecución del ciclo de aprendizaje sobre este dominio queda planteada como trabajo futuro una vez adaptados sus codificadores y presupuestos de cómputo.

En paralelo, entre ambas fases de búsqueda sintética se desarrolla la segunda línea experimental manteniendo fija la configuración de aprendizaje. Esta línea comprende dos etapas: primero, una exploración preliminar de cinco combinaciones de observación y recompensa (variantes A–E) con MAPPO en la red heterogénea $4 \times 4_H$ (Sección 3.5.1); segundo, una comparación pareada en las tres redes sintéticas entre la formulación puramente local (variante A) y la formulación con observación y recompensa vecinales acopladas (variante C), detallada en la Sección 3.5.

La Figura 3.1 sintetiza los dos recorridos del diseño experimental y sus dependencias funcionales. El panel (a) representa el protocolo de búsqueda en dos fases, selección de configuraciones, entrenamiento final y evaluación de los cuatro algoritmos en las redes sintéticas, junto con la etapa prevista de extensión al escenario urbano. El panel (b) muestra la línea de análisis de acoplamiento espacial en MAPPO, compuesta por la prueba preliminar A–E en $4 \times 4_H$ y la comparación pareada A/C en las tres redes sintéticas. El resultado de esta segunda línea motiva la adopción de la formulación local en la fase 2 de búsqueda.

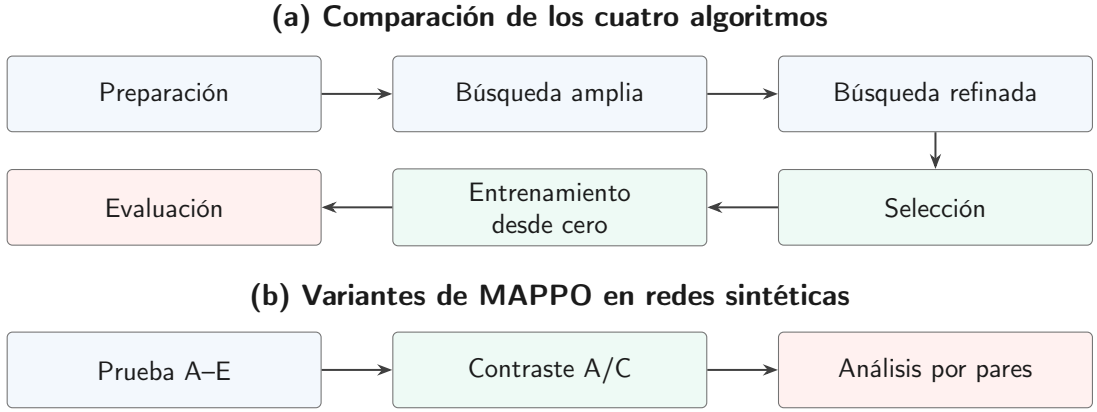


Figura 3.1: Etapas y dependencias funcionales del diseño experimental: (a) ajuste y comparación de algoritmos; (b) contraste de formulaciones local y vecinal en MAPPO.

La métrica primaria de desempeño vehicular es la espera media acumulada hasta el cierre del horizonte por los vehículos que ingresan a la red durante la ventana de evaluación, incluidos aquellos cuyos viajes permanecen en curso al finalizar el episodio. Este indicador se analiza conjuntamente con el volumen de vehículos insertados y la tasa de viajes completados para verificar que una reducción en la espera no responda a una restricción artificial del flujo admitido en los bordes de la red [JLW+24]. Las definiciones formales de todas las métricas y poblaciones se presentan en la Sección 3.6.1.

3.2. Escenarios de simulación

La geometría de cada escenario determina los carriles, las fases y las relaciones de vecindad entre agentes, mientras que la demanda define los viajes que deben atender. Los cuatro escenarios se simulan en SUMO [ALBBW+18, DLR26]. A continuación se describen sus condiciones de simulación y la construcción del escenario urbano.

3.2.1. Preparación de las redes de tráfico

Para cada red de tráfico se definieron calzadas, carriles por aproximación y longitudes de almacenamiento mediante `netedit` en los dominios sintéticos y a partir de cartografía de OSM en el escenario de Mendoza. Los programas semafóricos de tiempo fijo incluidos en las redes sintéticas se conservaron como línea base de referencia (Tabla 3.1), mientras que para Mendoza se construyó un plan coordinado de tiempo fijo descrito en la Sección 3.2.3 y detallado en la Sección A.4 del Apéndice A.

La Tabla 3.1 resume las secuencias cronológicas de fases de los programas de referencia sintéticos en ciclos de 90 segundos, cuya representación gráfica se incluye en la Figura A.4 del Apéndice A. En la notación empleada, V_i indica la duración de verde de la fase i, A_i el intervalo de cambio amarillo (3 s) y R el intervalo de todo rojo o despeje (1 s). En las redes sintéticas, las intersecciones se nombran combinando la columna (letras B a E de oeste a este) y la fila (dígitos 1 a 4 de sur a norte), como se ilustra en la Figura 3.2; así, C1, C4 y D1 designan tres cruces específicos de la red $4 \times 4_H$ con secuencias diferenciadas. Las restricciones temporales del control aprendido se detallan en la Sección 3.4.1.

Tabla 3.1: Duraciones de los programas semafóricos de referencia en las redes de tráfico sintéticas (segundos).

Red de tráfico e intersecciones	**Secuencia cronológica de fases e intervalos**	**Ciclo**
3×1: todas (3)	$V_1 = 42,\ A_1 = 3,\ V_2 = 42,\ A_2 = 3$	90 s
3×3: todas (9)	$V_1 = 42,\ A_1 = 3,\ V_2 = 42,\ A_2 = 3$	90 s
$4 \times 4_H$: 13 intersecciones con secuencia común	$V_1 = 42,\ A_1 = 3,\ V_2 = 42,\ A_2 = 3$	90 s
$4 \times 4_H$: C1	$V_1 = 38,\ A_1 = 3,\ V_2 = 37,\ A_2 = 3,$ $V_3 = 6$ (giro protegido), $A_3 = 3$	90 s
$4 \times 4_H$: C4 y D1	$V_1 = 42,\ A_1 = 3,\ V_2 = 41,\ A_2 = 3,$ $R = 1$ (todo rojo)	90 s

3.2.2. Redes de tráfico sintéticas

Todas las redes de tráfico sintéticas comparten las mismas reglas operacionales. El límite de velocidad vial es 13,89 metros por segundo (50 kilómetros por hora), incluso si el tipo de vehículo admite una velocidad mayor. La flota del archivo `urban_vehicle_mix.add.xml` combina tres perfiles, resumidos en la Tabla 3.2. Las variables a_{veh} y d_{veh} denotan la aceleración y la desaceleración, en metros por segundo cuadrado; $v_{\text{máx}}$ es la velocidad máxima, en metros por segundo [DLR26].

Tabla 3.2: Perfiles de vehículos de las redes de tráfico sintéticas.

Perfil (tipo SUMO)	**Proporción**	$\boldsymbol{a_{\text{veh}}}$ **(m/s²)**	$\boldsymbol{d_{\text{veh}}}$ **(m/s²)**	$\boldsymbol{v_{\text{máx}}}$ **(m/s)**
Lento (`SLOWVEHTYPE`)	25 %	1,9	4,0	8,33
Estándar (`MEDVEHTYPE`)	60 %	2,6	4,5	13,89
Rápido (`FASTVEHTYPE`)	15 %	3,1	5,0	19,44

Los vehículos miden 4,5 metros y mantienen una separación mínima de 2,5 metros (`minGap`). Se utiliza el modelo de seguimiento de Krauss, que determina cómo un vehículo ajusta su velocidad respecto del que circula delante. Su parámetro adimensional $\sigma = 0{,}5$ regula la variabilidad aleatoria de la conducción [Kra98, DLR26].

Las rutas se generaron con `random_routes_taz.py` a partir de matrices origen-destino (*Origin-Destination*, OD), que distribuyen el flujo entre las zonas TAZ perimetrales de cada red. Los archivos expresan la demanda solicitada en vehículos por hora mediante el atributo `vehsPerHour` de SUMO. La construcción de las matrices y las asignaciones principales se detallan en la Sección A.5.3 del Apéndice A.

La Figura 3.2 compara las tres redes de tráfico mediante geometrías extraídas de sus archivos XML. Los puntos identifican intersecciones controladas y las flechas indican sentidos de circulación. Cada calzada se divide en franjas que representan sus carriles; los

sentidos opuestos se dibujan a ambos lados del eje vial, con escalas y anchos convencionales por panel. Las zonas TAZ quedan definidas por los tramos de ingreso y egreso del perímetro. Mientras que las redes 3×1 y 3×3 poseen cuatro accesos por cruce y dos carriles por sentido, la red $4 \times 4_H$ combina tramos de entre uno y cuatro carriles por sentido e incorpora arterias de mano única:

- **Panel (a) — Corredor 3×1:** consta de tres cruces alineados (B1, C1 y D1). El 75 % del flujo solicitado se concentra en ambos sentidos de la arteria horizontal y el 25 % restante se reparte entre los pares transversales válidos, lo que permite analizar la propagación longitudinal de colas y la coordinación sobre una corriente dominante.

- **Panel (b) — Cuadrícula 3×3:** dispone de nueve cruces en una grilla regular con dimensiones locales uniformes de observación y acción ($m_i = 8$, $K_i = 2$), orientada a evaluar el acoplamiento bidimensional entre intersecciones interiores y perimetrales.

- **Panel (c) — Cuadrícula heterogénea $4 \times 4_H$:** abarca dieciséis cruces organizados en las columnas B a E (oeste a este) y las filas 1 a 4 (sur a norte), con ausencia deliberada de los tramos horizontales B4–C4 y D1–E1. La fila 2 conforma una arteria de tres carriles por sentido, mientras que el acceso oeste hacia B1 y el tramo B1–C1 poseen un solo carril por sentido; la columna C cuenta con cuatro carriles hacia el sur y dos hacia el norte, y la columna D opera en sentido único hacia el norte. Asimismo, la intersección C1 (destacada con un rombo) admite tres acciones ($K_{\mathrm{C1}} = 3$ por incluir fase de giro protegido), frente a las dos acciones de los quince cruces restantes.

Las redes sintéticas tienen una separación nominal de 100 metros entre centros de cruces consecutivos. En 3×3 y $4 \times 4_H$, la longitud efectiva de almacenamiento de cada tramo es ligeramente menor porque los carriles de aproximación finalizan en el borde de la caja interna de la intersección. Para caracterizar las redes de manera uniforme se contabilizan únicamente los tramos de circulación y sus carriles de aproximación y salida, excluyendo los conectores internos con los que SUMO modela las trayectorias dentro del cruce.

El número de carriles entrantes controlados por el agente i se denota por m_i, y el de fases seleccionables, por K_i. El grado de vecindad es la cantidad de vecinos directos de un agente; el criterio de conexión se precisa en la Sección 3.3.1. La Tabla 3.3 reúne la topología, la demanda y las dimensiones del control; los intervalos describen la variación entre agentes de una misma red.

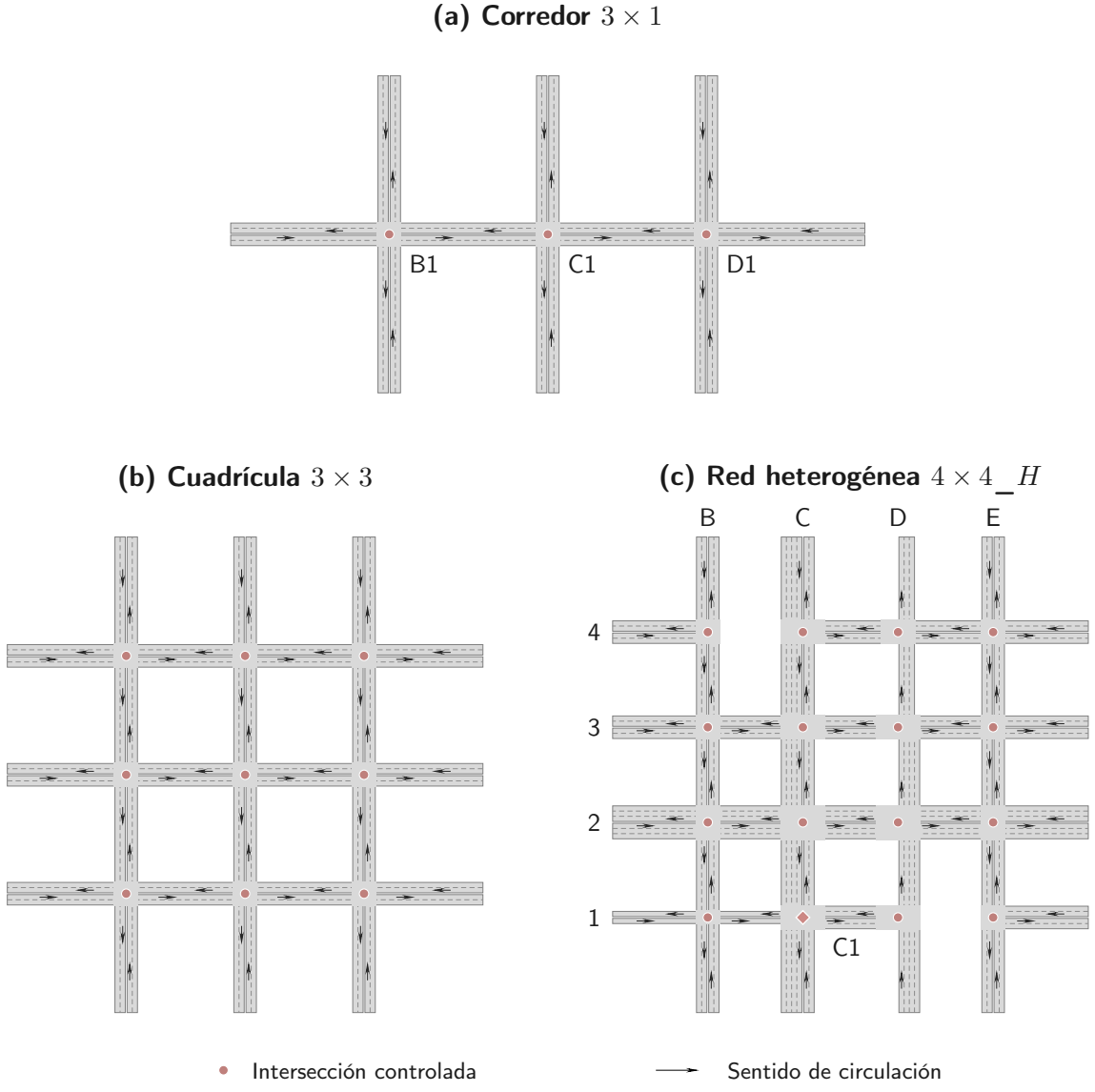


Figura 3.2: Redes sintéticas: (a) corredor 3×1, (b) cuadrícula 3×3 y (c) red heterogénea $4 \times 4_H$.

Tabla 3.3: Topología, control y demanda de las redes sintéticas. Se excluyen los tramos y carriles internos de los cruces.

Característica	**3 × 1**	**3 × 3**	**4 × 4_*H***
Agentes	3	9	16
Tramos de circulación	20	48	71
Carriles	40	96	166
Carriles por tramo y sentido	2	2	1–4
Carriles entrantes controlados (m_i)	8	8	5–12
Acciones por agente (K_i)	2	2	2 o 3
Grado de vecindad	1–2	2–4	1–4
Zonas de análisis de tránsito	8	12	16
Flujo solicitado (vehículos por hora)	3.260	7.087	7.202

Las vías y las asignaciones por par OD de las tres redes se detallan en la Sección A.5.3 del Apéndice A.

3.2.3. Red de tráfico de la Ciudad de Mendoza

Con el propósito de contar con un entorno urbano realista de alta demanda y topología irregular, se construyó y verificó operacionalmente un modelo microscópico del microcentro de la Ciudad de Mendoza. El escenario resultante consolida una demanda nominal de 220.000 viajes diarios distribuida entre 39 zonas TAZ y operada mediante 73 controladores semafóricos de tiempo fijo con ciclo común de 90 segundos. En el marco de esta tesina, dicho escenario se somete a una comprobación operacional bajo control de tiempo fijo que sienta la línea base para futuras campañas de aprendizaje multiagente.

Dominio y datos disponibles

La red vial se extrajo inicialmente de OSM, se proyectó en coordenadas del sistema de coordenadas universal transversal de Mercator (*Universal Transverse Mercator*, UTM) (zona 19S, referencia geodésica Sistema Geodésico Mundial 1984 (*World Geodetic System 1984*, WGS84)) y se depuró mediante imágenes satelitales y relevamientos de vista de calle para corregir sentidos de circulación, número de carriles por aproximación, conectores de giro e ingresos laterales en arterias como Catamarca, Buenos Aires y Leandro N. Alem. El área modelada abarca los corredores de José Vicente Zapata y Colón al sur, Manuel Belgrano al oeste, Las Heras al norte y la avenida General San Martín como eje estructurante norte–sur. La topología final comprende 255 tramos de circulación, 554 carriles (incluidos los internos de cruce), 752 conexiones y 74 nodos semaforizados agrupados en 73 controladores; sobre ella se definen 39 zonas TAZ con 520 pares OD físicamente conectables, de los cuales 517 registran flujo activo.

Para caracterizar la variación horaria de las solicitudes se utilizaron registros de cámaras termográficas ubicadas en Costanera y en los accesos Este y Sur. En particular, la

curva temporal de demanda toma como referencia la cámara 169 —asociada al ingreso este por avenida José Vicente Zapata (‘TAZ 28‘)—, cuyo registro abarca 32 días hábiles completos con un volumen medio de 25.175,84 vehículos diarios. Dado que un conjunto acotado de conteos de borde no basta por sí solo para identificar unívocamente una matriz origen–destino urbana sin información adicional de asignación [vZW80], este perfil empírico se utilizó para modular la distribución temporal de los arribos, mientras que la asignación espacial entre pares OD se estructuró priorizando los corredores principales de entrada y salida. La cámara de aforo se ubica fuera del borde modelado, por lo que su correspondencia con el acceso oriental es una aproximación pendiente de contraste por carril y sentido.

La Figura 3.3 ilustra la traza vial del dominio modelado: los puntos rojos indican la ubicación de los nodos semaforizados, las etiquetas con flechas azules señalan las avenidas de referencia y el óvalo resalta el acceso oriental correspondiente a la zona TAZ 28, vinculado al punto de aforo de la cámara 169. El mapa completo de las 39 zonas TAZ se incluye en la Figura A.5 del Apéndice A.

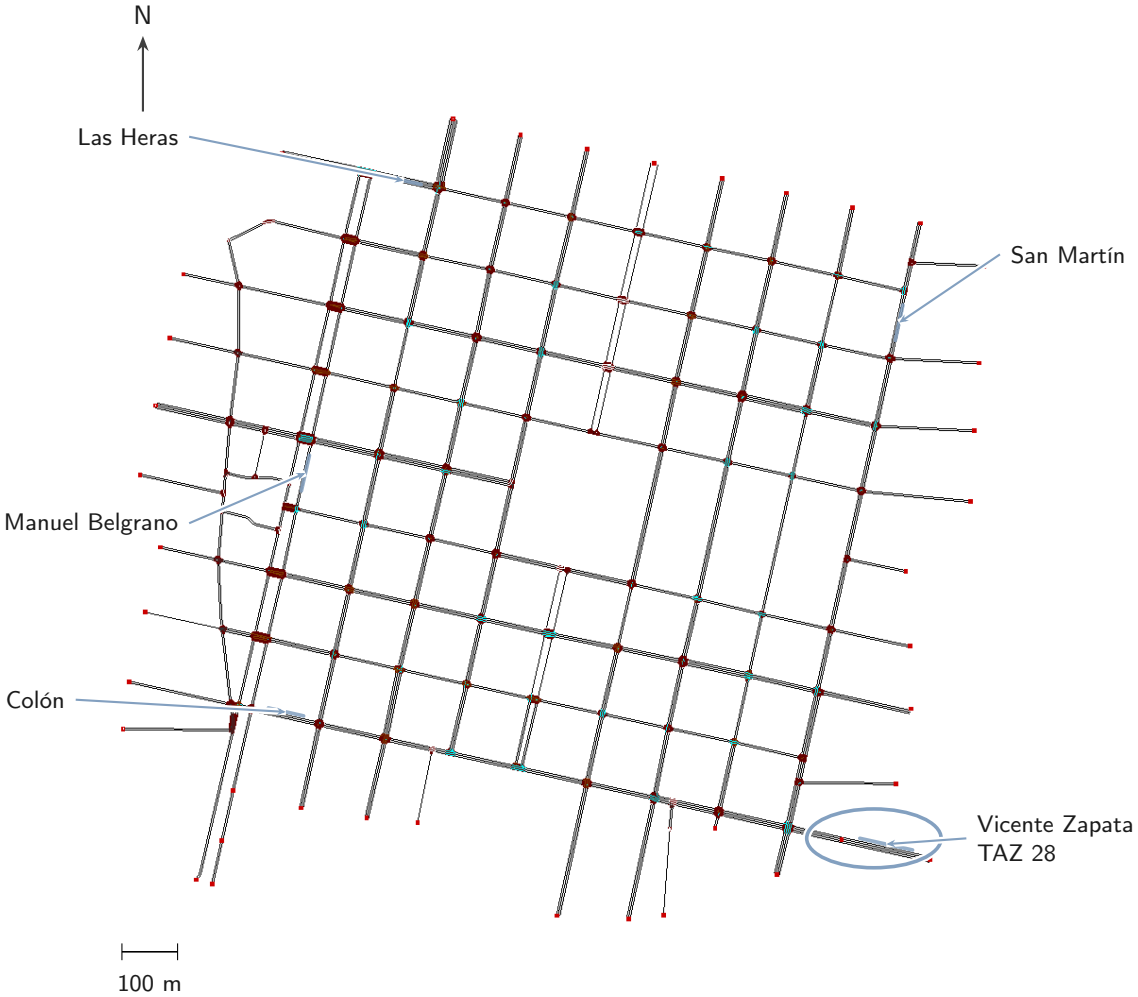


Figura 3.3: Dominio urbano de Mendoza, nodos semaforizados (puntos rojos) e ingreso oriental asociado a la zona TAZ 28.

Construcción del plan fijo

Dado que la red preliminar combinaba 17 controladores de tiempo fijo con 56 programas accionados heredados de la conversión inicial, se estandarizó la totalidad del sistema a planes de tiempo fijo con un ciclo común de 90 segundos, siguiendo la separación clásica entre ciclo, reparto de verdes y coordinación arterial [Fed08, cap. 6]. En los 56 controladores convertidos se preservaron las secuencias de estados y los intervalos de despeje, repartiendo el tiempo efectivo de ciclo en proporción a las duraciones medias observadas de cada fase accionada y garantizando umbrales mínimos de servicio. Posteriormente, se ajustaron los desfases de once intersecciones respecto de un reloj común para reducir llegadas en rojo sobre los corredores principales, obteniendo el plan definitivo (denominado P2), cuyas tablas completas se reportan en la Sección A.4.2 del Apéndice A.

Construcción de la demanda y selección del volumen nominal

A partir de la matriz base de flujos horarios se realizó una redistribución acotada de tasas entre pares OD —conservando invariantes los totales horarios y los marginales esperados de cada origen y destino— con el fin de reducir la exposición de la demanda al acceso recto de Tiburcio Benegas entre las 09:00 y las 10:00 (intervalo $[10.800, 14.400)$ s), movimiento al que el catálogo histórico de rutas asignaba 87 de los 209 itinerarios del par $10 \rightarrow 27$; la relación entre ese movimiento y los fallos observados orienta una hipótesis de diseño y no constituye una causa comprobada. Se adoptó un volumen nominal de 220.000 viajes diarios (configuración denominada O2) por ser el mayor nivel de carga que completó las seis réplicas de verificación en ambos horizontes, superando los 210.000 viajes del tratamiento O1 —admisible con menor volumen absoluto servido— y quedando por debajo de los 225.000 viajes de O2 reescalada, que no completó la tanda requerida en el horizonte de doce horas. La especificación algebraica de las tasas y el detalle de los pares ajustados se documentan en la Sección A.4.3 del Apéndice A.

La Figura 3.4 muestra el perfil escalonado de solicitudes esperadas por hora a lo largo de las doce horas modeladas (06:00 a 18:00 en la convención horaria del escenario, equivalentes a 152.168,75 solicitudes esperadas sobre el nominal diario de 24 horas). El sombreado claro abarca el horizonte de evaluación largo de doce horas ($[0, 43.200)$ s) y la franja más oscura destaca el horizonte corto de dos horas (06:30 a 08:30, $[1.800, 9.000)$ s), mientras que la redistribución puntual de tasas de la configuración O2 actúa en la cuarta hora de simulación (09:00 a 10:00). Debido a que la inserción vehicular sigue un proceso estocástico [Ecl26a], el número efectivo de vehículos cargados fluctúa alrededor de la esperanza teórica en cada semilla, y aquellos vehículos que no hallan brecha suficiente en el carril de entrada aguardan en una cola virtual de ingreso.

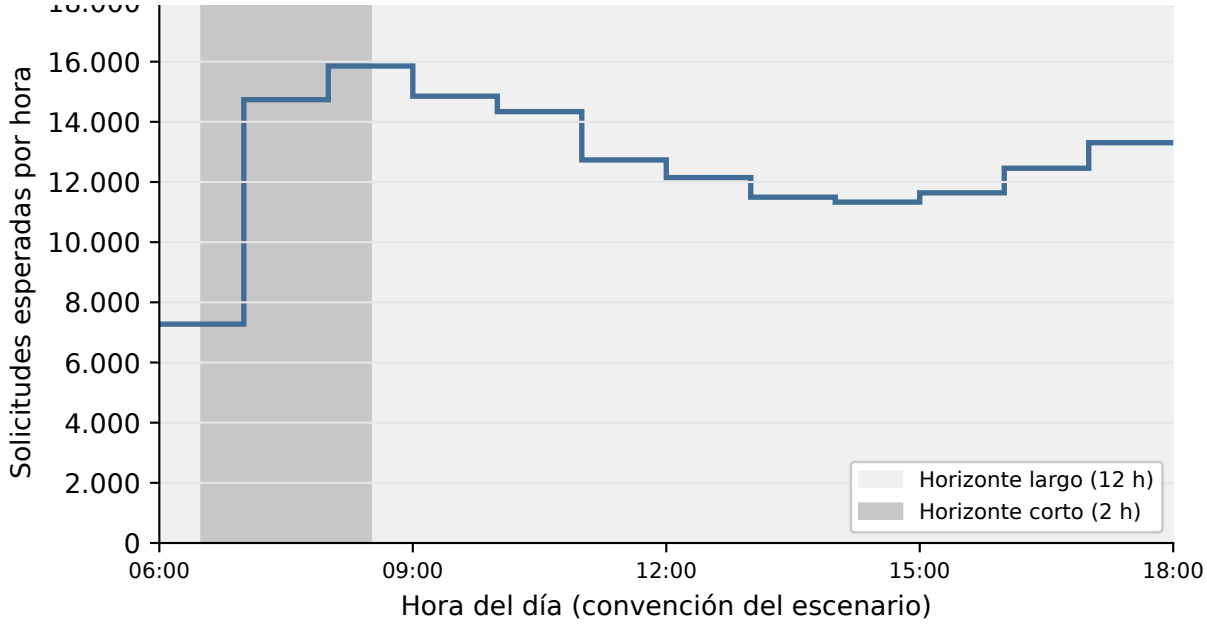


Figura 3.4: Solicitudes esperadas por hora y horizontes de evaluación en el escenario de Mendoza: corto (06:30–08:30, 2 h) y largo (06:00–18:00, 12 h).

Comprobación operacional, aceptación y alcance

La comprobación operacional evaluó el funcionamiento conjunto de la red de Mendoza con el plan fijo P2 y la demanda O2 sobre seis semillas independientes en cada horizonte (dos y doce horas), iniciando cada simulación con la red vacía. Las seis semillas por horizonte responden a un presupuesto del proyecto y no a un cálculo de precisión estadística; como son simulaciones de horizonte finito, la condición inicial debe considerarse al interpretar los resultados [Law15]. Se definió como criterio de aceptación que las seis réplicas de cada ventana completaran la totalidad del horizonte sin teletransportes, colisiones detectadas ni descartes de carga, alcanzando una proporción de viajes finalizados sobre vehículos cargados de al menos el 75 %. Los parámetros de ejecución y el registro por semilla se detallan en la Sección A.4.4 del Apéndice A, y la síntesis cuantitativa se reporta en la Tabla 4.1 del Capítulo 4.

Siguiendo la distinción metodológica clásica entre verificación operativa de un modelo de simulación y validación empírica frente al sistema físico real [Sar13, WVW19], esta comprobación acredita la consistencia interna y la estabilidad de ejecución del escenario urbano bajo control de tiempo fijo, sin reclamar una calibración de campo completa —la cual requeriría aforos independientes por movimiento y mediciones de tiempos de viaje en los corredores interiores—. Asimismo, dado que el horizonte corto de dos horas (06:30–08:30) concluye antes de la franja de redistribución (09:00–10:00), las corridas de dos horas verifican el régimen matutino de carga creciente, en tanto que el desempeño de doce horas captura el ciclo diurno completo.

El carácter adaptativo de esta construcción, las condiciones de interpretación de sus

resultados y el alcance de la inferencia sobre el escenario urbano se delimitan en la Sección 3.7.2.

3.3. Sistema de control multiagente

Con las redes y sus demandas definidas, el sistema implementa el ciclo de interacción agente–entorno presentado en el Capítulo 2: transforma las mediciones microscópicas del tránsito en observaciones locales para cada agente y traduce sus decisiones en estados semafóricos dentro del simulador. Los cuatro algoritmos evaluados comparten esta interfaz común.

Cada agente i gobierna las señales de una intersección (o de un conjunto de nodos asociados a un mismo controlador semafórico), caracterizada por m_i carriles entrantes controlados y K_i fases verdes seleccionables. Los intervalos de cambio amarillo y el estado inicial de despeje en todo rojo son gestionados automáticamente por el entorno durante las transiciones y la inicialización, por lo que no forman parte del espacio de decisiones discretas del agente.

El paquete desarrollado, `marlTLC`, acopla el simulador SUMO con el marco distribuido Ray RLlib [LLN+18a] a través del entorno `SumoRllibEnv`, utilizando `libsumo` como enlace en memoria y PyTorch [PGM+19] para el cómputo y la optimización de las redes neuronales. La Figura 3.5 esquematiza el flujo de datos entre los subsistemas: el componente de muestreo de RLlib consulta las políticas activas para seleccionar acciones y recolecta del entorno las observaciones, recompensas e indicadores de cierre; con ellos ensambla lotes de transiciones que transfiere al módulo de aprendizaje (*Learner*), encargado de actualizar los pesos neuronales y sincronizarlos nuevamente con los trabajadores de muestreo. De manera desacoplada respecto de este lazo de optimización, el entorno gestiona la carga de estados de SUMO y el volcado de registros de viaje (`TripInfo`) para la evaluación. La arquitectura de clases de `marlTLC` se detalla en la Sección A.1 del Apéndice A.

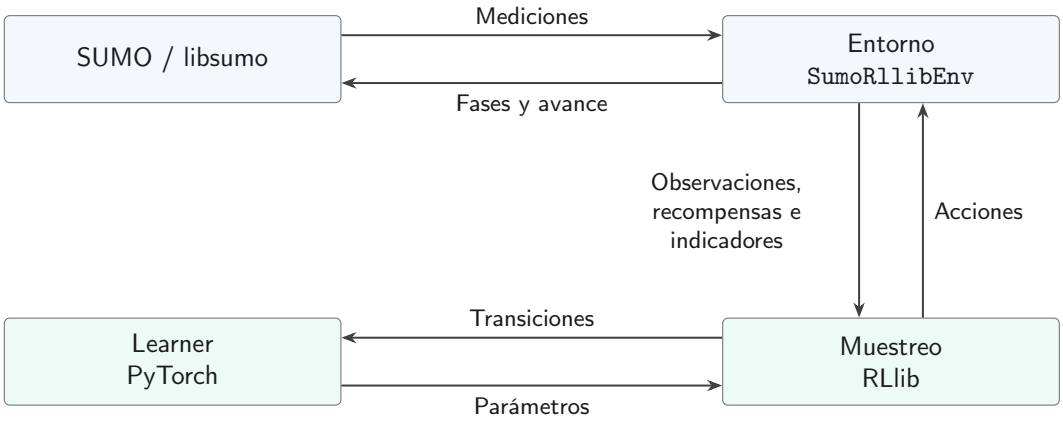


Figura 3.5: Intercambios de información entre simulación, entorno, muestreo y aprendizaje en `marlTLC`.

3.3.1. Entorno de simulación

Al inicio de cada experimento, el entorno avanza la simulación durante el período de calentamiento configurado y almacena en memoria una instantánea del estado de SUMO correspondiente al comienzo de la ventana de control. En cada invocación de `reset`, el entorno restaura dicha instantánea, reinicia los contadores internos y la contabilidad de espera por carril, y sitúa los semáforos en el estado inicial de seguridad (todo rojo). Posteriormente, en cada llamada a `step`, aplica las acciones seleccionadas por los agentes, ejecuta las transiciones amarillas cuando corresponde, avanza la simulación hasta el siguiente instante de decisión y retorna las nuevas observaciones, recompensas e indicadores de episodio.

El episodio concluye al agotarse el horizonte temporal estipulado o, cuando se activa la opción `terminate_on_empty`, al vaciarse por completo la red y la cola de inserción (verificado mediante `getMinExpectedNumber`). La interfaz sigue el estándar de Gymnasium/RLlib diferenciando las banderas de terminación (`terminated`) y truncamiento por límite de tiempo (`truncated`). En las campañas sintéticas se utilizó la configuración `time_limit_as_truncation=False`, bajo la cual el cierre por horizonte activa ambos indicadores e inhibe el *bootstrapping* del valor del estado final en la estimación de ventaja generalizada (véase la Ecuación (2.18) del Capítulo 2).

Finalmente, la topología de acoplamiento entre intersecciones se construye de forma automática a partir del grafo vial: el entorno define como vecinos directos a dos agentes siempre que al menos un conector saliente de un carril controlado por el primero desemboque en un carril entrante controlado por el segundo, registrando una relación de adyacencia simétrica para ambos.

3.3.2. Observación, acciones y recompensas

La formulación del problema representa el estado operativo de cada intersección a partir de la fase activa y de la espera vehicular por carril, restringe las acciones mediante una máscara dependiente del tiempo transcurrido y define la recompensa a partir de la variación de la espera entre pasos de decisión consecutivos.

Observaciones y codificación de la espera

En cada paso de decisión t, el agente i recibe una observación local $o_i^{(t)}$ conformada por el índice de su fase activa $p_i^{(t)}$ (que incluye un código específico para el estado inicial de todo rojo) y las esperas codificadas de sus m_i carriles entrantes controlados:

$$o_i^{(t)} = \big(p_i^{(t)}, e(w_{i,1}^{(t)}), \ldots, e(w_{i,m_i}^{(t)})\big), \tag{3.1}$$

donde $w_{i,l}^{(t)}$ denota la espera acumulada atribuida al carril l del agente i (en veh·s) y $e(\cdot)$ es una función de discretización en I niveles enteros dentro del conjunto $\{0, 1, \ldots, I-1\}$.

Las políticas operan de manera reactiva sobre $o_i^{(t)}$, sin almacenar un historial recurrente de observaciones pasadas.

Para calcular $w_{i,l}^{(t)}$, el entorno consulta la espera acumulada de cada vehículo presente en el carril mediante el método `getAccumulatedWaitingTime` de SUMO, que integra el tiempo detenido dentro de una ventana móvil de memoria [Ecl26d] (especificada en la Sección A.4 del Apéndice A). A fin de evitar el doble cómputo cuando un vehículo avanza de una cuadra a otra dentro de esa ventana, el entorno descuenta de la lectura de SUMO las esperas previamente registradas por ese mismo vehículo en otros carriles, truncando el resultado en cero. La espera total en la intersección del agente i se obtiene sumando sus carriles controlados: $W_i^{(t)} = \sum_{l=1}^{m_i} w_{i,l}^{(t)}$. Cabe notar que esta magnitud instantánea por carril difiere del atributo `TripInfo.waitingTime` empleado en la evaluación final, el cual acumula sin ventana de olvido la espera total de cada viaje de extremo a extremo [Ecl26e].

La función codificadora $e(x)$ proyecta el valor continuo $x \in [0, M_{\text{lane}}]$ sobre I niveles combinando una transformación logarítmica —que otorga mayor poder discriminativo a las esperas bajas y moderadas— con una componente lineal, seguida de un redondeo hacia arriba (ceil); su formulación algebraica exacta se desarrolla en la Sección A.6.1 del Apéndice A. El número de niveles I forma parte de los hiperparámetros explorados en las búsquedas, mientras que la cota de escala M_{lane} (`max_lane_value`) se fijó por escenario en el percentil 99 del máximo por paso de $w_{i,l}^{(t)}$ tras 300 s de calentamiento, resultando en 1.184, 1.136 y 712 veh·s para las redes 3×1, 3×3 y $4 \times 4_H$, respectivamente. En la variante vecinal C, el vector de observación local (3.1) se amplía añadiendo un término codificado $e\big(W_j^{(t)}/m_j\big)$ por cada vecino directo $j \in \mathcal{N}(i)$, escalado según el umbral vecinal M_{vec} documentado en la Sección A.6.4 del Apéndice A.

La Figura 3.6 ilustra la curva escalonada producida por $e(x)$ para $I = 22$ niveles y $M_{\text{lane}} = 1.136$ veh·s (red 3×3). Se aprecia cómo los intervalos de cuantización se estrechan cerca del origen —por ejemplo, $e(0) = 0$, $e(120) = 10$ y $e(600) = 18$— y se ensanchan progresivamente hacia el extremo superior de la escala.

Acciones permitidas y cambios de fase

A partir de la observación recibida, el agente i selecciona una fase verde dentro del conjunto discreto $\{0, 1, \ldots, K_i - 1\}$. En cada instante de decisión, una máscara de acciones restringe la elección a las fases compatibles con las restricciones de tiempo mínimo de verde y de transición amarilla detalladas en la Sección 3.4.1. En los algoritmos basados en gradiente de políticas (IPPO, MAPPO y HAPPO), las acciones inválidas se anulan desplazando sus *logits* antes de la normalización *softmax*, mientras que en IDQN se excluyen del cálculo del máximo valor de acción, según se formaliza en la Sección A.6.2 del Apéndice A.

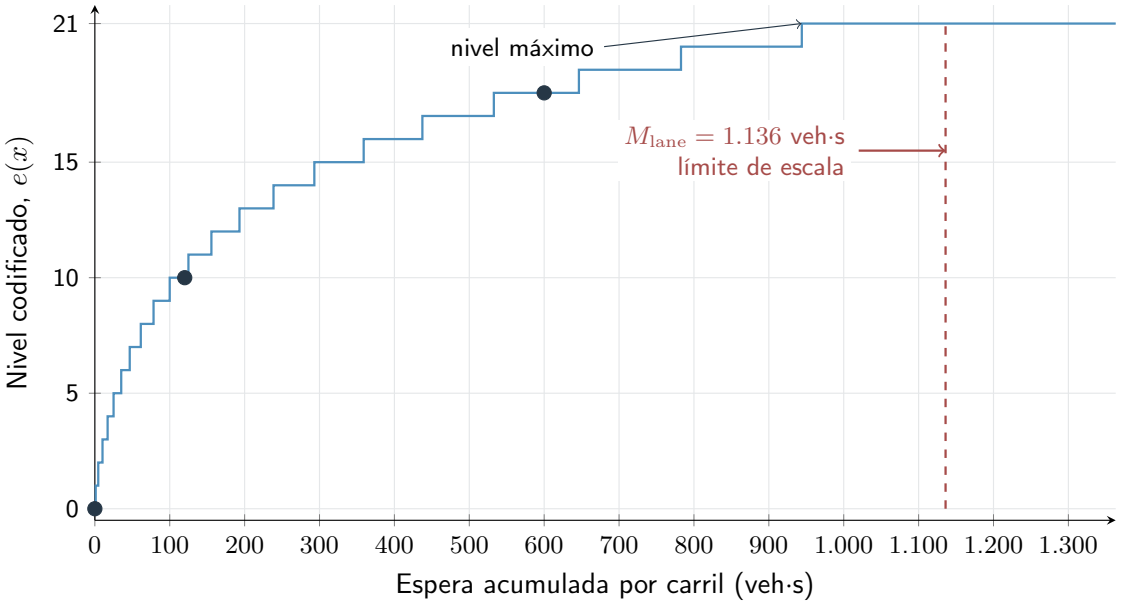


Figura 3.6: Curva de cuantización del codificador local de espera para $I = 22$ niveles y $M_{\text{lane}} = 1.136$ veh·s.

Recompensas locales y vecinales

La señal de recompensa local $L_i^{(t)}$ mide la reducción de la espera acumulada en la intersección i entre dos pasos de decisión consecutivos (en veh·s), de modo que toma valores positivos cuando la espera disminuye:

$$L_i^{(t)} = W_i^{(t-1)} - W_i^{(t)}. \tag{3.2}$$

En las formulaciones que incorporan acoplamiento espacial, cada agente i complementa su señal local con el promedio de las variaciones de espera de sus vecinos directos $j \in \mathcal{N}(i)$, normalizadas por el número de carriles entrantes m_j de cada vecino para compensar diferencias de capacidad en redes heterogéneas como $4\times4_H$. Dado que toda intersección controlada posee al menos un vecino en los escenarios estudiados ($|\mathcal{N}(i)| \geq 1$), el término vecinal por carril $N_{i,\text{carril}}^{(t)}$ y la recompensa compuesta $R_i^{(t)}$ ponderada por el coeficiente de cooperación $\rho_{\text{coop}} \in [0, 1)$ quedan definidos por:

$$N_{i,\text{carril}}^{(t)} = \frac{1}{|\mathcal{N}(i)|} \sum_{j \in \mathcal{N}(i)} \frac{L_j^{(t)}}{m_j}, \tag{3.3}$$

$$R_i^{(t)} = (1 - \rho_{\text{coop}}) L_i^{(t)} + \rho_{\text{coop}} N_{i,\text{carril}}^{(t)}, \tag{3.4}$$

mientras que para un nodo aislado hipotético ($|\mathcal{N}(i)| = 0$) el código anula $N_{i,\text{carril}}^{(t)}$ y conserva $R_i^{(t)} = L_i^{(t)}$. En todos los casos, cada agente recibe su propia recompensa escalar $R_i^{(t)}$ durante el entrenamiento; cuando $\rho_{\text{coop}} = 0$, la Ecuación (3.4) se reduce exactamente a la recompensa local $L_i^{(t)}$. El coeficiente ρ_{coop} actúa como peso de combinación entre señales

de escalas distintas y no representa un porcentaje fijo de la recompensa resultante.

La Tabla 3.4 sintetiza la configuración de observaciones y recompensas empleada en cada etapa experimental sobre las redes sintéticas. En la fase 1 de búsqueda se exploró $\rho_{\text{coop}} \in [0;\ 0{,}95)$ manteniendo observaciones locales y promediando las señales vecinales sin división por m_j; a partir de las pruebas de variantes y en la fase 2 refinada se adoptó la formulación puramente local ($\rho_{\text{coop}} = 0$), en tanto que la comparación pareada A/C contrasta la variante local (A, $\rho_{\text{coop}} = 0$) frente a la variante vecinal normalizada por carril (C, $\rho_{\text{coop}} = 0{,}1$).

Tabla 3.4: Observaciones y recompensas de las búsquedas sintéticas y de la comparación A/C.

Procedimiento	**Observación**	**Recompensa**	**Peso vecinal**
Fase 1 sintética	Local	Local y vecinal, sin normalización por carriles	[0; 0,95)
Búsqueda refinada sintética	Local	Local	0
Comparación: local (A)	Local	Local	0
Comparación: vecinal (C)	Vecinal	Local y promedio vecinal normalizado por carril	0,1

3.3.3. Agentes y algoritmos de aprendizaje

Sobre la formulación general de los algoritmos de aprendizaje multiagente desarrollada en la Sección 2.3 y en las Subsecciones 2.3.5 y 2.3.6 del Capítulo 2, el paquete `marlTLC` introduce decisiones específicas de arquitectura e implementación para adecuar los cuatro algoritmos a una interfaz común en Ray RLlib:

- **Parámetros independientes por intersección:** cada agente cuenta con su propia red neuronal sin compartición de pesos (*parameter sharing*), lo que permite acomodar de manera directa las dimensiones heterogéneas de entrada (m_i) y de salida (K_i) presentes en $4 \times 4_H$. En IDQN, cada agente entrena su red de valores de acción empleando un búfer de repetición de experiencias priorizado (`Prioritized EpisodeReplayBuffer`).
- **Crítico centralizado con salidas por agente en MAPPO y HAPPO:** siguiendo el esquema CTDE (Figura 2.19), el crítico centralizado concatena las observaciones de todos los agentes en un único vector de entrada y produce N salidas escalares de valor —una por agente— asociadas a sus respectivas recompensas individuales,

estandarizando las ventajas resultantes por agente antes de actualizar las políticas. Para acotar el impacto de valores atípicos, la función de pérdida del crítico satura en un umbral c_v (`vf_clip_param`) el error cuadrático medio entre agentes de cada transición, según se formaliza en la Ecuación (A.6) del Apéndice A.

- **Ordenamiento híbrido y alcance teórico en HAPPO:** al inicio de cada época de optimización, la adaptación implementada actualiza primero el crítico y luego recorre secuencialmente los actores utilizando el lote completo de transiciones para cada agente (Sección 2.3.6). El orden de actualización $i_1, \ldots, i_N$ se determina en cada época mediante el hiperparámetro `greedy_selection_prob`: en cada posición se elige con esa probabilidad al agente pendiente que registra la mayor media de ventaja absoluta estandarizada $|\hat{A}_i|$ y, con la probabilidad complementaria, se selecciona un agente al azar entre los restantes. Cabe recordar que, dado que el entorno asigna recompensas individuales por intersección en lugar de una recompensa cooperativa única, esta adaptación aprovecha el mecanismo algorítmico de actualización secuencial con factor acumulado $M_{i_{1:m}}$, pero opera fuera del supuesto de recompensa compartida requerido por la garantía teórica de mejora monótona en HAML (Sección 2.3.6).

La Figura 3.7 contrasta el flujo de actualización de actores entre ambos métodos centralizados para un caso ilustrativo de tres agentes (i_1, i_2, i_3). En el panel (a), correspondiente a MAPPO, los actores se actualizan en paralelo empleando únicamente sus ventajas locales $\hat{A}_i$. En el panel (b), correspondiente a HAPPO, los actores se actualizan de manera secuencial y cada agente transfiere al siguiente el factor acumulado de razones de verosimilitud posoptimización (representado mediante las flechas horizontales discontinuas). Durante la fase de ejecución descentralizada, ambos métodos prescinden del crítico y seleccionan las fases semafóricas consultando únicamente el actor local de cada intersección.

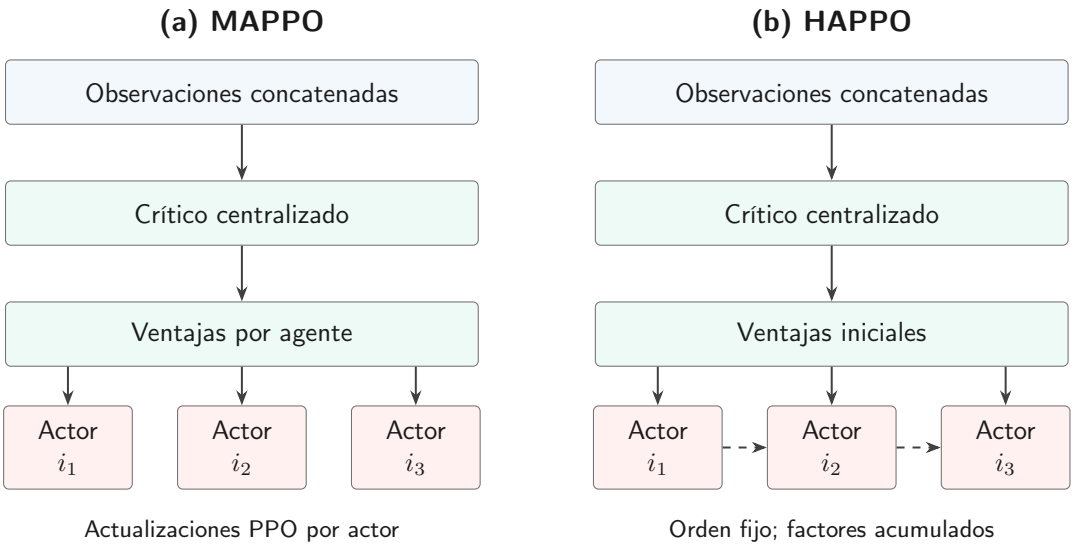


Figura 3.7: Esquema de actualización de actores con tres agentes ilustrativos: (a) actualización paralela en MAPPO; (b) actualización secuencial con propagación del factor posoptimización en HAPPO.

3.4. Entrenamiento y búsqueda de hiperparámetros

Con el entorno y las arquitecturas de agentes definidos, el ajuste mediante optimización de hiperparámetros (*Hyperparameter Optimization*, HPO) explora sistemáticamente el espacio de configuraciones de aprendizaje para ofrecer una comparación equitativa entre los cuatro algoritmos. Cada ensayo entrena una configuración candidata durante un presupuesto acotado de iteraciones de entrenamiento, donde cada iteración agrupa un lote fijo de episodios recolectados en paralelo.

Para administrar eficientemente el presupuesto computacional se combinó muestreo adaptativo con detención temprana. La generación de configuraciones candidatas se realizó mediante Optuna utilizando el estimador estimador de Parzen estructurado en árbol (*Tree-structured Parzen Estimator*, TPE) [BBBK11, ASY+19], mientras que la orquestación paralela de ensayos se delegó a Ray Tune [LLN+18b] acoplado al planificador asíncrono algoritmo asíncrono de reducción sucesiva a la mitad (*Asynchronous Successive Halving Algorithm*, ASHA) [LJR+20]. Tras un período de gracia inicial en el que ningún ensayo es interrumpido, ASHA compara periódicamente el desempeño intermedio de las corridas activas y retiene únicamente la fracción superior definida por su factor de reducción.

3.4.1. Condiciones de simulación y entrenamiento

La Tabla 3.5 resume los períodos de calentamiento y la duración efectiva de la ventana de control para cada procedimiento experimental.

Tabla 3.5: Calentamiento y duración del control por procedimiento.

Campaña	**Calentamiento**	**Duración del control**
HPO sintética, fase 1	600 s	3.600 s
HPO sintética, fase 2	600 s	3.600 s
Pruebas preliminares de MAPPO	300 s	1.800 s
Comparación local y vecinal de MAPPO	300 s	3.600 s
Entrenamiento y evaluación de modelos finales sintéticos	600 s	3.600 s
Mendoza, aceptación con programas fijos (12 h)	No se añade	43.200 s
Mendoza, aceptación con programas fijos (2 h)	No se añade	7.200 s
Mendoza, ajuste de hiperparámetros (horizonte corto)	Por definir	7.200 s
Mendoza, evaluación de políticas (horizonte corto)	Por definir	7.200 s
Mendoza, evaluación de políticas (horizonte largo)	Por definir	43.200 s

En todas las campañas de aprendizaje, los agentes toman decisiones con un paso de control de $\Delta t = 5\,\mathrm{s}$. Si un agente decide mantener la fase activa, el intervalo de cinco segundos transcurre íntegramente en verde. Por el contrario, cuando solicita un cambio de fase habilitado por la máscara, el entorno ejecuta una transición amarilla de 4 s seguida de 1 s de verde de la nueva fase dentro de ese mismo paso de decisión, e impone una permanencia mínima de 5 s en verde antes de admitir un nuevo cambio (la transición inicial desde el estado de todo rojo al comenzar el episodio se realiza directamente sin intervalo amarillo).

La Figura 3.8 ilustra el efecto combinado del paso de decisión ($\Delta t = 5\,\mathrm{s}$), el intervalo amarillo (4 s) y el tiempo mínimo de verde (5 s). Tomando como origen ($t = 0\,\mathrm{s}$) el instante en que se acepta una solicitud de cambio de fase, los primeros 4 s corresponden al intervalo amarillo y la nueva fase verde se inicia en $t = 4\,\mathrm{s}$. En la decisión siguiente ($t = 5\,\mathrm{s}$), la nueva fase acumula apenas 1 s de verde, por lo que la máscara bloquea cualquier cambio y obliga a mantenerla. Aunque los 5 s reglamentarios de verde mínimo se cumplen en $t = 9\,\mathrm{s}$, la discreción del reloj de control hace que la habilitación efectiva de un nuevo cambio ocurra recién en la decisión de $t = 10\,\mathrm{s}$, tras haberse completado 6 s efectivos de verde.

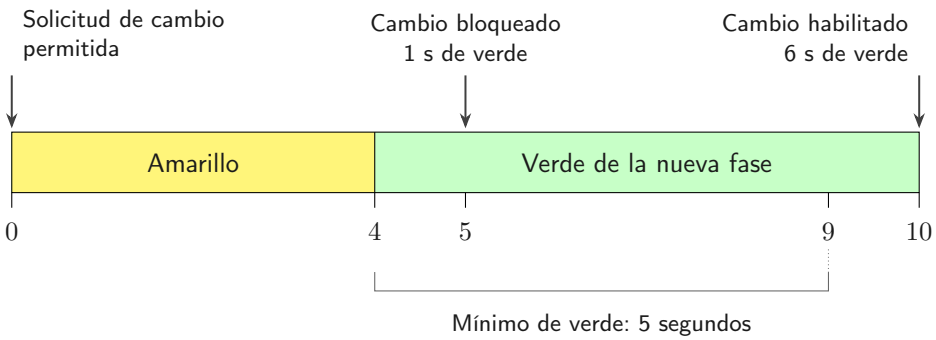


Figura 3.8: Línea temporal de transición de fase y bloqueo por máscara de acciones con decisiones cada $\Delta t = 5\,\mathrm{s}$.

Para garantizar la trazabilidad estocástica en las campañas sintéticas se controlaron por separado cuatro fuentes de números pseudoaleatorios: la semilla de entrenamiento de las redes neuronales, la semilla microscópica de SUMO, la semilla generadora de rutas y la semilla del muestreador TPE. Los valores específicos empleados en la fase 1, en la fase 2 y en la comparación A/C se reportan en las Tablas A.18 y A.19 y en la Sección A.6.4 del Apéndice A. Asimismo, durante el entrenamiento se intercalan periódicamente episodios deterministas de seguimiento diagnóstico —en los que los métodos basados en PPO eligen la acción permitida de máxima probabilidad e IDQN la de mayor valor estimado—, cuyos registros se almacenan por separado sin intervenir en el cálculo del objetivo de optimización.

3.4.2. Función objetivo de la búsqueda

Con el propósito de favorecer configuraciones que alcancen simultáneamente una baja demora vehicular y una evolución estable al final del entrenamiento, la búsqueda de hiperparámetros minimiza un objetivo compuesto J_r (en segundos por vehículo). En cada iteración de entrenamiento r que produce un registro válido, se calcula primero la espera media por vehículo insertado w_r, obtenida dividiendo la suma de los tiempos de espera (`TripInfo.waitingTime`) de todos los episodios de aprendizaje completados en esa iteración por el total de vehículos insertados en ellos.

A continuación, se descartan las primeras cinco mediciones válidas del ensayo y se retiene una ventana deslizante con hasta las diez mediciones válidas más recientes. Para cada par de mediciones consecutivas dentro de esa ventana ($s = 1, \ldots, n$, con $n \leq 9$ diferencias disponibles), se registra el incremento no negativo de espera $\Delta w_s = \max(0, w_s - w_{s-1})$. Ordenando dichos incrementos de mayor a menor como $\Delta w_{(1)} \geq \Delta w_{(2)} \geq \cdots \geq \Delta w_{(n)}$, se define una penalización por inestabilidad reciente T_r igual al promedio del 20 % de los incrementos más severos ($k = \lceil 0{,}2n \rceil$), inspirada en el concepto de valor en riesgo condicional (*Conditional Value at Risk*, CVaR) sobre una muestra finita. De este modo,

el objetivo J_r evaluado en la iteración r queda expresado como:

$$J_r = w_r + T_r, \qquad \text{con} \quad T_r = \begin{cases} \dfrac{1}{k}\displaystyle\sum_{j=1}^{k} \Delta w_{(j)}, & \text{si } n > 0 \;\; \big(k = \lceil 0{,}2n \rceil\big), \\ 0, & \text{si } n = 0. \end{cases} \tag{3.5}$$

3.4.3. Fase 1: búsqueda amplia

En las tres redes sintéticas, la fase 1 cruza los cuatro algoritmos con dos arquitecturas de perceptrón multicapa: `h128` y `h256` (dos capas ocultas de 128 y 256 unidades, respectivamente), conformando $4 \times 3 \times 2 = 24$ estudios independientes. Cada estudio dispone de un presupuesto de 100 ensayos iniciales (ampliables a 150) con un tope de 40 iteraciones de entrenamiento por ensayo, regulado por ASHA con un período de gracia de 10 iteraciones y un factor de reducción igual a 4, e intercalando un episodio de seguimiento diagnóstico cada 10 iteraciones.

Dentro de este espacio inicial, los algoritmos de la familia PPO exploran la tasa de aprendizaje, el umbral de recorte del actor, los coeficientes de entropía y de divergencia KL, el parámetro λ de GAE, la cota de pérdida del crítico y el tamaño de lote, mientras que IDQN explora el horizonte de retorno de múltiples pasos, la frecuencia de actualización de la red objetivo y la agenda de exploración ε-voraz. Todos los estudios fijan el factor de descuento en $\gamma = 0{,}99$ y exploran conjuntamente la cantidad de niveles I del codificador y el peso vecinal ρ_{coop} mediante una distribución uniforme continua, una distribución loguniforme o muestreos discretos, cuyos dominios completos se detallan en las Tablas A.14, A.15 y A.16 del Apéndice A [SWD+17, YVV+22, ELR23].

3.4.4. Fase 2: búsqueda con dominios reducidos

El protocolo refinado de fase 2 utiliza los resultados de la fase 1 de cada red sintética para acotar los rangos de búsqueda en torno a las regiones de mejor desempeño observadas. Para cada combinación de algoritmo, escenario y arquitectura se ordenan según el objetivo J_r los ensayos de fase 1 que completaron las 40 iteraciones sin errores; en las variables numéricas se parte del intervalo abarcado por los diez mejores ensayos y se amplía cada extremo en un 10 % del ancho del dominio original, limitando el resultado a sus fronteras (y conservando el dominio original si el intervalo de los veinte mejores ensayos disponibles cubre al menos el 70 % de su ancho, o si alguno de los diez mejores queda a una distancia de un extremo no mayor que el 5 % de ese ancho), mientras que en las variables categóricas se retienen las alternativas presentes entre los veinte mejores ensayos disponibles. Las reglas formales de reducción y los doce dominios resultantes se documentan en la Sección A.5.7 del Apéndice A.

Sobre estos dominios reducidos se ejecuta una nueva exploración desde cero fijando la formulación puramente local ($\rho_{\text{coop}} = 0$), motivada tanto por la degradación observada en

la fase 1 para valores elevados de ρ_{coop} como por el contraste pareado A/C desarrollado en la Sección 3.5. Cada estudio de fase 2 evalúa hasta 100 ensayos con un presupuesto ampliado a 60 iteraciones por ensayo, empleando ASHA con un período de gracia de 30 iteraciones, factor de reducción igual a 2 y episodios de seguimiento diagnóstico cada 15 iteraciones (véase la Sección A.5.6 del Apéndice A).

Como punto de partida determinista del muestreador TPE en la fase 2, se construye para cada estudio una configuración inicial combinando la mediana de los hiperparámetros numéricos y la moda de los categóricos entre los cinco mejores ensayos de la fase 1. La Figura 3.9 esquematiza esta transición entre ambas etapas de búsqueda, distinguiendo el subconjunto de diez ensayos que delimita el nuevo dominio y el subconjunto de cinco ensayos que define la configuración inicial.

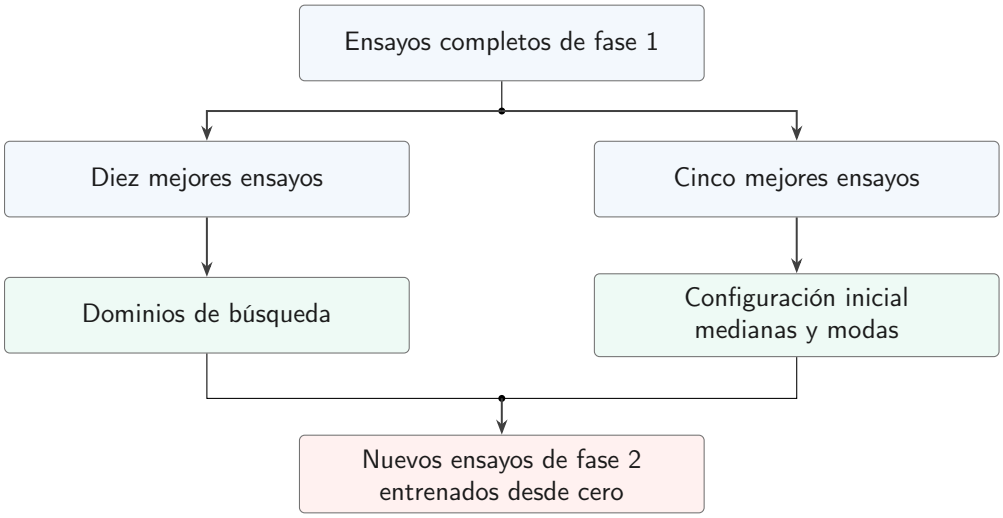


Figura 3.9: Derivación de dominios reducidos y configuración inicial en la transición de la fase 1 a la fase 2 de búsqueda.

3.4.5. Criterios de selección de configuraciones

Concluida la fase 2 en cada combinación sintética de algoritmo y escenario, un procedimiento de selección filtra los candidatos antes del entrenamiento final desde cero, asegurando que una baja espera media no sea producto de retener vehículos fuera de la red:

1. **Admisibilidad técnica y preselección:** se verifican la completitud de las 60 iteraciones de entrenamiento, la validez de los registros de métricas y la integridad de los archivos de estado (*checkpoint*), formando una lista corta de hasta tres configuraciones candidatas por algoritmo y escenario ordenadas por su espera diagnóstica (véase la Sección A.5.6 del Apéndice A).

2. **Evaluación en panel común y filtros operacionales:** los candidatos preseleccionados para los cuatro algoritmos de una misma red se evalúan sobre un panel

común de tres semillas de SUMO. Tomando como referencia empírica de flujo el máximo de vehículos insertados en ese panel para cada red y semilla, se exige que cada episodio registre: (i) cero inicios de teletransporte, (ii) una tasa de completitud de vehículos insertados (*Completion of Inserted Vehicles*, CI) de al menos 0,95 (Ecuación (3.7)), y (iii) un volumen de inserciones no inferior al 95 % de la referencia empírica del panel.

3. **Elección final:** entre los candidatos que superan los tres filtros operacionales se selecciona aquel de menor espera media por vehículo insertado. Las semillas de este panel de selección (950001, 950003 y 950007) quedan reservadas exclusivamente para esta etapa y no intervienen en la evaluación final, que emplea un conjunto propio de semillas (Sección 3.6.4).

Para el escenario de Mendoza se requiere un protocolo de selección propio, aún no ejecutado: los criterios del diseño anterior se conservan como antecedente en la Sección A.5.8 del Apéndice A y no constituyen una selección válida para la demanda O2 y el plan P2.

3.5. Comparación local y vecinal de MAPPO

La segunda línea experimental examina en MAPPO si ampliar la observación local con el estado de las intersecciones adyacentes e incorporar un término de cooperación vecinal en la recompensa mejora el desempeño respecto de una formulación estrictamente local. Manteniendo fija una arquitectura e hiperparámetros comunes de entrenamiento, el estudio comprende dos etapas: primero, una prueba exploratoria de cinco variantes (A–E) en la red heterogénea $4\times4_H$; segundo, una comparación pareada en las tres redes sintéticas entre la formulación local (variante A, $\rho_{\text{coop}} = 0$) y la formulación vecinal normalizada por carril (variante C, $\rho_{\text{coop}} = 0{,}1$). Dado que la variante C modifica conjuntamente el vector de observación y la función de recompensa, el contraste evalúa el efecto combinado de ambos elementos de acoplamiento.

3.5.1. Prueba preliminar de formulaciones

En la etapa exploratoria se entrenaron cinco variantes (A–E) sobre la red $4 \times 4_H$ empleando MAPPO con arquitectura `h128` y tres semillas de entrenamiento durante 20 iteraciones por política, en episodios de 1.800 s precedidos por 300 s de calentamiento. La Tabla 3.6 resume la definición de cada variante: A corresponde a la línea base local; B, C y D incorporan acoplamiento vecinal con $\rho_{\text{coop}} = 0{,}1$ bajo distintas normalizaciones por número de carriles; y E evalúa una recompensa local inspirada en el concepto de presión de PressLight [WCZ+19] utilizando conteos de vehículos detenidos entre carriles de entrada y salida. Los criterios de verificación técnica de estas corridas se detallan en la Sección A.5.2 del Apéndice A.

Tabla 3.6: Variantes examinadas en la prueba preliminar sobre la red $4 \times 4_H$.

Variante	**Observación**	**Recompensa**
A	Local: fase activa y señales de carriles propios.	Diferencia local de espera acumulada ($\rho_{\text{coop}} = 0$).
B	Local y suma de espera acumulada de cada vecino.	Diferencia local y vecinal sin normalización por carriles ($\rho_{\text{coop}} = 0{,}1$).
C	Local y vecinal: espera media por carril de cada vecino.	Diferencia local y vecinal normalizada por carriles ($\rho_{\text{coop}} = 0{,}1$).
D	Local, suma de espera vecinal y $\ln(1 + m_j)$ de cada vecino.	Igual que C ($\rho_{\text{coop}} = 0{,}1$).
E	Local: fase activa y señales de carriles propios.	Negativo de la diferencia absoluta de vehículos detenidos entre carriles de entrada y salida.

3.5.2. Entrenamiento de las variantes en las tres redes sintéticas

Para la comparación multiescenario entre las variantes A y C se adoptó en las tres redes sintéticas (3×1, 3×3 y $4 \times 4_H$) una misma receta de hiperparámetros de MAPPO con arquitectura `h128`, especificada en la Sección A.6.3 del Apéndice A. La Figura 3.10 contrapone la estructura de ambas formulaciones para un agente i con dos vecinos ilustrativos (j y k): en el panel (a), correspondiente a la variante local A, la observación del actor contiene únicamente la fase propia p_i y los m_i carriles locales codificados, y la recompensa coincide con L_i; en el panel (b), correspondiente a la variante vecinal C, el vector de observación incorpora además los niveles codificados de espera media por carril de cada vecino directo —lo que incrementa proporcionalmente la dimensión de entrada de la primera capa del actor y del crítico centralizado— y la recompensa combina linealmente L_i y $N_{i,\text{carril}}$ con peso $\rho_{\text{coop}} = 0{,}1$. El alcance inferencial de este contraste, que evalúa el efecto combinado de la observación y la recompensa vecinales, se delimita en la Sección 3.7.2.

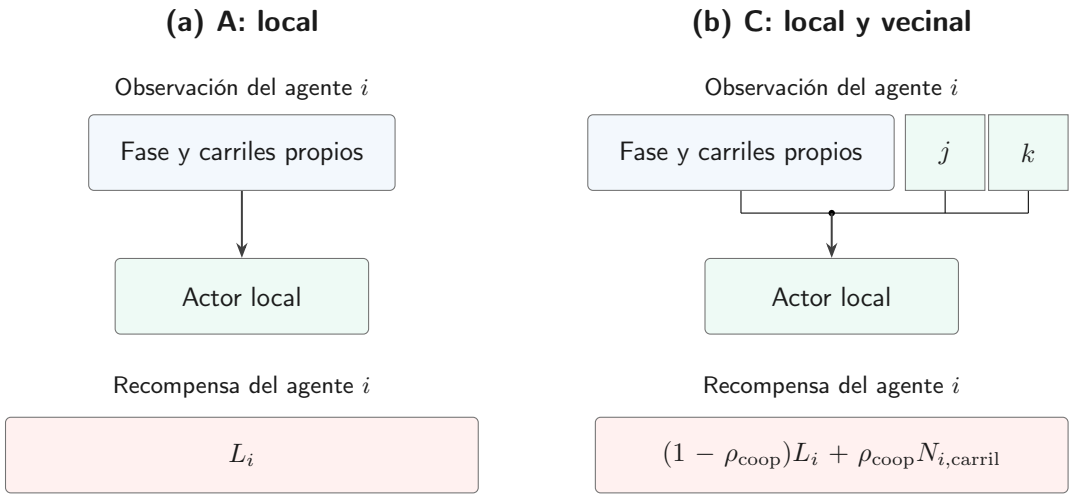


Figura 3.10: Estructura de la observación del actor y de la señal de recompensa en MAPPO: (a) formulación local A; (b) formulación vecinal C.

Con el fin de controlar la variabilidad inherente a la inicialización neuronal y al muestreo estocástico del entrenamiento, el experimento se estructuró mediante un diseño de bloques pareados: cada par de políticas está constituido por una política entrenada bajo la variante A y otra bajo la variante C compartiendo exactamente la misma semilla de entrenamiento. Se entrenaron 24 pares independientes por cada red sintética (72 pares y 144 políticas en total) a lo largo de 40 iteraciones por corrida, reteniendo para la evaluación el modelo de la iteración final. La justificación del tamaño muestral de 24 pares por red se detalla en la Sección A.6.3 del Apéndice A.

3.6. Evaluación y análisis de políticas

Una vez entrenadas las políticas, su desempeño se evalúa con los pesos neuronales congelados y selección determinista de acciones. A continuación se formalizan las métricas globales y locales de evaluación, el protocolo de inferencia estadística por pares para el contraste A/C y el procedimiento de evaluación de los modelos finales de los cuatro algoritmos.

3.6.1. Métricas de desempeño y vehículos evaluados

Las métricas se organizan en dos niveles: indicadores globales a escala de red, calculados a partir de las trayectorias individuales registradas en `TripInfo` y de los conteos agregados de `Summary`, e indicadores locales a escala de intersección, construidos sobre las mediciones por carril de cada agente.

Métricas globales de viaje y atención a la demanda

En los experimentos de aprendizaje sobre las redes sintéticas, la población de referencia es el conjunto de vehículos evaluados $\mathcal{V}_{\text{ins}}$, definido como el conjunto de vehículos cuyo instante efectivo de inserción en la red (`depart`) pertenece a la ventana de

control $[t_{\text{inicio}}, t_{\text{fin}})$. Para capturar tanto los viajes finalizados como aquellos que continúan circulando al cierre de la ventana, las simulaciones de evaluación activan la opción `-tripinfo-output.write-unfinished` de SUMO [Ecl26e].

La Figura 3.11 ilustra el criterio de pertenencia a $\mathcal{V}_{\text{ins}}$ mediante cuatro trayectorias tipo a través de la ventana $[t_{\text{inicio}}, t_{\text{fin}})$ (franja sombreada), donde el círculo vacío indica el instante de salida programado, el círculo lleno la inserción efectiva y el cuadrado la llegada a destino:

- **Caso 1:** el vehículo ingresa durante el calentamiento ($t < t_{\text{inicio}}$), por lo que queda excluido de $\mathcal{V}_{\text{ins}}$ aunque concluya su recorrido dentro de la ventana.
- **Caso 2:** el vehículo ingresa y arriba a destino dentro de $[t_{\text{inicio}}, t_{\text{fin}})$; integra $\mathcal{V}_{\text{ins}}$, aporta su espera completa y se contabiliza como viaje finalizado.
- **Caso 3:** el vehículo ingresa dentro de la ventana pero continúa en tránsito en t_{fin}; integra $\mathcal{V}_{\text{ins}}$ aportando su espera acumulada hasta el cierre e interviene en el denominador de la tasa de completitud, pero no en su numerador.
- **Caso 4:** el vehículo tiene salida programada pero permanece retenido en la cola externa de inserción hasta t_{fin}, por lo que no ingresa a $\mathcal{V}_{\text{ins}}$ y se refleja en el volumen de inserciones y en el retraso de ingreso (`departDelay`), que no computa en la espera de viaje.

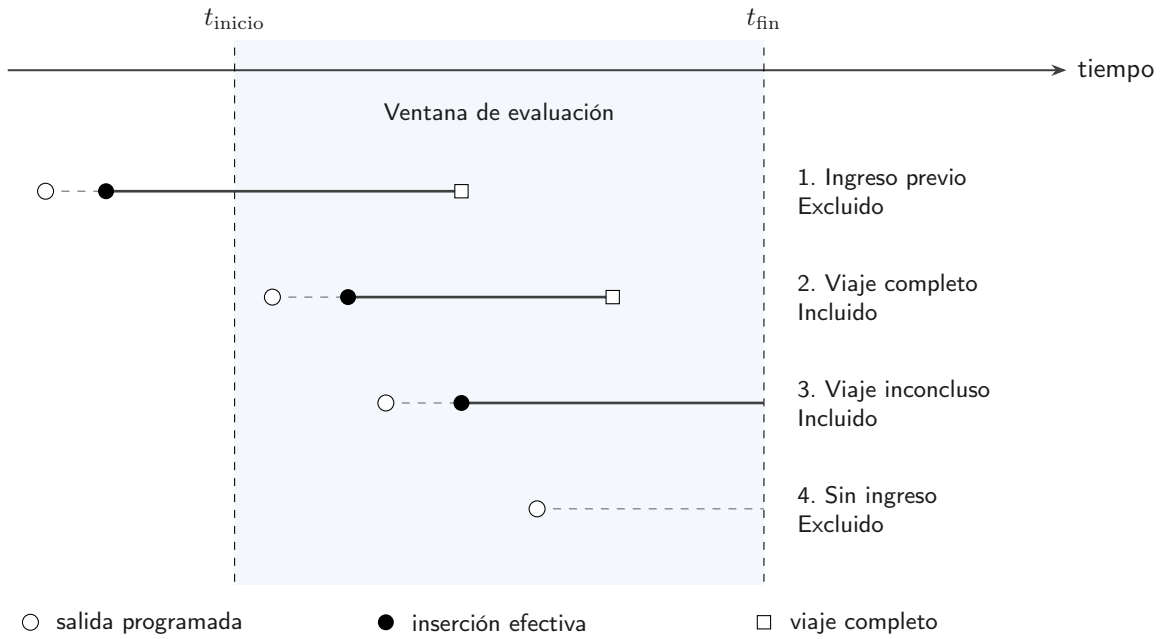


Figura 3.11: Criterio de inclusión en el conjunto de vehículos evaluados ($\mathcal{V}_{\text{ins}}$) según el instante efectivo de inserción.

Sobre este conjunto de vehículos insertados $\mathcal{V}_{\text{ins}}$ (con $|\mathcal{V}_{\text{ins}}| > 0$) se definen tres métricas globales:

- **Espera media por vehículo insertado ($\bar{w}_{\text{ins}}$):** promedio de `waitingTime` (w_v, segundos acumulados a velocidad $\leq 0{,}1\,\text{m/s}$ sin paradas programadas) en segundos por vehículo:
$$\bar{w}_{\text{ins}} = \frac{1}{|\mathcal{V}_{\text{ins}}|} \sum_{v \in \mathcal{V}_{\text{ins}}} w_v, \qquad w_v = \texttt{waitingTime}_v. \tag{3.6}$$

- **Tasa de completitud sobre vehículos insertados (CI):** proporción de vehículos de $\mathcal{V}_{\text{ins}}$ que alcanzan su destino antes de t_{fin}:
$$CI = \frac{|\{v \in \mathcal{V}_{\text{ins}} : \text{completó su viaje antes de } t_{\text{fin}}\}|}{|\mathcal{V}_{\text{ins}}|}. \tag{3.7}$$

- **Pérdida temporal media ($\bar{\ell}$):** promedio del atributo `timeLoss` sobre $\mathcal{V}_{\text{ins}}$ (en segundos por vehículo), que cuantifica el exceso de tiempo de viaje respecto de la circulación en flujo libre [Ecl26e].

Por su parte, en la comprobación operacional del escenario de Mendoza (cuyas corridas se inician desde la red vacía sin calentamiento previo), el balance de flujo al cierre de cada horizonte se resume a partir de los cinco contadores nativos del archivo `Summary` de SUMO [Ecl26c]: los vehículos cargados L (`loaded`), los vehículos insertados I (`inserted`), los viajes completados A (`arrived`), los vehículos en circulación R (`running`) y los vehículos pendientes de ingreso W (`waiting`), vinculados por las identidades de conservación $L = I{+}W$ e $I = A{+}R$. A partir de L, I y A se definen la tasa de completitud (*Completion Rate*, CR), la tasa de inserción (*Insertion Rate*, IR) y el cociente de llegadas sobre inserciones del horizonte:
$$CR = \frac{A}{L}, \qquad IR = \frac{I}{L}, \qquad \frac{A}{I} = \frac{CR}{IR}. \tag{3.8}$$

Métricas locales por agente y equidad espacial

A escala de las intersecciones controladas se calculan dos indicadores complementarios a partir de las esperas por carril $W_i^{(t)}$:

- **Promedio de espera acumulada por agente:** cociente entre la suma temporal de $W_i^{(t)}$ a lo largo de la ventana de control y la suma de los conteos de vehículos presentes $n_i^{(t)}$ en los carriles del agente i en cada paso de decisión (en segundos). Si el denominador es nulo, se informa cero junto con el conteo nulo.

- **Desigualdad espacial de espera (coeficiente de Gini, $G_{\mathbf{wait}}$):** para cada agente $i \in \{1, \ldots, N\}$, se calcula primero la media temporal $\bar{w}_i$ de los cocientes por paso $W_i^{(t)}/n_i^{(t)}$ (asignando cero en los pasos con $n_i^{(t)} = 0$). Ordenando dichas medias de menor a mayor como $\bar{w}_{(1)} \leq \bar{w}_{(2)} \leq \cdots \leq \bar{w}_{(N)}$, la equidad en el reparto de demoras entre intersecciones se cuantifica mediante el coeficiente adimensional de Gini:

$$G_{\text{wait}} = \frac{2\sum_{i=1}^{N} i \cdot \bar{w}_{(i)} - (N+1)\sum_{i=1}^{N} \bar{w}_{(i)}}{N\sum_{i=1}^{N} \bar{w}_{(i)}}. \tag{3.9}$$

  Un valor de G_{wait} cercano a cero indica que la demora media por vehículo presente se distribuye de manera uniforme entre los agentes de la red, mientras que valores elevados reflejan concentración de las colas en un subconjunto reducido de cruces; un valor exactamente nulo con suma de medias positiva indica igualdad entre esas medias y no ausencia de espera en la red.

Incidencias de la simulación

Como control de integridad microscópica, durante las ventanas de evaluación se contabilizan los inicios de teletransporte mediante la consulta `getStartingTeleportIDList` de TraCI, exigiendo cero eventos para considerar operativamente admisible una ejecución.

3.6.2. Evaluación de las formulaciones local (A) y vecinal (C) de MAPPO

Cada una de las 144 políticas entrenadas en la campaña pareada A/C (24 pares por cada una de las tres redes sintéticas) se evaluó sobre cinco réplicas independientes utilizando las semillas de SUMO 71001 a 71005 —comunes a ambas variantes y disjuntas de las semillas de entrenamiento—, totalizando 720 episodios de evaluación en la ventana $[t_{\text{inicio}}, t_{\text{fin}}) = [300, 3.900)$ s tras restaurar el estado posterior a los 300 s de calentamiento.

3.6.3. Análisis por pares y criterio de decisión

Siguiendo las recomendaciones metodológicas para inferencia estadística en aprendizaje por refuerzo profundo [ASC+21], la unidad de observación independiente es el par de entrenamiento $s \in \{1, \ldots, 24\}$, mientras que las cinco corridas de evaluación ($q \in \{1, \ldots, 5\}$) se promedian dentro de cada política para reducir la varianza de simulación. Sea $w_{s,b,q}$ la espera media por vehículo insertado ($\bar{w}_{\text{ins}}$) del par s bajo la variante $b \in \{A, C\}$ en la semilla de evaluación q. La media por política $\bar{w}_{s,b}$, la diferencia absoluta pareada d_s y el cambio relativo pareado z_s se definen como:

$$\begin{aligned} \bar{w}_{s,b} &= \frac{1}{5}\sum_{q=1}^{5} w_{s,b,q}, \\ d_s = \bar{w}_{s,C} - \bar{w}_{s,A}, &\qquad z_s = \frac{\bar{w}_{s,C} - \bar{w}_{s,A}}{\bar{w}_{s,A}}, \end{aligned} \tag{3.10}$$

donde valores negativos de d_s y z_s indican una menor espera a favor de la variante vecinal C. De manera análoga, para la tasa de completitud CI se calcula la diferencia pareada $d_{\mathrm{CI},s} = \overline{\mathrm{CI}}_{s,C} - \overline{\mathrm{CI}}_{s,A}$. Los estimadores puntuales reportados por escenario son los promedios muestrales entre los 24 pares: $\bar{d}$, $\bar{z}$ y $\bar{d}_{\mathrm{CI}}$.

La incertidumbre muestral se cuantifica mediante *bootstrap* no paramétrico con 20.000 remuestreos con reemplazo de los 24 pares de cada red, reportando intervalos de confianza percentiles al 95 % y al 98,33 % (este último correspondiente al nivel global del 95 % con corrección de Bonferroni para las tres redes sintéticas, $\alpha = 0{,}05/3$), complementados con intervalos paramétricos basados en la distribución t de Student como verificación de sensibilidad (Sección A.5.9 del Apéndice A).

Se adopta una decisión favorable a la incorporación de acoplamiento vecinal (variante C) en una red si y solo si se verifican simultáneamente tres condiciones prácticas:

- **Reducción relevante de la espera:** el extremo superior del intervalo de confianza del 98,33 % para el cambio relativo medio $\bar{z}$ es estrictamente menor que $-0{,}05$ (reducción de al menos el 5 %).
- **Preservación de la completitud (CI):** el extremo inferior del intervalo de confianza del 98,33 % para la diferencia media $\bar{d}_{\mathrm{CI}}$ es superior a $-0{,}005$ (merma máxima admisible de 0,5 puntos porcentuales).
- **Ausencia de penalización por teletransportes:** el número de inicios de teletransporte de la variante C no supera al de la variante A en ninguna evaluación pareada con igual semilla.

Las tres redes sintéticas (3×1, 3×3 y $4 \times 4_H$) se evalúan y reportan en pie de igualdad con sus respectivos 24 pares completos.

3.6.4. Entrenamiento y evaluación de los modelos finales

Para la comparación final entre IDQN, IPPO, MAPPO y HAPPO, el protocolo estipula entrenar desde cero un modelo definitivo por cada combinación de algoritmo y escenario que disponga de una configuración seleccionada en la Sección 3.4.5, utilizando una semilla de aprendizaje inédita y el presupuesto completo de 60 iteraciones de la fase 2.

El punto de control (*checkpoint*) de la última iteración de cada modelo final se evalúa de manera determinista a lo largo de 20 simulaciones con semillas de SUMO nuevas (hasta 240 episodios en total para las doce combinaciones sintéticas), manteniendo el calentamiento de 600 s y la ventana de control de 3.600 s (Tabla 3.5). Para cada combinación y métrica se reportan la media muestral y la desviación estándar con divisor $n - 1 = 19$ sobre las 20 réplicas de evaluación, caracterizando el desempeño operativo de las políticas obtenidas sin aplicar descartes retroactivos.

3.7. Reproducibilidad y alcance

La trazabilidad experimental del estudio se sustenta en la preservación de los archivos de red y demanda, las configuraciones de SUMO, las semillas de cada etapa, los dominios e hiperparámetros efectivos y los puntos de control de los modelos entrenados, cuyo inventario se documenta en la Sección A.5 del Apéndice A.

3.7.1. Entorno de cómputo y envío de trabajos

Las campañas de búsqueda de hiperparámetros y la comparación multiescenario A/C se ejecutaron en el clúster de alto desempeño Toko (UNCuyo), utilizando trabajos por lotes gestionados mediante Slurm que preservan de forma aislada los registros y artefactos de cada combinación experimental con aceleración por unidad de procesamiento gráfico (*Graphics Processing Unit*, GPU). La organización de las colas de ejecución y el resguardo de resultados se detallan en la Sección A.5.1 del Apéndice A.

3.7.2. Alcance y limitaciones

El diseño metodológico delimita el alcance de las conclusiones que se extraen en el Capítulo 4. En la comparación de los cuatro algoritmos se entrena un modelo final por combinación de algoritmo y escenario y se lo evalúa sobre veinte realizaciones estocásticas de tráfico, por lo que la dispersión observada en esa línea refleja la variabilidad entre episodios de simulación y no la variabilidad entre entrenamientos independientes [ASC+21]; esta última queda capturada explícitamente en la línea de comparación A/C mediante sus 24 pares de políticas entrenadas desde cero por red. Igualar el presupuesto máximo de iteraciones de búsqueda entre algoritmos con distintas dinámicas de muestreo (*on-policy* en la familia PPO frente a repetición de experiencias *off-policy* en IDQN) establece un marco práctico de comparación bajo recursos finitos, y sus resultados deben interpretarse dentro de los dominios explorados, las poblaciones vehiculares definidas y las condiciones operacionales verificadas en este capítulo.

El contraste A/C evalúa el efecto combinado de la observación y la recompensa vecinales, y no el de cada elemento por separado: la variante C modifica simultáneamente el vector de observación y la función de recompensa e incrementa la dimensión de entrada del actor y del crítico centralizado, de modo que el contraste tampoco separa el aporte de la información vecinal del aumento del número de parámetros ni anticipa el desempeño de variantes con búsquedas de hiperparámetros independientes. La elección de C entre las variantes vecinales no respondió a un criterio previo documentado, por lo que el resultado no establece que C sea la mejor formulación vecinal, y el alcance de esta línea se restringe a MAPPO. La planificación del tamaño muestral se basó en una diferencia absoluta y en un intervalo del 95 %, mientras que el criterio de decisión emplea el efecto relativo y un intervalo del 98,33 %; aquella previsión no garantiza por sí sola la precisión de este último criterio.

El escenario urbano de Mendoza se construyó de forma adaptativa: los ajustes de su demanda se decidieron a partir de fallos observados en corridas preliminares y tres de las semillas ya se habían utilizado con la demanda de 210.000 viajes, de modo que sus resultados con la redistribución final no se observaron antes de fijarla; no se efectuó una comparación completa entre las dos distribuciones con todas las semillas y horizontes comunes, y el margen del 20 % de cambio por par se comprobó respecto de la redistribución inmediatamente anterior y no de forma acumulada respecto de la demanda inicial, con dos pares fuera de ese margen acumulado. En general, las simulaciones tienen horizonte finito, la demanda pendiente de ingreso está permitida y debe informarse junto con los viajes completados, y las seis semillas por horizonte responden a un presupuesto del proyecto y no a un cálculo de precisión estadística: finalizar un horizonte sin teletransportes no demuestra que las colas se mantengan estables fuera de ese horizonte ni que el desempeño observado se conserve en ventanas más extensas. El objetivo J_r de la búsqueda ordena las configuraciones por su demora y por un término de inestabilidad reciente, pero no mide por sí solo la variabilidad entre entrenamientos ni demuestra estabilidad futura.

4

Análisis y discusión de resultados

Este capítulo organiza los resultados por procedimiento: búsqueda de hiperparámetros, evaluación de los modelos finales, comparación pareada A/C de MAPPO y comprobación operacional de Mendoza. Sus métricas y criterios se interpretan según las unidades experimentales definidas en la metodología.

La comprobación de Mendoza corresponde al escenario de 220.000 viajes nominales diarios y programas semafóricos fijos. Sus conteos describen la operación simulada; no evalúan políticas aprendidas.

4.1. Resultados del ajuste de hiperparámetros

La presentación de HPO sigue las dos fases de búsqueda en las tres redes sintéticas. Extender el aprendizaje a Mendoza requiere adaptar y verificar el procedimiento para la demanda y los programas fijos descritos en la metodología. Los dominios y criterios se interpretan según el escenario al que corresponden; la comparación A/C y la comprobación operacional urbana son procedimientos distintos de la búsqueda de hiperparámetros.

4.1.1. Estado de la consolidación y de las búsquedas

4.1.2. Resultados del ajuste y análisis de sensibilidad

4.1.3. Selección de configuraciones de HPO

4.2. Evaluación de políticas y comprobación de escenarios

La comparación A/C se presenta por separado de la selección de configuraciones mediante HPO. Esta tesina analiza las tres redes sintéticas por separado y conserva los pares de entrenamiento de cada una. El informe histórico separó la red 3×3 del análisis principal después de observar los resultados; ese antecedente se distingue del análisis adoptado aquí, como se explica en la sección 3.6.3.

4.2.1. Criterios de evaluación y métricas de comparación

4.2.2. Desempeño en escenarios sintéticos: 3×1, 3×3, 4×4 heterogéneo

4.2.3. Comprobación operacional del escenario de la Ciudad de Mendoza

Las seis ejecuciones de cada ventana completaron el horizonte y cumplieron los criterios de aceptación definidos en la metodología. La Tabla 4.1 resume los doce casos: no hubo teletransportes, colisiones detectadas ni descartes, y al menos el 75 % de los vehículos cargados llegó a destino. La media de viajes completados asigna igual peso a las seis ejecuciones; los rangos muestran sus extremos. Los casos individuales y sus registros se conservan en el apéndice.

Tabla 4.1: Comprobación del escenario urbano con programas fijos: síntesis por ventana.

Ventana	Aceptados	Completados (media)	Pendientes (rango)	Llegaron/ cargados (%)
2 h	6/6	19.896,5	4.813–5.020	76,32–77,17 %
12 h	6/6	125.080,5	25.984–26.529	82,11–82,31 %

En doce horas se completaron, en promedio, 125.080,5 viajes, frente a 152.168,75 solicitudes esperadas por las tasas. Las solicitudes realizadas fluctúan: los cargados estuvieron entre 151.529 y 153.148. Al cierre permanecían entre 778 y 951 vehículos circulando y entre 25.984 y 26.529 pendientes de ingreso. Por tanto, el cumplimiento del criterio de aceptación no equivale a atender toda la demanda ni a servir 220.000 viajes en doce horas. El nominal diario y los conteos del horizonte describen magnitudes diferentes.

La Figura 4.1 muestra la evolución de los vehículos pendientes de ingreso en las seis ejecuciones largas. Cada trazo corresponde a una ejecución; el rayado identifica la hora en que se redistribuyeron las solicitudes. Las trayectorias muestran acumulación de vehículos fuera de la red durante el horizonte, pero no permiten establecer la estabilidad de las colas para tiempos indefinidos.

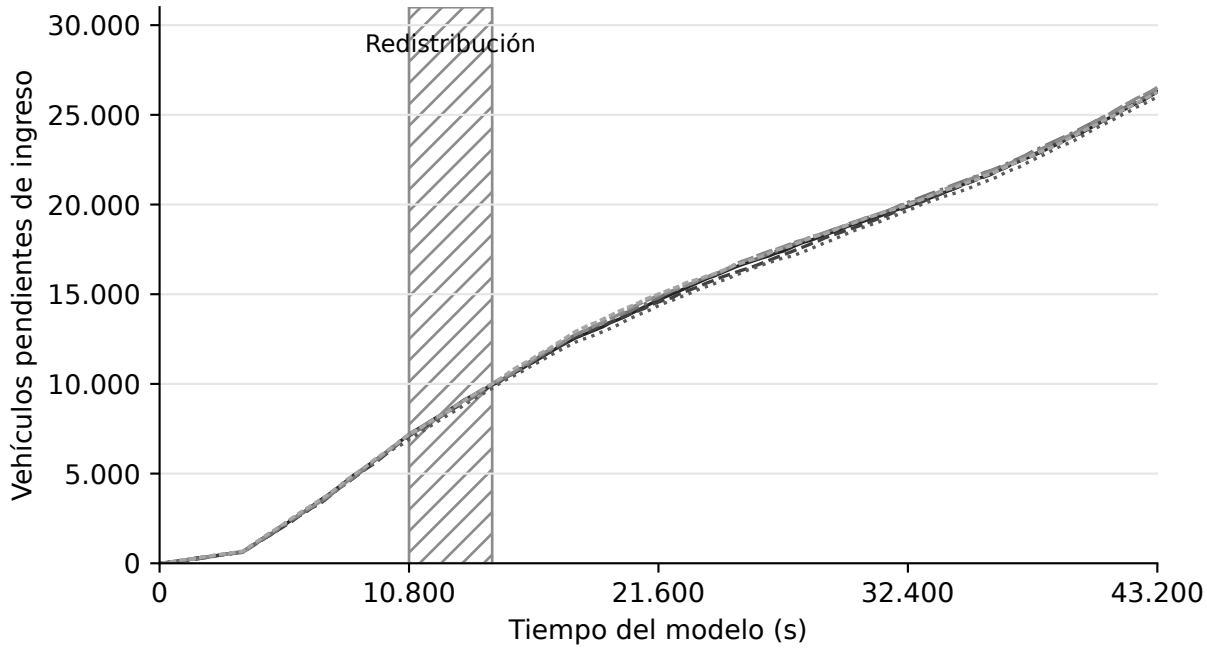


Figura 4.1: Vehículos pendientes de ingreso durante las seis ejecuciones de doce horas. El rayado señala la hora en que se redistribuyeron las solicitudes.

En las corridas de aceptación con programas fijos y horizonte de dos horas llegó a destino entre el 76,32 % y el 77,17 % de los vehículos cargados. Este horizonte termina antes de la redistribución de solicitudes, por lo que no evalúa su efecto. Las corridas no corresponden al ajuste de hiperparámetros previsto para el aprendizaje urbano; los detalles se conservan en el apéndice.

Una ejecución larga con la demanda redistribuida completó el horizonte en condiciones en que la referencia uniforme había fallado. Esta diferencia describe los casos observados, pero no identifica una causa única: los cambios pueden alterar las solicitudes, su orden y la secuencia aleatoria. La selección adaptativa y la reutilización de semillas limitan la generalización. Los resultados de la configuración urbana anterior se conservan en el apéndice como antecedentes; no son resultados del escenario actual ni evaluaciones de políticas aprendidas.

4.3. Discusión e interpretación de resultados

En esta sección se integran los resultados presentados y se discuten sus principales implicancias en relación con los objetivos del trabajo. La lectura de la desigualdad espacial de espera requiere considerar que el coeficiente G_{wait} pondera por igual las intersecciones

con demandas desiguales, de modo que un valor bajo puede reflejar una distribución uniforme de la demora media y, al mismo tiempo, disparidades de carga estructural entre cruces.

5

Conclusiones

Este capítulo sintetiza los hallazgos del estudio, relaciona la evidencia experimental con los objetivos planteados y delimita el alcance de las conclusiones. También identifica las limitaciones del modelo y propone líneas de trabajo futuro. Las afirmaciones finales deben interpretarse dentro de los escenarios, métricas y protocolos descritos en la metodología.

A

Arquitectura de software y especificaciones experimentales

Este apéndice complementa el Capítulo 3 documentando el diseño orientado a objetos del paquete `marlTLC`, las especificaciones detalladas del escenario urbano de Mendoza, los dominios y semillas de las campañas de búsqueda y evaluación, y las formulaciones algebraicas del codificador de observaciones y de la pérdida del crítico. En las tablas y registros del escenario de Mendoza se emplea la nomenclatura interna de las versiones de trabajo: `v14` designa la red mixta preliminar y su demanda de partida (260.000 viajes nominales); `O0`, el reescalado uniforme inicial a 230.000 viajes; `O1`, la primera redistribución espacial de flujos; `O2`, la redistribución final adoptada a 220.000 viajes nominales diarios; y `P2`, el plan coordinado de 73 programas de tiempo fijo de 90 segundos utilizado en la comprobación operacional.

A.1. Diseño orientado a objetos del entorno de simulación

El diseño del entorno separa la gestión del simulador microscópico de la interfaz de aprendizaje multiagente. La clase abstracta `BaseSumoEnvironment` encapsula la inicialización de SUMO, el avance temporal y las consultas de bajo nivel a través de la interfaz de programación de aplicaciones (*Application Programming Interface*, API) de `libsumo` y TraCI. A partir de ella, la subclase `SumoRllibEnv` adapta el ciclo de simulación al contrato `MultiAgentEnv` de Ray RLlib. La Tabla A.1 sintetiza el rol de cada biblioteca principal dentro de la arquitectura.

Tabla A.1: Herramientas y responsabilidades en el sistema experimental.

Herramienta	Función
SUMO y `libsumo`	Simulación microscópica del tránsito e interfaz de control en memoria.
`SumoRllibEnv`	Adaptación del simulador a la interfaz multiagente de entrenamiento.
Ray RLlib	Orquestación distribuida de los procesos de muestreo y optimización.
PyTorch	Cómputo de redes neuronales y actualización de parámetros por gradiente.

La Figura A.1 presenta el diagrama de clases UML de las entidades centrales del entorno. En la columna izquierda, `BaseSumoEnvironment` administra el archivo de configuración, los parámetros temporales (`delta_time`, `sim_time`, `warming_time`), la colección de intersecciones semaforizadas y las operaciones de guardado y recarga de instantáneas de calentamiento; `SumoRllibEnv` especializa dicha base exponiendo los espacios de observación y acción por agente y retornando las tuplas de avance multiagente requeridas por RLlib. En la columna derecha, cada instancia de `TrafficLight` modela un controlador semafórico con su fase activa, sus carriles entrantes asociados (`Lane`), su estrategia de recompensa (`RewardFn`) y sus referencias a controladores vecinos, mientras que `Lane` encapsula el cómputo de las esperas vehiculares acumuladas.

A.2. Jerarquía de funciones de recompensa y patrón *Strategy*

El cálculo de la señal de recompensa se desacopla del bucle principal del entorno mediante los patrones de diseño ***Strategy*** y ***Composite***, lo que permite intercambiar formulaciones locales y vecinales mediante parámetros de configuración. La clase abstracta `RewardFn` define la operación polimórfica `compute_reward(tl_id)`, implementada por estrategias locales como `DiffAccWaitingTimeNormalized` y por agregadores espaciales como `NeighborAvgReward` y `NeighborhoodRelevanceReward` (Figura A.2).

A.3. Patrones de diseño de la arquitectura

La arquitectura de `marlTLC` se apoya en seis patrones de diseño principales:

1. **Estrategia (*Strategy*):** encapsula las formulaciones de recompensa tras la interfaz común `RewardFn`, permitiendo alternar entre señales locales, vecinales y basadas en presión sin modificar `TrafficLight`.

2. **Método de fábrica y registro (*Factory Method* / *Registry*):** instancia funciones de recompensa y módulos neuronales a partir de diccionarios de configuración

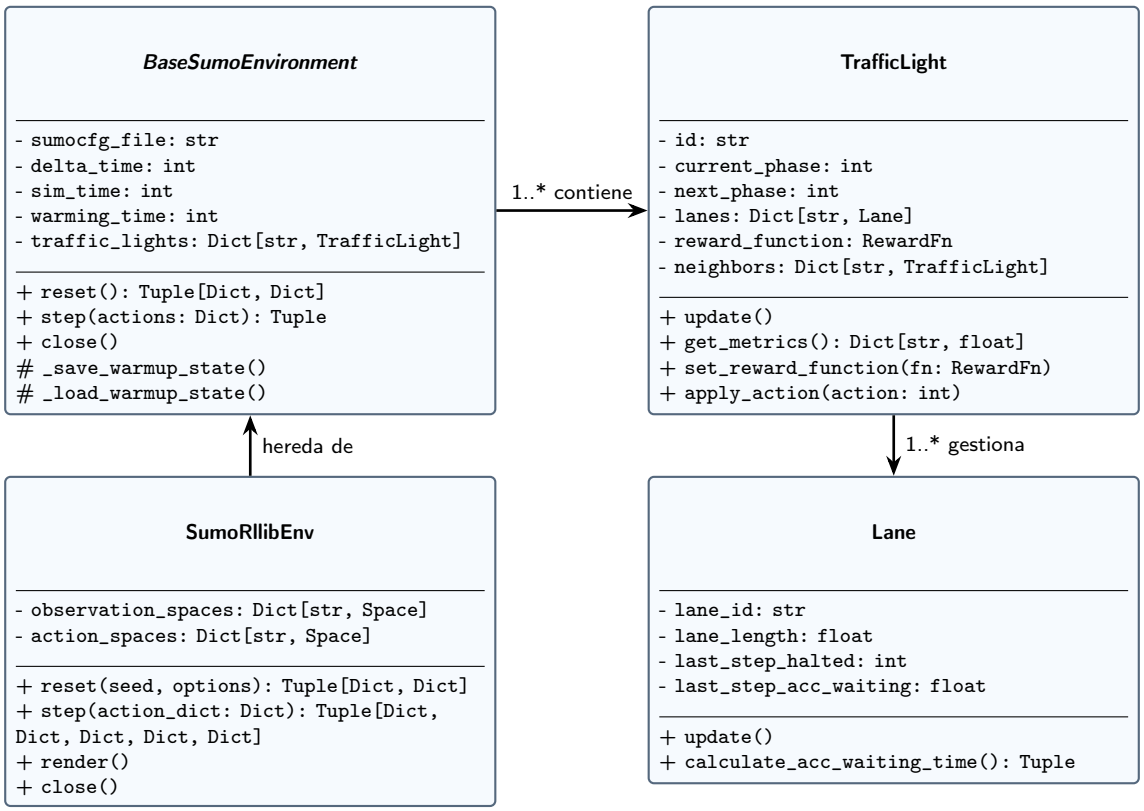


Figura A.1: Diagrama de clases UML del entorno: `SumoRllibEnv` especializa `BaseSumoEnvironment`, que agrega las instancias de `TrafficLight` y sus respectivos carriles (`Lane`).

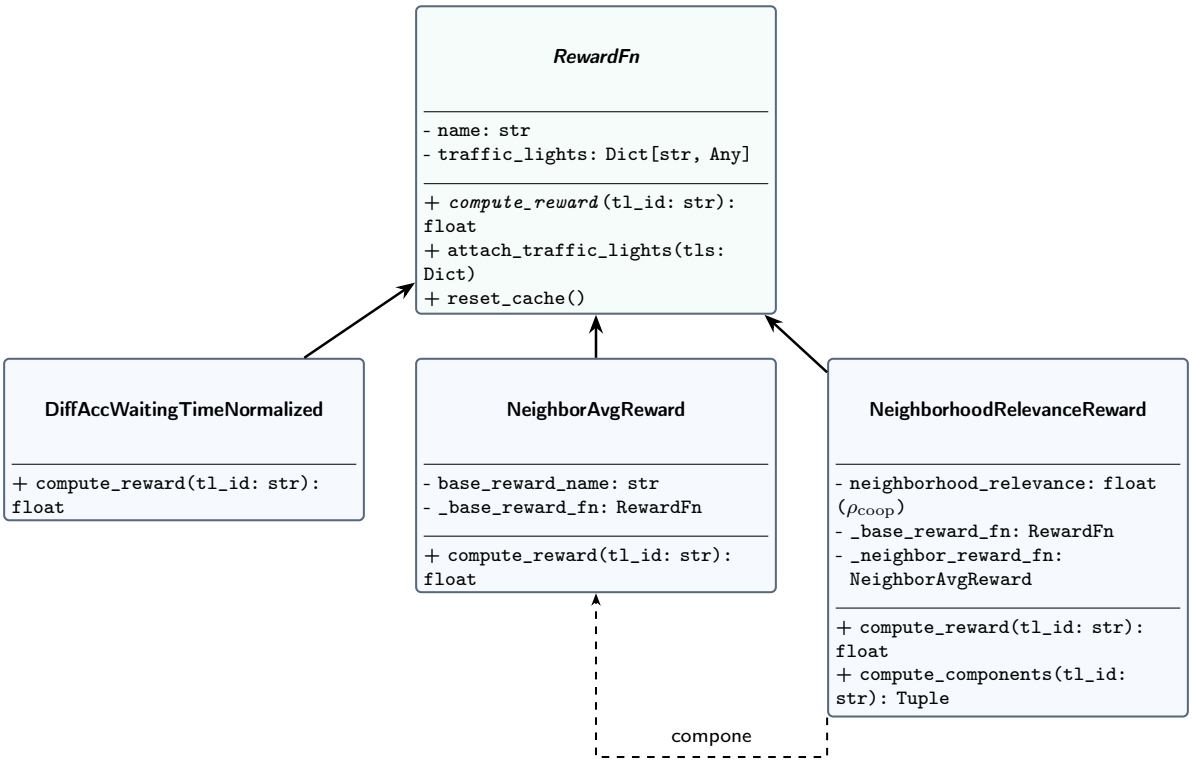


Figura A.2: Diagrama de clases UML del módulo de recompensas: las subclases concretas implementan la interfaz `RewardFn` y componen la señal local con el promedio vecinal ponderado por ρ_{coop}.

(`RewardFactory` y el decorador `@register_reward`), respetando el principio abierto/cerrado.

3. **Objeto compuesto (*Composite*):** combina señales locales y promedios de intersecciones adyacentes (`NeighborhoodRelevanceReward`) bajo la misma interfaz `RewardFn`.

4. **Subtipado estructural (*Protocol*):** desacopla la consulta de métricas vecinales (`NeighborInfoProvider`) de la implementación concreta de `TrafficLight`, facilitando las pruebas unitarias mediante objetos simulados (*mocks*).

5. **Adaptador (*Adapter*):** traduce la API procedural de TraCI y `libsumo` al contrato orientado a objetos `MultiAgentEnv` de Ray RLlib a través de `SumoRllibEnv`.

6. **Decorador (*Decorator*):** extiende declarativamente el registro de componentes y la aplicación de la máscara de acciones sobre los módulos en PyTorch.

A.4. Parámetros de SUMO y programas semafóricos de referencia

En todos los experimentos de aprendizaje sintéticos, el parámetro `waiting_time_memory` de SUMO —que fija la ventana temporal de la métrica `getAccumulatedWaitingTime` [Ecl26d]— se establece en 1.000 s. Las duraciones de los programas semafóricos de referencia de las tres redes sintéticas se presentaron en la Tabla 3.1 del Capítulo 3; a continuación se detallan la caracterización y la conversión a tiempo fijo del escenario urbano de Mendoza.

A.4.1. Mendoza: caracterización de la red y antecedentes de partida

En la red vial modelada del microcentro de Mendoza, la longitud acumulada de todos los carriles asciende a 66,3 km, con tramos individuales de entre 11 y 276 m (media cercana a 120 m) y velocidades máximas reglamentarias de entre 10 y 100 km/h. Los 73 controladores semafóricos gobiernan entre 3 y 10 carriles entrantes cada uno y presentan un grado de vecindad de entre 2 y 4 intersecciones adyacentes (media de 3,23); en términos de su espacio de decisiones verdes, 67 controladores admiten dos fases seleccionables, cuatro admiten una única fase, uno admite tres fases y uno admite cuatro fases.

La red preliminar de partida (`v14`, archivo `gran_mendoza2_fixed.net.xml`) combinaba 17 programas de tiempo fijo con 56 programas accionados por brecha vehicular (con límites `minDur` de 5, 7, 10 o 13 s y `maxDur` de 50 s), cuyas secuencias nominales se resumen en la Tabla A.2. Las corridas históricas preliminares sobre esa red mixta en las ventanas de dos horas ($[1.800, 9.000)$ s) y doce horas ($[0, 43.200)$ s) se reportan como antecedente en

la Tabla A.3, donde L, I y A denotan los vehículos cargados, insertados y completados, respectivamente (Ecuación (3.8)).

Tabla A.2: Duraciones nominales de la red mixta preliminar de Mendoza, en segundos.

Tipo	Cantidad	Secuencia de duraciones
Accionado	1	33, 6, 6, 6, 33, 6
Accionado	5	39, 6, 39, 6
Accionado	19	40, 5, 40, 5
Accionado	5	41, 4, 41, 4
Accionado	24	42, 3, 42, 3
Accionado	1	81, 4, 5
Accionado	1	82, 3, 5
Fijo	1	24, 3, 24, 3, 6, 3, 24, 3
Fijo	2	39, 6, 39, 6
Fijo	3	40, 5, 40, 5
Fijo	1	41, 4, 41, 4
Fijo	8	42, 3, 42, 3
Fijo	2	82, 3, 5

Tabla A.3: Comprobaciones históricas sobre la red mixta preliminar (`v14`).

Ventana	Cargados (L)	Insertados (I)	Completados (A)	A/L	I/L	A/I
2 horas	30.629	23.735	22.702	0,7412	0,7749	0,9565
12 horas	180.163	143.819	143.022	0,7938	0,7983	0,9945

Para construir el perfil horario de demanda a partir de los registros de las cámaras termográficas (agregados en intervalos de 15 minutos y filtrados por zonas válidas `ZoneID` 1–19 y clases vehiculares `ClassID` 1–3), se tomó la curva horaria media $\bar{q}_{\text{obs}}(h)$ de la cámara 169 (asociada al ingreso oriental del tramo `93904440#0` en la zona TAZ 28, con media de 25.175,84 veh/día durante 32 días hábiles) y se escaló inicialmente a un volumen diario nominal $V_{\text{diario}} = 260.000$ veh/día mediante el factor de expansión adimensional $f_{\text{exp}} = 10{,}32735993$ ($\Delta h = 1$ h):

$$f_{\text{exp}} = \frac{V_{\text{diario}}}{\sum_{h=0}^{23} \bar{q}_{\text{obs}}(h)\,\Delta h}, \tag{A.1}$$

$$Q_{\text{red}}(h) = \bar{q}_{\text{obs}}(h) \cdot f_{\text{exp}}, \tag{A.2}$$

donde $Q_{\text{red}}(h)$ expresa el flujo horario nominal de la red (en vehículos por hora).

Sobre las 39 zonas TAZ (numeradas de 0 a 38, con 1.482 pares posibles), una búsqueda en anchura (búsqueda en anchura (*Breadth-First Search*, BFS)) sobre el grafo vial determinó 520 pares OD físicamente conectables. La distribución espacial canónica divide

estos pares en tres subconjuntos disjuntos: $\mathcal{P}_{\text{fijo}}$ (473 pares principales que concentran una proporción fija total de 0,60), $\mathcal{P}_{\text{aleat}}$ (44 pares secundarios que se reparten el 0,40 restante mediante pesos uniformes $u_{od} \sim \mathcal{U}(0,1)$) y $\mathcal{P}_{\text{excl}}$ (tres pares sin demanda asignada):

$$p_{od} = \begin{cases} p_{od}^{\text{fijo}}, & (o,d) \in \mathcal{P}_{\text{fijo}}, \\ 0, & (o,d) \in \mathcal{P}_{\text{excl}}, \\ \dfrac{0,40\, u_{od}}{\sum_{(o',d') \in \mathcal{P}_{\text{aleat}}} u_{o'd'}}, & (o,d) \in \mathcal{P}_{\text{aleat}}. \end{cases} \tag{A.3}$$

Dado un flujo horario $Q_{od}(h) = Q_{\text{red}}(h)\, p_{od} > 0$, la tasa de arribos por segundo es $\lambda_{od} = Q_{od}/3.600$ y los intervalos entre solicitudes sucesivas H_{od} se muestrean según una distribución exponencial a partir de $U \sim \mathcal{U}(0,1)$:

$$H_{od} = -\frac{\ln(U)}{\lambda_{od}}. \tag{A.4}$$

A.4.2. Construcción de los programas semafóricos fijos

Para transformar los 56 controladores accionados de la red preliminar a programas de tiempo fijo con ciclo común de 90 s (conservando los 17 programas originalmente fijos, el orden de fases y los intervalos de despeje), se midieron las duraciones efectivas de sus 224 fases a lo largo de entre 121 y 373 ciclos completos por controlador. Reservando primero la suma de duraciones de las fases fijas $\mathcal{F}$ (d_p) dentro del ciclo de 90 s, el tiempo remanente G se repartió entre las fases variables $\mathcal{V}$ en proporción a sus duraciones medias observadas $\overline{d}_p$:

$$G = 90 - \sum_{p \in \mathcal{F}} d_p, \qquad g_p \approx G\, \frac{\overline{d}_p}{\sum_{j \in \mathcal{V}} \overline{d}_j}, \tag{A.5}$$

aplicando cotas mínimas de verde y ajuste de restos enteros hasta sumar exactamente 90 s. Esta ecuación describe la construcción de los programas; no estima un ciclo óptimo ni acredita la adecuación física de los intervalos de despeje.

Esta conversión inicial (con un desfase de 52 s en el controlador `676748154`) conformó el plan base de tiempo fijo denominado `T0`. A continuación, se optimizaron los desfases de once intersecciones arteriales para minimizar la llegada de pelotones en rojo según los tiempos de recorrido en flujo libre y los volúmenes de las rutas principales, dando lugar al plan fijo final `P2`. La Tabla A.4 detalla los once desfases modificados entre `T0` y `P2`, mientras que las Tablas A.5 a A.7 especifican el desfase y la secuencia completa de duraciones (en segundos) de los 73 controladores del plan `P2` (resguardados en formato tabular en `datos/mendoza220k/programas_tls_completos.csv`).

Tabla A.4: Desfases modificados entre el plan fijo inicial (T0) y el plan fijo coordinado final (P2).

Controlador	T0 (s)	P2 (s)
12990936866	0	1
12990936872	0	21
287042851	0	24
287042852	0	18
287042859	0	82
287042860	0	73
287048129	0	75
287055445	0	13
676748153	0	15
676748171	0	75
cluster_1086742636_2624642126	0	13

Tabla A.5: Programas fijos del plan P2: controladores 1–25.

Controlador	Desfase (s)	Duraciones en orden (s)
10815785964	0	82, 3, 5
10936722273	0	41, 5, 39, 5
10936722274	0	46, 3, 38, 3
12990936866	1	34, 5, 46, 5
12990936872	21	39, 3, 45, 3
1840278959	0	33, 3, 51, 3
1840278966	0	42, 3, 42, 3
1840279037	0	46, 5, 34, 5
1840279039	0	30, 5, 50, 5
1840279043	0	36, 3, 48, 3
1866455049	0	39, 6, 39, 6
286045064	0	64, 3, 20, 3
286045221	0	82, 3, 5
287040660	0	42, 5, 38, 5
287040669	0	44, 3, 40, 3
287040677	0	59, 3, 25, 3
287040684	0	33, 3, 51, 3
287040690	0	47, 5, 33, 5
287040708	0	35, 5, 45, 5
287040724	0	41, 6, 37, 6
287040734	0	53, 5, 27, 5
287040821	0	38, 3, 46, 3
287042850	0	28, 5, 52, 5
287042851	24	27, 5, 53, 5
287042852	18	33, 5, 47, 5

Tabla A.6: Programas fijos del plan P2: controladores 26–50.

Controlador	Desfase (s)	Duraciones en orden (s)
287042859	82	35, 5, 45, 5
287042860	73	38, 5, 42, 5
287044657	0	42, 6, 36, 6
287044661	0	66, 3, 18, 3
287044665	0	24, 3, 24, 3, 6, 3, 24, 3
287048117	0	20, 6, 31, 6, 21, 6
287048125	0	28, 5, 52, 5
287048129	75	42, 5, 38, 5
287055420	0	44, 3, 40, 3
287055424	0	43, 5, 37, 5
287055428	0	39, 3, 45, 3
287055431	0	59, 3, 25, 3
287055445	13	51, 3, 33, 3
287055449	0	34, 4, 48, 4
287055452	0	81, 4, 5
287055464	0	41, 4, 41, 4
287055465	0	42, 4, 40, 4
5364445246	0	41, 3, 43, 3
5364445248	0	45, 3, 39, 3
5364445249	0	39, 3, 45, 3
5364451013	0	48, 5, 32, 5
676748101	0	41, 4, 41, 4
676748105	0	31, 4, 51, 4
676748107	0	42, 6, 36, 6
676748143	0	32, 3, 52, 3

Tabla A.7: Programas fijos del plan P2: controladores 51–73.

Controlador	Desfase (s)	Duraciones en orden (s)
676748153	15	46, 3, 38, 3
676748154	52	49, 3, 35, 3
676748155	0	39, 3, 45, 3
676748156	0	40, 3, 44, 3
676748161	0	40, 5, 40, 5
676748167	0	42, 3, 42, 3
676748171	75	30, 5, 50, 5
676748182	0	35, 5, 45, 5
676748185	0	51, 6, 27, 6
676748186	0	59, 3, 25, 3
676748187	0	39, 3, 45, 3
cluster_1086062886_286045446	0	45, 6, 33, 6
cluster_1086062891_676748165	0	42, 3, 42, 3
cluster_1086062935_287048132	0	40, 5, 40, 5
cluster_1086062975_287055453	0	23, 4, 59, 4
cluster_1086742602_437558121	0	42, 3, 42, 3
cluster_1086742611_437558117	0	40, 5, 40, 5
cluster_1086742616_437558119	0	82, 3, 5
cluster_1086742636_2624642126	13	42, 3, 42, 3
cluster_1086742680_2606023806_437558123	0	42, 3, 42, 3
cluster_1086742689_437558120	0	42, 3, 42, 3
cluster_1086742734_437558124	0	42, 3, 42, 3
cluster_2606030890_437558126_623948419	0	39, 6, 39, 6

A.4.3. Transformaciones de la demanda y selección del nominal

La obtención de la demanda final `O2` a partir de la demanda preliminar `v14` se realizó en tres pasos conservando el soporte de 517 pares OD activos y las 6.204 celdas flujo–hora:

1. **Reescalado inicial (`O0`) y primera redistribución (`O1`):** las tasas de `v14` se escalaron por 230/260 (`O0`) y luego se aplicaron 436 intercambios heurísticos de flujo (`O1`) acotados al rango [0,80; 1,20] de cada celda original (variación total del 2,968 %), preservando exactamente los totales horarios, las marginales esperadas por origen y destino y todas las celdas vinculadas a la zona TAZ 28.

2. **Reescalado a 220.000 viajes y redistribución local en Tiburcio Benegas (`O2`):** tras reescalar `O1` por el factor 220/230, se introdujo un intercambio balanceado entre cuatro pares OD exclusivamente en la cuarta hora de simulación ($[10.800, 14.400)$ s, 09:00–10:00) para aliviar la carga sobre el acceso recto de Tiburcio Benegas. Como muestra la Tabla A.8, en el catálogo histórico de rutas el par $10 \to 27$ presentaba 87 de sus 209 itinerarios sobre dicho movimiento, por lo que se redujo su tasa en $\delta_\lambda = 0{,}001117230520$ veh/s (20 % de la menor tasa del cuarteto) y se compensó de forma cruzada con los pares $36 \to 15$, $10 \to 15$ y $36 \to 27$ (Tabla A.9), manteniendo inalteradas las sumas marginales por origen y destino (8,044 solicitudes esperadas transferidas sobre las 14.852,23 de esa hora). El margen del 20 % es local respecto de `O1`: frente a `O0` reescalada, el cociente `O2`/`O0` del par $10 \to 15$ es aproximadamente 1,214586 y el de $36 \to 15$, 0,768017, por lo que no se conserva íntegramente la envolvente inicial ni se revalidaron para `O2` los topes de exposición de `O1`.

Tabla A.8: Rutas del catálogo histórico asociadas con el movimiento recto analizado en Tiburcio Benegas.

Par OD	Rutas del catálogo	Rutas con el movimiento
10→27	209	87
36→15	196	0
10→15	214	0
36→27	191	0

Tabla A.9: Tasas de arribo de los cuatro pares que intervienen en el intercambio final `O2` durante el intervalo $[10.800, 14.400)$ s.

Par OD	O1 a 220.000 (veh/s)	O2 (veh/s)
10→15	0,006507082443	0,007624312963
10→27	0,006493809948	0,005376579428
36→15	0,005586152603	0,004468922083
36→27	0,006399436153	0,007516666673

La Tabla A.10 sintetiza el barrido de niveles nominales diarios evaluados bajo el plan fijo `P2`. El nivel de 220.000 viajes con la distribución `O2` fue seleccionado como escenario definitivo de trabajo por constituir el máximo volumen que completó satisfactoriamente las seis semillas de verificación en ambos horizontes (en la semilla común 104759 a doce horas, `O2` a 220.000 completó 125.100 viajes con $CR = 0{,}8220$, superando en volumen absoluto atendido los 120.274 viajes completados por `O1` a 210.000 con $CR = 0{,}8280$).

Tabla A.10: Síntesis de los tratamientos evaluados para la elección del volumen nominal diario de Mendoza.

Nominal diario	Tratamiento	Resultado en las corridas de verificación
260.000	`v14` mixto	Combina controladores accionados y fijos; antecedente de partida.
230.000	`O1`/`P2`	Dos semillas completan doce horas; la semilla 104759 se satura antes del cierre.
225.000	`O2` reescalada/`P2`	No completa la tanda requerida de seis réplicas en el horizonte de doce horas.
220.000	`O1` reescalada/`P2`	Supera el horizonte de dos horas con la semilla 104759, pero falla en doce horas.
220.000	`O2`/`P2`	Las seis semillas completan satisfactoriamente los horizontes de dos y doce horas.
210.000	`O1` reescalada/`P2`	Las seis semillas completan ambos horizontes con menor volumen absoluto servido.

A.4.4. Entradas, ejecución y evidencia final

Las doce corridas de comprobación operacional del escenario `O2`/`P2` se ejecutaron con el binario `v1_25_0+0570-f59cd14e561` de SUMO empleando las seis semillas predefinidas (104759, 104729, 104771, 104861, 104879 y 104891) sobre las ventanas de dos horas ($[1.800, 9.000)$ s, 06:30–08:30) y doce horas ($[0, 43.200)$ s, 06:00–18:00), partiendo en todos los casos desde la red vacía. El inventario completo de archivos de entrada y salida, junto con sus sumas de verificación criptográfica SHA-256, se preserva en `datos/mendoza220k/fuentes_locales_sha256.json` y es validado automáticamente mediante el script reproducible `datos/mendoza220k/generar.py`.

La Tabla A.11 enumera las opciones de configuración de SUMO utilizadas en la campaña, y las Tablas A.12 y A.13 reportan los contadores exactos al cierre de cada una de las doce ejecuciones (donde L, I, A, R y W denotan los vehículos cargados, insertados, completados, aún circulando al cierre y pendientes de ingreso, respectivamente, y $CR = A/L$), todas las cuales finalizaron con cero teletransportes, cero colisiones detectadas y cero descartes, verificando exactamente las identidades de conservación $L = I + W$ e $I = A + R$. La ausencia de colisiones detectadas depende de las opciones de simulación

empleadas, en particular de que no se activó `collision.check-junctions`, y no certifica la seguridad física de los cruces.

Tabla A.11: Opciones efectivas de SUMO en las corridas de comprobación operacional de Mendoza.

Opción	Valor
Paso de simulación	1 segundo
`time-to-teleport`	300 segundos
`max-num-teleports`	0
`max-depart-delay`	-1 (sin descarte por demora de ingreso)
`collision.action`	`warn`
`junction-taz`	`true`
`summary-output.period`	1 segundo
`tripinfo-output.write-unfinished`	`true`
`tripinfo-output.write-undeparted`	`true`
`xml-validation`	`never` (entradas validadas previamente)

Tabla A.12: Corridas de aceptación con programas fijos (`O2`/`P2`): horizonte de dos horas.

Semilla	L	I	A	R	W	A/L
104759	26.142	21.211	20.032	1.179	4.931	0,766276
104729	25.912	20.892	19.857	1.035	5.020	0,766324
104771	25.668	20.798	19.702	1.096	4.870	0,767571
104861	25.717	20.901	19.846	1.055	4.816	0,771707
104879	25.986	21.173	20.045	1.128	4.813	0,771377
104891	26.070	21.088	19.897	1.191	4.982	0,763214

Tabla A.13: Corridas de aceptación con programas fijos (`O2`/`P2`): horizonte de doce horas.

Semilla	L	I	A	R	W	A/L
104759	152.181	125.898	125.100	798	26.283	0,822047
104729	152.418	126.107	125.156	951	26.311	0,821137
104771	151.529	125.545	124.723	822	25.984	0,823097
104861	152.153	125.832	124.977	855	26.321	0,821390
104879	153.148	126.619	125.841	778	26.529	0,821695
104891	151.752	125.491	124.686	805	26.261	0,821643

A.5. Especificaciones para reproducir los experimentos

Este apéndice distingue los valores observados en configuraciones y archivos de ejecución de los parámetros, dominios y semillas que aparecen en los diseños de búsqueda. En este contexto, fase 1 y fase 2 designan etapas de búsqueda de hiperparámetros.

A.5.1. Infraestructura y almacenamiento de las ejecuciones

Los programas de envío de trabajos, o *scripts*, especifican recursos solicitados; se distinguen de los registros de asignación efectiva. La metodología resume las plataformas utilizadas en la subsección 3.7.1.

Los trabajos se envían con `sbatch`, comando del gestor de trabajos herramienta simple de Linux para gestión de recursos (*Simple Linux Utility for Resource Management*, SLURM). En las búsquedas sintéticas, fase 1 utiliza `submit_phase1.sh` y fase 2 especifica `submit_phase2_refined.sh`.

Las configuraciones de los scripts corresponden a una combinación de algoritmo, red de tráfico y arquitectura, y solicitan un nodo de cómputo, una tarea de ejecución, diez núcleos de unidad central de procesamiento (*Central Processing Unit*, CPU) y una partición con aceleración por GPU. El manual institucional describe una partición de Toko con hasta ocho núcleos; por ello no permite confirmar los recursos físicos asignados a cada trabajo [Tok24].

El script refinado también especifica el uso exclusivo del nodo y un límite de tiempo. La configuración de Ray limita a un ensayo simultáneo por trabajo; ASHA define las detenciones anticipadas según los parámetros de cada búsqueda.

Los registros de las ejecuciones documentadas se guardaron en almacenamiento persistente y se copiaron al espacio temporal del nodo durante la ejecución. Aunque los scripts configuraron un recurso de GPU para Ray, no solicitaron una tarjeta física mediante un identificador de recurso de SLURM; por ello, los archivos del proyecto no permiten asociar cada trabajo con un modelo específico de GPU.

A.5.2. Comprobaciones preliminares

En una comprobación de 3×3, cerca del 70 % de los vehículos cargados —incluidos en la demanda de SUMO— completó el viaje; este dato no se extrapola a las otras redes. Esa comprobación incluyó 600 segundos de calentamiento y 3600 segundos de control.

En la prueba preliminar de formulaciones de MAPPO (subsección 3.5.1), B y D incluyen ejecuciones completadas y no se clasifican como fallos técnicos. Esta comprobación distingue el estado de ejecución de las diferencias de desempeño.

A.5.3. Detalles de las redes sintéticas

La Tabla 3.3 reúne las dimensiones comparables de las redes. Este apartado conserva las excepciones topológicas y las asignaciones de demanda por par origen–destino (OD) que explican la función de cada escenario.

En el corredor 3×1, los cruces B1, C1 y D1 se disponen en línea. El 75 % del flujo solicitado se asigna a los pares de zonas $3 \rightarrow 4$ (40 %) y $4 \rightarrow 3$ (35 %); el 25 % restante se distribuye de manera reproducible entre los pares válidos.

La red 3×3 forma una cuadrícula regular. Sus asignaciones principales son 20 % para $5 \rightarrow 6$, 17,5 % para $6 \rightarrow 5$, 17,5 % para $10 \rightarrow 1$ y 20 % para $1 \rightarrow 10$; el 25 % restante se reparte entre los pares válidos.

En $4 \times 4_H$, la fila 2 tiene tres carriles por sentido, mientras que el sector A1–B1–C1 de la fila 1 tiene uno por sentido. La columna C es asimétrica, con cuatro carriles hacia el sur y dos hacia el norte. La columna D solo permite circular hacia el norte: tiene cuatro carriles hasta D3 y tres desde D3 hasta el borde norte. Las demás vías tienen dos carriles por sentido donde existe circulación. Faltan los enlaces horizontales B4–C4 y D1–E1, por lo que la red no es una malla completa.

C1 tiene tres acciones seleccionables. C4 y D1 tienen dos, aunque sus programas de referencia contienen cinco fases, porque las fases amarillas y el estado final de todo rojo no son acciones de los agentes. B4 y E1 tienen un vecino; los demás agentes tienen entre dos y cuatro. La demanda asigna 20 % a $5 \rightarrow 9$, 17,5 % a $6 \rightarrow 5$, 17,5 % a $13 \rightarrow 1$ y 20 % a $1 \rightarrow 13$; el 25 % restante se distribuye entre pares válidos. Estas diferencias de capacidad, conectividad y demanda se presentan como características del escenario, no como una variación controlada de un único factor.

A.5.4. Espacio de búsqueda inicial de fase 1

Esta sección especifica los dominios iniciales por algoritmo y los factores que se mantienen fijos. Los rangos se contrastaron con el código de `experiments/marl_experiments/tuning/shared/search_space.py` y con la revisión documental del 10 de octubre de 2026 conservada en `docs/initial_search_space_criteria.md`, dentro de `marl-tlc`. El protocolo anterior de Mendoza contemplaba estos dominios, salvo el peso vecinal, fijado en cero después de A/C. Las tablas posteriores de presupuestos y semillas corresponden a las búsquedas sintéticas; las condiciones urbanas se reúnen en la subsección A.5.8. Esta reconstrucción permite identificar la especificación consultada; una clave almacenada en un archivo de configuración no demuestra por sí sola cómo operó cada ejecución histórica.

Las Tablas A.14 y A.15 reúnen los dominios por familia algorítmica. Cada fila identifica el parámetro por su función; la Tabla A.17 conserva su correspondencia con las claves del código. Uniforme continua y loguniforme tienen el significado explicado en la sección 3.4.3; uniforme entera permite cualquiera de los enteros del rango, incluidos ambos extremos, y categórico restringe la búsqueda a las alternativas enumeradas. Los intervalos continuos

conservan la notación de las distribuciones declaradas; la inclusión de cero como extremo no garantiza una propuesta exactamente nula. En las llamadas enteras del código, el límite superior es exclusivo: por ejemplo, `randint(4,65)` permite los enteros de 4 a 64. La excepción urbana del peso vecinal también se aplica a estas tablas. Otros factores, como el límite de escala del codificador de espera acumulada por carril, se fijan por escenario. La Tabla A.16 especifica esos factores para los estudios sintéticos documentados.

En la familia PPO, el margen de recorte limita la razón de probabilidades del actor y el parámetro de GAE regula la estimación de ventajas. La entropía favorece la exploración; los parámetros de divergencia KL configuran la penalización y su adaptación. El límite del gradiente acota la magnitud del gradiente utilizado para actualizar los parámetros. Las cantidades de episodios, recorridos de optimización y minilotes controlan la recolección y reutilización de experiencias. En MAPPO y HAPPO, la cota de valor limita la pérdida cuadrática del crítico, según la sección A.6; no se interpreta como un límite físico de espera. En HAPPO, la probabilidad de ordenación determina con qué frecuencia se elige el siguiente actor según la magnitud de su ventaja, en lugar de elegirlo al azar; su aplicación se precisa después de la tabla.

Tabla A.14: Espacio inicial de fase 1 para MAPPO, IPPO y HAPPO. Las excepciones por algoritmo se indican en las filas.

Parámetro	Dominio	Muestreo
Peso de recompensa vecinal	[0; 0,95)	Uniforme continua
Tasa de aprendizaje	$[3 \times 10^{-5}; 10^{-3}]$	Loguniforme
Recorte del actor	[0,10; 0,22)	Uniforme continua
Parámetro de GAE	[0,88; 0,99)	Uniforme continua
Peso de entropía	[0; 0,01)	Uniforme continua
Límite del gradiente	[0,25; 5]	Loguniforme
Coeficiente de penalización KL	[0,05; 0,5]	Loguniforme
Objetivo de divergencia KL	[0,005; 0,03]	Loguniforme
Cota de pérdida de valor	[5; 20]	Loguniforme
Recorridos de optimización	5–15; HAPPO: 5–18	Uniforme entera
Episodios por lote	2–12	Uniforme entera
Niveles del codificador	4–64	Uniforme entera
Tamaño del minilote*	{128; 256; 512}	Categórico
Probabilidad de ordenación (HAPPO)	[0,20; 0,90)	Uniforme continua

El asterisco de `minibatch_size` señala una diferencia de implementación: el módulo de aprendizaje de HAPPO consultado procesa el lote completo por actor, aunque la configuración registre un tamaño de minilote. En ese módulo, la clave registrada no determina una subdivisión efectiva del lote. La probabilidad de ordenación, en cambio, interviene al elegir el siguiente actor: selecciona al de mayor magnitud media de ventaja con la probabilidad configurada y, en caso contrario, elige al azar entre los restantes. Su dominio

excluye tanto la ordenación completamente aleatoria como la selección siempre guiada por ese criterio.

En las redes sintéticas, para PPO, entre 2 y 12 episodios nominales por lote equivalen a un objetivo de 1440 a 8640 pasos de decisión, al multiplicar por $3600/5 = 720$ pasos por episodio controlado. La recolección de episodios completos puede producir un tamaño efectivo distinto. Los niveles del codificador modifican la representación de las observaciones; el peso vecinal modifica la recompensa. No son parámetros del optimizador y se distinguen de la inclusión de información vecinal en la observación, que permanece desactivada durante esta búsqueda.

En IDQN, la probabilidad de exploración desciende desde uno hasta el valor final propuesto, durante el número de pasos configurado. La red objetivo se actualiza con la frecuencia indicada, y el horizonte de varios pasos fija cuántas transiciones intervienen en el retorno utilizado para aprender. Sus frecuencias y contadores corresponden a pasos de entrenamiento o muestreo según la interfaz de RLlib, no a segundos de SUMO. El lote de IDQN contiene transiciones y no equivale al número de episodios utilizado para los métodos PPO.

Tabla A.15: Espacio inicial de fase 1 para IDQN. Los contadores se expresan en pasos según la interfaz del algoritmo.

Parámetro	Dominio	Muestreo
Peso de recompensa vecinal	[0; 0,95)	Uniforme continua
Tasa de aprendizaje	$[10^{-5}; 10^{-3}]$	Loguniforme
Transiciones por lote	{64; 128; 256; 512}	Categórico
Límite del gradiente	{0,5; 1; 2; 5; 10}	Categórico
Actualización de red objetivo	{500; 1.000; 2.000; 5.000}	Categórico
Pasos para reducir exploración	{5.000; 10.000; 20.000}	Categórico
Exploración final	[0,02; 0,10)	Uniforme continua
Horizonte del retorno	{1; 3; 5}	Categórico
Pasos antes de aprender	{1.000; 2.000; 5.000}	Categórico
Niveles del codificador	4–64	Uniforme entera

La Tabla A.16 separa los factores de estudio y los valores fijos de los parámetros anteriores. La arquitectura se conserva dentro de cada estudio para evitar que RLlib 2.53 altere la arquitectura al almacenarla dentro de otros campos de configuración, según la decisión registrada en `experiments/marl_experiments/tuning/shared/architectures.py`. El límite de escala del codificador de espera acumulada por carril se calibra por escenario; su fijación no implica que el percentil utilizado sea superior a otras alternativas.

Tabla A.16: Factores de estudio y valores fijos documentados para la fase 1 sintética.

Factor	Valor o alternativa
Arquitectura por estudio	h128 o h256: dos capas de 128 o 256 unidades
Descuento	$\gamma = 0{,}99$
Medición de espera	Espera acumulada por carril (veh·s)
Límite de escala del codificador, 3×1	1184 veh·s
Límite de escala del codificador, 3×3	1136 veh·s
Límite de escala del codificador, $4 \times 4_H$	712 veh·s
Información observada	Observación local y máscara de acciones
Penalización KL	Activada en los métodos PPO

La arquitectura permanece fija por estudio, y el límite de escala del codificador de espera acumulada por carril varía según el escenario. La medición de espera y el descuento son comunes a los cuatro algoritmos. Variar el peso de recompensa vecinal no activa la observación vecinal. En los métodos PPO, la penalización KL permanece activada, aunque su coeficiente y su objetivo se exploran.

Los rangos y distribuciones documentan cómo se acotó la exploración. No se recuperó una justificación contemporánea para todos sus límites exactos, ni una comparación que validara las alternativas de cada fila. La calibración de espera y la separación de arquitecturas tienen un respaldo local identificable; los demás límites se conservan como decisiones del diseño exploratorio. Las propuestas adaptativas de TPE, las detenciones de ASHA y la única semilla de entrenamiento por escenario limitan las inferencias sobre sensibilidad y variabilidad. La especificación no garantiza cobertura exhaustiva del espacio ni optimalidad global.

Tabla A.17: Correspondencia entre los parámetros de búsqueda y sus claves de configuración.

Parámetro	Clave en el código
Peso de recompensa vecinal	`neighborhood_relevance`
Tasa de aprendizaje	`lr`
Recorte del actor	`clip_param`
Parámetro de GAE	`lambda_gae`
Peso de entropía	`entropy_coeff`
Límite del gradiente	`grad_clip`
Coeficiente de penalización KL	`kl_coeff`
Objetivo de divergencia KL	`kl_target`
Cota de pérdida de valor	`vf_clip_param`
Recorridos de optimización	`num_epochs`
Episodios por lote	`train_batch_episodes`
Niveles del codificador	`encoder_intervals`
Tamaño del minilote	`minibatch_size`
Probabilidad de ordenación (HAPPO)	`greedy_selection_prob`
Transiciones por lote	`train_batch_size_per_learner`
Actualización de red objetivo	`target_network_update_freq`
Pasos para reducir exploración	`epsilon_timesteps`
Exploración final	`epsilon_final`
Horizonte del retorno	`n_step`
Pasos antes de aprender	`num_steps_sampled_before_learning_starts`

A.5.5. Semillas de fase 1 en las redes sintéticas

La Tabla A.18 reúne las semillas observadas en las configuraciones serializadas de fase 1 y las asociadas a los perfiles canónicos de generación de rutas. La semilla de entrenamiento inicializa el aprendizaje y la simulación SUMO durante el entrenamiento; la de evaluación controla SUMO en las evaluaciones diagnósticas.

Tabla A.18: Semillas por escenario sintético en la fase 1.

Escenario	Entrenamiento	Evaluación	Rutas
3×1	101	100101	1333938385
3×3	137	100137	810840844
$4 \times 4_H$	179	100179	1871007461

El código de fase 1 define 7001 como semilla TPE por defecto, independiente de las semillas de entrenamiento.

A.5.6. Arquitecturas y semillas de la búsqueda refinada sintética

Este apartado detalla la búsqueda refinada sintética, posterior a la prueba A–E y a la comparación A/C de MAPPO. En las tres redes sintéticas, cada red deriva sus dominios de su propia fase 1. El antecedente urbano de v14 se conserva en la Sección A.5.8.

El protocolo sintético de fase 2 fija un máximo de 100 configuraciones por combinación de algoritmo y escenario y de 60 iteraciones por ensayo. La configuración de ASHA fija 30 iteraciones de gracia y un factor de reducción de 2; además, establece una ejecución concurrente por combinación y un episodio de seguimiento sin exploración cada 15 iteraciones. Los fallos, las detenciones anticipadas y las interrupciones consumen parte del presupuesto. Las semillas y las arquitecturas se establecen por algoritmo y escenario. Las fuentes históricas con rangos o arquitecturas distintos conservan su identidad.

El estado del modelo al final de un ensayo se guarda en un archivo *checkpoint*, que puede servir para reanudar o evaluar el entrenamiento. Su existencia no libera presupuesto si el ensayo está incompleto ni garantiza que la política sea admisible para selección. En la búsqueda sintética, la fase 1 adapta las propuestas mediante TPE y las detenciones mediante ASHA, sin reservar una prueba en $\rho_{\text{coop}} = 0$; por eso sus registros no identifican la causa de las diferencias asociadas al peso vecinal ni demuestran que el valor local sea un óptimo.

La selección de configuraciones, el entrenamiento final desde cero y la evaluación comparativa son etapas posteriores a las búsquedas. Las semillas 950001, 950003 y 950007 se utilizan en la evaluación de candidatos sintéticos descrita en el apartado de admisibilidad y selección de la subsección A.5.6.

Tabla A.19: Arquitecturas y semillas de la búsqueda refinada para las doce combinaciones sintéticas. La semilla de evaluación es común a los algoritmos del mismo escenario; las semillas de TPE no son réplicas de entrenamiento.

Algoritmo	Escenario	Arquitectura	Entrenamiento	Evaluación	TPE
IPPO	3×1	h256	910001	920001	930001
IPPO	3×3	h128	910002	920003	930002
IPPO	$4 \times 4_H$	h128	910003	920004	930003
MAPPO	3×1	h128	910004	920001	930004
MAPPO	3×3	h128	910005	920003	930005
MAPPO	$4 \times 4_H$	h128	910006	920004	930006
HAPPO	3×1	h256	910007	920001	930007
HAPPO	3×3	h256	910008	920003	930008
HAPPO	$4 \times 4_H$	h128	910009	920004	930009
IDQN	3×1	h256	910010	920001	930010
IDQN	3×3	h128	910011	920003	930011
IDQN	$4 \times 4_H$	h256	910012	920004	930012

Las arquitecturas h128 y h256 contienen dos capas ocultas de 128 y 256 unidades, res-

pectivamente. La arquitectura asignada a cada combinación prevista se indica en la Tabla A.19. Son preferencias de búsqueda derivadas de la fase 1, no evidencia de superioridad bajo la recompensa local. Las semillas TPE controlan las propuestas; las semillas SUMO, la simulación. Ninguna sustituye una réplica de entrenamiento.

Admisibilidad y selección de candidatos

Para los candidatos sintéticos, la selección exige que el ensayo haya alcanzado 60 iteraciones, que sus métricas sean numéricas y finitas y que existan al menos 15 registros finitos de métricas de tránsito. También se comprueba que la configuración efectiva y el archivo del modelo final estén completos y puedan leerse. Por sí solo, el estado `COMPLETE` de Optuna indica que el ensayo terminó sin registrar un fallo, pero no garantiza el cumplimiento de estos requisitos.

La lista inicial de candidatos sintéticos se construye con `build_shortlist`, en `experiments/marl_experiments/tuning/phase2_refined/closure_audit.py`, de la copia `phase2-cluster` de `marl-tlc`. Entre los ensayos técnicamente admisibles se consideran aquellos con espera diagnóstica disponible. Se priorizan los que registran cero inicios de teletransporte y una CI diagnóstica de al menos 0,95; después se ordenan por menor espera diagnóstica y por el identificador de la política. Se toma el primero de cada arquitectura disponible y se completa la lista, hasta tres candidatos, con los restantes en ese orden. Este ordenamiento preliminar no sustituye los filtros del panel común de selección. La adaptación de la selección urbana prevista se conserva como antecedente en la Sección A.5.8 de este apéndice; el escenario O2/P2 requiere un protocolo de selección propio que aún no se ha ejecutado.

Cada candidato se evalúa con las semillas SUMO 950001, 950003 y 950007, comunes a los algoritmos de una misma red. Los tres episodios deben registrar cero inicios de teletransporte, una CI de al menos 0,95 y una cantidad de vehículos insertados igual o superior al 95 % de la referencia empírica para esa red y semilla.

Para cada red y semilla, la referencia de inserciones es el máximo observado entre los episodios del panel inicial, que reúne los candidatos de los cuatro algoritmos. La lista de candidatos y las semillas se conservan antes de iniciar el panel mediante `closure_panel.py`. Una vez reunidas sus evaluaciones, `select_recipes`, en `experiments/marl_experiments/tuning/phase2_refined/closure.py`, calcula ese máximo antes de aplicar los filtros y lo utiliza para todos los candidatos del panel. El campo `teleports` de estas evaluaciones corresponde a `teleport_starts`, obtenido mediante TraCI.

Entre quienes cumplen los tres filtros se elige el candidato con menor espera media en las tres evaluaciones. En caso de empate exacto, se priorizan la mayor CI media, el menor tiempo de entrenamiento registrado y, por último, el identificador de la política. Si no se puede completar la auditoría o faltan evaluaciones, la combinación se informa como incompleta; si ningún candidato cumple los filtros, se informa que no hay candidato ad-

misible. El procedimiento de selección termina al identificar una configuración admisible. El entrenamiento final desde cero y su evaluación se describen en la subsección 3.6.4 de la metodología.

El criterio `final_declared_budget` del contrato de entrenamiento identifica el estado de la última iteración del presupuesto completo declarado en `training_iterations`. La comprobación del modelo restaurado exige que su número de iteración coincida con ese valor. El entrenamiento final adopta este criterio para elegir el *checkpoint*; los filtros de selección de las redes sintéticas se aplican al candidato que aporta la configuración, mientras que los diagnósticos del modelo final se interpretan en su evaluación independiente.

A.5.7. Dominios declarados y significado de los parámetros

Las Tablas A.20–A.23 presentan los doce dominios sintéticos declarados para la búsqueda refinada, uno por combinación de algoritmo, escenario sintético y arquitectura prevista. Los valores corresponden al archivo `experiments/marl_experiments/tuning/phase2_refined/informed_domains_rho0_v3.json`, localizado en la copia de trabajo `phase2-cluster` del proyecto `marl-tlc`. El diseño anterior también contemplaba Mendoza en la fase 2 de HPO, con dominios propios del escenario urbano; no integra estas doce combinaciones sintéticas. La columna «Muestreo» indica el tipo de selección sin abreviaturas: uniforme continua, loguniforme, uniforme entera o categórico, con los significados descritos al presentar el espacio inicial. En el muestreo entero, $[a; b)$ permite los enteros desde a hasta $b - 1$: el extremo superior queda excluido. En los dominios continuos, los corchetes indican los límites declarados; no garantizan que se propongan sus extremos. Las llaves enumeran las únicas alternativas del muestreo categórico.

Tabla A.20: Dominios declarados para la búsqueda `phase2_refined` de HAPPO por escenario y arquitectura.

Parámetro	Muestreo	3×1 (h256)	3×3 (h256)	$4 \times 4_{\mathrm{H}}$ (h128)
Recorte del actor	Uniforme continua	[0,1; 0,22]	[0,1; 0,22]	[0,1; 0,22]
Niveles del codificador	Uniforme entera	[4; 27)	[5; 37)	[4; 65)
Peso de entropía	Uniforme continua	[0,0043479; 0,0079967]	[0; 0,01]	[0; 0,01]
Límite del gradiente	Loguniforme	[0,64487; 4,7566]	[0,26841; 1,3289]	[0,25; 5]
Probabilidad de ordenación	Uniforme continua	[0,40484; 0,82866]	[0,44174; 0,82189]	[0,48405; 0,77601]
Coeficiente de penalización KL	Loguniforme	[0,05; 0,5]	[0,066096; 0,19175]	[0,05; 0,5]
Objetivo de divergencia KL	Loguniforme	[0,015783; 0,03]	[0,005; 0,03]	[0,005; 0,03]
Parámetro de GAE	Uniforme continua	[0,88683; 0,93862]	[0,90514; 0,9443]	[0,88; 0,99]
Tasa de aprendizaje	Loguniforme	$[2{,}707 \times 10^{-4}; 1 \times 10^{-3}]$	$[3 \times 10^{-5}; 1 \times 10^{-3}]$	$[3 \times 10^{-5}; 1 \times 10^{-3}]$
Tamaño del minilote*	Categórico	{256; 512}	{128; 512}	{256; 512}
Recorridos de optimización	Uniforme entera	[5; 19)	[5; 19)	[5; 19)
Episodios por lote	Uniforme entera	[2; 13)	[2; 13)	[2; 13)
Cota de pérdida de valor	Loguniforme	[8,402; 18,471]	[6,674; 20]	[5; 11,218]

Tabla A.21: Dominios declarados para la búsqueda `phase2_refined` de IDQN por escenario y arquitectura.

Parámetro	Muestreo	3×1 (h256)	3×3 (h128)	$4 \times 4_{\mathrm{H}}$ (h256)
Niveles del codificador	Uniforme entera	[10; 62)	[7; 33)	[4; 65)
Exploración final	Uniforme continua	[0,02; 0,047523]	[0,023555; 0,066094]	[0,02; 0,1]
Pasos para reducir exploración	Categórico	{10.000; 20.000}	{10.000; 20.000}	{10.000; 20.000}
Límite del gradiente	Categórico	{0,5; 1; 2; 5; 10}	{0,5; 1; 2}	{1; 2; 5; 10}
Tasa de aprendizaje	Loguniforme	$[9{,}879 \times 10^{-5}; 1 \times 10^{-3}]$	$[1 \times 10^{-5}; 1 \times 10^{-3}]$	$[8{,}135 \times 10^{-5}; 7{,}889 \times 10^{-4}]$
Horizonte del retorno	Categórico	{3; 5}	{1; 3}	{1; 3; 5}
Pasos antes de aprender	Categórico	{1.000; 2.000; 5.000}	{1.000; 2.000; 5.000}	{1.000; 2.000; 5.000}
Actualización de red objetivo	Categórico	{500; 1.000; 2.000; 5.000}	{500; 2.000; 5.000}	{500; 1.000; 2.000}
Transiciones por lote	Categórico	{64; 128; 256}	{256; 512}	{64; 128; 256}

Tabla A.22: Dominios declarados para la búsqueda refinada de IPPO por escenario y arquitectura.

Parámetro	Muestreo	3×1 (h256)	3×3 (h128)	$4 \times 4_{\mathrm{H}}$ (h128)
Recorte del actor	Uniforme continua	[0,1; 0,22]	[0,1; 0,22]	[0,15327; 0,19443]
Niveles del codificador	Uniforme entera	[4; 65)	[4; 65)	[4; 65)
Peso de entropía	Uniforme continua	[0,0038889; 0,0095621]	[0,00067906; 0,0074136]	[0,0036258; 0,01]
Límite del gradiente	Loguniforme	[0,25; 5]	[0,25; 5]	[0,42722; 1,5143]
Coeficiente de penalización KL	Loguniforme	[0,055016; 0,21664]	[0,05; 0,2904]	[0,05; 0,5]
Objetivo de divergencia KL	Loguniforme	[0,005; 0,03]	[0,005; 0,03]	[0,005; 0,03]
Parámetro de GAE	Uniforme continua	[0,88; 0,99]	[0,89174; 0,96008]	[0,88; 0,99]
Tasa de aprendizaje	Loguniforme	$[3 \times 10^{-5}; 1 \times 10^{-3}]$	$[1{,}136 \times 10^{-4}; 9{,}113 \times 10^{-4}]$	$[1{,}322 \times 10^{-4}; 4{,}772 \times 10^{-4}]$
Tamaño del minilote	Categórico	{128; 512}	{128; 512}	{128; 512}
Recorridos de optimización	Uniforme entera	[5; 16)	[5; 16)	[5; 16)
Episodios por lote	Uniforme entera	[2; 13)	[2; 13)	[2; 13)
Cota de pérdida de valor	Loguniforme	[5; 20]	[5; 20]	[5; 20]

Tabla A.23: Dominios declarados para la búsqueda refinada de MAPPO por escenario y arquitectura.

Parámetro	Muestreo	3×1 (h128)	3×3 (h128)	$4 \times 4_{\mathrm{H}}$ (h128)
Recorte del actor	Uniforme continua	[0,1; 0,22]	[0,1; 0,22]	[0,1; 0,22]
Niveles del codificador	Uniforme entera	[4; 65)	[8; 45)	[4; 65)
Peso de entropía	Uniforme continua	[0,00015809; 0,0062478]	[0; 0,01]	[0,0021511; 0,0080125]
Límite del gradiente	Loguniforme	[0,28422; 0,80645]	[0,25; 5]	[1,1032; 5]
Coeficiente de penalización KL	Loguniforme	[0,05; 0,13242]	[0,05; 0,5]	[0,14905; 0,38494]
Objetivo de divergencia KL	Loguniforme	[0,005; 0,03]	[0,01384; 0,028938]	[0,005; 0,01363]
Parámetro de GAE	Uniforme continua	[0,88703; 0,92246]	[0,88; 0,99]	[0,88; 0,99]
Tasa de aprendizaje	Loguniforme	$[4{,}285 \times 10^{-5}; 1{,}695 \times 10^{-4}]$	$[1{,}947 \times 10^{-4}; 1 \times 10^{-3}]$	$[3 \times 10^{-5}; 1 \times 10^{-3}]$
Tamaño del minilote	Categórico	{256; 512}	{128; 512}	{128; 256; 512}
Recorridos de optimización	Uniforme entera	[5; 16)	[6; 13)	[5; 16)
Episodios por lote	Uniforme entera	[2; 13)	[2; 13)	[2; 13)
Cota de pérdida de valor	Loguniforme	[5; 20]	[5; 20]	[5; 20]

El refinamiento sintético aplica las mismas reglas a las tres redes. El diseño anterior también las contemplaba para v14; no constituye una adaptación a O2/P2. Para cada combinación de algoritmo, escenario y arquitectura, requiere al menos diez ensayos de fase 1 que hayan completado el presupuesto sin errores, pertenezcan al dominio inicial y

tengan un objetivo numérico y finito. Los ordena de menor a mayor por ese objetivo y, en caso de empate, por identificador.

Como vector inicial, utiliza la mediana de los parámetros numéricos y la moda de los categóricos entre los cinco mejores; esa combinación puede no coincidir con un ensayo histórico. La fase 2 entrena desde cero, sin incorporar puntuaciones, archivos de estado del modelo (*checkpoint*) ni bases Optuna de fase 1. Cada red conserva sus propios dominios, derivados de sus ensayos de fase 1.

En parámetros numéricos se parte del intervalo de los diez mejores ensayos, se amplía cada extremo en un 10 % del ancho del dominio original y se limita a sus fronteras. Los parámetros loguniformes se tratan en escala logarítmica. Se conserva el dominio original si el intervalo de los veinte mejores ensayos disponibles cubre al menos el 70 % de su ancho o si alguno de los diez mejores queda a una distancia de un extremo no mayor que el 5 % de ese ancho. Para los parámetros enteros se calcula el ancho entre el menor y el mayor valor admisibles; las distancias del 5 % y del 10 % se redondean hacia arriba, con un margen mínimo de una unidad para la ampliación. El extremo superior del dominio resultante se expresa como exclusivo. En parámetros categóricos se consideran las alternativas de los veinte mejores disponibles; si solo aparece una, se añade la siguiente en el orden del dominio original o, si ocupa la última posición, la anterior.

El asterisco de `minibatch_size` en la Tabla A.20 indica que el parámetro se registra, pero HAPPO procesa el lote completo por actor. En PPO, `encoder_intervals` fija los niveles del codificador, `clip_param` limita la razón de probabilidades del actor y `entropy_coeff` pondera la entropía. `kl_coeff` pondera la penalización KL entre la política recolectora y la política actual. `kl_target` registra el objetivo asociado a la adaptación de ese coeficiente y se distingue de la ponderación que aplica `kl_coeff`. `num_epochs` indica los recorridos de optimización del lote; `minibatch_size` fija el tamaño de sus subdivisiones cuando se usan.

En las redes sintéticas, para PPO, el código multiplica `train_batch_episodes` por los pasos de control del episodio ($3600/5 = 720$) para calcular el tamaño del lote por componente de aprendizaje; ese cálculo no incluye un factor adicional por el número de procesos de simulación. En IDQN, `num_steps_sampled_before_learning_starts` fija los pasos muestreados antes de iniciar el aprendizaje; `target_network_update_freq` fija el intervalo de actualización de la red objetivo; `train_batch_size_per_learner` es el tamaño del lote de transiciones; `n_step`, el horizonte del valor objetivo temporal; `epsilon_final`, el nivel final de exploración; y `epsilon_timesteps`, el horizonte de su programación. c_v = `vf_clip_param` es la cota superior de la pérdida cuadrática de valor, no un recorte PPO convencional de predicciones.

A.5.8. Antecedente del protocolo de aprendizaje urbano

El protocolo siguiente es un antecedente ligado a la red mixta y demanda v14. Sus valores, semillas, versiones y presupuestos no se transfieren a O2/P2; esta actualización no acredita campañas de aprendizaje sobre el nuevo escenario. El protocolo `mendoza_rho0_v1` reúne las condiciones urbanas en `experiments/marl_experiments/tuning/urban/protocol.json`. El manifiesto `docs/mendoza_preparation/preparation_manifest.json` identifica los recursos congelados de v14, sus hashes y las versiones de software. Este protocolo utiliza libsumo 1.26.0; sus condiciones se distinguen de las comprobaciones históricas con el ejecutable SUMO 1.25.0.

Los archivos `nets/Mendoza/mendoza_hpo_v14.sumocfg` y `nets/Mendoza/mendoza_evaluation_v14.sumocfg` conservan la red mixta y la demanda v14. El primero comienza en $t = 1.500$ segundos, incluye 300 segundos de calentamiento y controla $[1.800, 9.000)$; el segundo comprende $[0, 43.200)$, sin calentamiento. Estas ventanas pertenecen al diseño anterior de aprendizaje. Ambos fijan `time-to-teleport`=300 segundos y `max-num-teleports`=-1. El primero regula el umbral de espera por bloqueo que puede iniciar un teletransporte; el segundo permite registrarlos sin detener la simulación al ocurrir uno [DLR26, Ecl26f]. La selección prevista compara sus inicios con la referencia mixta, según la Sección A.5.8. La inicialización y el cierre por truncamiento se describen en la subsección 3.3.1. Este protocolo activa `-save-state.rng` y conserva un estado posterior al calentamiento para cada semilla de SUMO, lo que permite restaurar también los generadores aleatorios [Ecl26b]; la comparación A/C, en cambio, no guarda los generadores aleatorios, de modo que la restauración del estado vehicular no garantiza por sí sola la repetición exacta de sus secuencias.

La calibración del codificador se registra en `docs/mendoza_preparation/encoder_calibration.json`. Utiliza los programas de referencia, las semillas 5001 y 5002, 300 segundos de calentamiento, 7200 segundos de control y una memoria de espera de 1000 segundos. Cada segundo se calcula el máximo, entre los carriles controlados, de la suma de `getAccumulatedWaitingTime` de sus vehículos. Las dos simulaciones aportan 14.400 muestras; su percentil 99 es 5396,04 veh $\cdot$ s y se redondea al entero más próximo para fijar $M_{\text{lane}} = 5396$ veh $\cdot$ s. Esta suma conserva esperas anteriores de los vehículos, mientras que el entorno atribuye la espera a cada carril; por ello, la calibración proporciona una envolvente conservadora de la señal utilizada. El protocolo anterior preveía mantener fijo ese valor durante las búsquedas y el entrenamiento final; no es una calibración de O2/P2.

El diseño anterior sitúa la fase 1 urbana después de A–E y A/C y utiliza observación local y $\rho_{\text{coop}} = 0$, conforme a la decisión explicada en la subsección 3.4.3. Comparte con los demás escenarios las dos arquitecturas, de dos capas ocultas con 128 o 256 unidades por capa, y los límites de propuestas, iteraciones, ASHA y seguimiento diagnóstico. El presupuesto previsto era de ocho estudios urbanos y 800 propuestas iniciales para los

cuatro algoritmos; no es un presupuesto ejecutado ni asignado a O2/P2.

En los métodos de la familia PPO, cada episodio controlado aporta $7200/5 = 1440$ pasos nominales; el tamaño objetivo del lote por componente de aprendizaje es esa cantidad multiplicada por `train_batch_episodes`.

La fase 2 urbana prevista utilizaba los límites de propuestas e iteraciones y las condiciones de ASHA, concurrencia y seguimiento comunes a los cuatro escenarios, definidos en la subsección A.5.6.

El diseño preveía derivar los dominios urbanos exclusivamente de los ensayos de fase 1 de Mendoza, con las reglas comunes de la subsección A.5.7. El archivo de dominios conserva los identificadores de los ensayos y los hashes de sus fuentes.

El registro `experiments/marl_experiments/tuning/urban/seeds.json` separa las semillas del aprendiz, de SUMO de entrenamiento, de SUMO diagnóstico y de Optuna para cada fase, algoritmo y arquitectura. También registra la selección, el entrenamiento final y la evaluación final. Se genera con `random.Random`, semilla maestra 20261013 y muestreo sin reemplazo de enteros desde 1.000.001 hasta $2^{31} - 1$, excluyendo las semillas históricas identificadas y las de calibración. La semilla de SUMO se pasa mediante `-seed`; el reinicio del entorno recupera el estado asociado a ella. Las semillas del aprendiz y de SUMO del entrenamiento final son nuevas respecto de las búsquedas y la selección.

La lista de candidatos se construye con `experiments/marl_experiments/tuning/urban/selection.py`. La auditoría comprueba el presupuesto completo, la ausencia de errores técnicos, el objetivo y la espera diagnóstica finitos, y la procedencia del protocolo y de las semillas. Contrasta el algoritmo, la arquitectura y los hiperparámetros con el estudio de origen y verifica la iteración del modelo restaurado.

El panel común de selección utiliza cinco semillas de SUMO: 1205571043, 1020770583, 1021545419, 927020871 y 984623645. Se aplican tanto a los candidatos como a la referencia.

El manifiesto reserva veinte semillas de evaluación final por algoritmo, sin imponer un panel común entre ellos. Así se evita reutilizar para la evaluación final las simulaciones que intervienen en la elección de configuraciones. El criterio de checkpoint se mantiene en la última iteración del presupuesto completo de la búsqueda que aporta la configuración.

Selección urbana prevista para v14

La lista urbana prevista reúne hasta tres candidatos por algoritmo que completan el presupuesto declarado y conservan métricas finitas, una configuración íntegra y un modelo restaurable de la última iteración. Se incluye el primer candidato de cada arquitectura disponible y se completa la lista por menor espera diagnóstica e identificador. La auditoría contrasta cada configuración y modelo con su ensayo de origen, como precisa la Sección A.5.8 del apéndice.

El protocolo anterior de selección compara cada candidato con los programas semafóri-

cos conservados en la red de Mendoza, tanto fijos como accionados (Sección A.4). Para esas comparaciones, especifica el escenario v14, el archivo de flujos congelado `nets/Mendoza/mendoza_12h.rou.xml` y la configuración `nets/Mendoza/mendoza_hpo_v14.sumocfg`. El control comprende $[1.800, 9.000)$ s, con 300 segundos de calentamiento desde $t = 1.500$ s.

En cada comparación se mantienen la misma semilla de SUMO, demanda, calentamiento y ventana. La referencia conserva el funcionamiento de sus programas; las políticas actúan con la inicialización descrita en la Sección 3.3.1.

El diseño adopta un panel común de cinco semillas de SUMO ($N = 5$) para todos los candidatos urbanos y la referencia. Las semillas se generan aleatoriamente, se registran antes de la selección y son distintas de las usadas en la búsqueda y en la evaluación final. Con hasta tres candidatos por algoritmo, el panel comprende hasta 65 simulaciones: cinco de referencia y hasta sesenta de candidatos. Este tamaño acota el presupuesto computacional de la selección, pero no garantiza potencia estadística ni estabilidad del ordenamiento.

Para cada semilla, las inserciones cuentan los vehículos cuyo instante efectivo de inserción pertenece a $[1.800, 9.000)$ s; la CI divide los vehículos de ese conjunto que completan el viaje antes del cierre por el total insertado. Se contabilizan también los inicios de teletransporte durante el control.

Una configuración es admisible solo si, en cada una de las cinco semillas, iguala o supera al programa de referencia en cantidad de vehículos insertados y en CI, y no registra más inicios de teletransporte. Los criterios se fijan en relación con el programa de referencia, sin umbrales absolutos de inserción o completitud.

La clasificación requiere simulaciones que alcancen el fin de la ventana, los tres registros para cada comparación y un denominador positivo para calcular la tasa y la espera; si alguno falta, no puede determinarse la admisibilidad del candidato.

Entre las configuraciones admisibles se elige la de menor media aritmética de las cinco esperas medias por simulación, calculadas sobre los vehículos insertados en el intervalo y ponderando por igual cada semilla. Ante un empate exacto, se prioriza la mayor media de las cinco CI, después el menor número medio de inicios de teletransporte y, finalmente, el identificador de la configuración. Si ninguna configuración satisface los criterios en las cinco semillas, no se designa una configuración urbana admisible para esa combinación.

A.5.9. Métricas y análisis por pares de A/C

En la implementación, el campo `mean_acc_waiting_time` registra en cada paso de decisión la espera acumulada en los carriles del agente dividida por la cantidad de vehículos presentes. Cuando no hay vehículos, el valor del paso es cero. Para calcular el Gini, el código promedia estos valores a lo largo del control para cada agente y da igual peso a todos los agentes.

Para cada una de las tres redes, las cinco evaluaciones de una política se promedian

antes de calcular las diferencias entre las 24 parejas de entrenamiento. El remuestreo conserva cada pareja completa y genera 20.000 conjuntos de 24 parejas con reemplazo. Para el intervalo percentil del 95 % se usan los percentiles 2,5 y 97,5; para el intervalo del 98,33 %, aproximadamente 0,83 y 99,17. Este nivel aplica la corrección de Bonferroni al nivel nominal de 0,05 dividido entre los tres contrastes A/C, uno por cada red (0,05/3), para cada medida por separado. No se ajusta conjuntamente entre métricas distintas.

Como análisis de sensibilidad, se calcula un intervalo basado en t de Student con 24 diferencias pareadas, bajo el supuesto de diferencias independientes y aproximadamente normales. Los intervalos percentiles y el intervalo t son aproximaciones; su cobertura no está garantizada, y los valores extremos se conservan para revisar su influencia.

El protocolo original de la campaña, identificado en `docs/PROTOCOLO_COOPERACION_MULTIESCENARIO.md`, incluye las tres redes. El informe histórico, en `cooperation_multiscenario_20260920/report/INFORME.md`, presenta como análisis principal 48 parejas de 3×1 y $4 \times 4_H$. El mismo informe separa los datos de 3×3 después de revisar los resultados porque su entrenamiento tuvo una dinámica distinta; señala que esa decisión de presentación no estaba fijada de antemano y que los registros de esa red se conservaron aparte. El criterio histórico de presentación se distingue del alcance metodológico adoptado aquí, que analiza por separado las tres redes; por tanto, no implica una exclusión previa de 3×3.

A.6. Detalles del aprendizaje y de la codificación

En los módulos de aprendizaje propios de MAPPO y HAPPO, $V_{b,i}$ es la predicción del crítico para el agente i en la muestra b y $y_{b,i}$ es su valor objetivo. Se denotan por N el número de agentes, por B la cantidad de muestras válidas del lote y por $c_v =$ `vf_clip_param` la cota superior configurada. Primero se calcula el error cuadrático promedio entre agentes de cada muestra, ℓ_b; después se limita al intervalo $[0, c_v]$ y se promedian las pérdidas de las muestras válidas para obtener la pérdida final del crítico, $\mathcal{L}_V$:

$$\ell_b = \frac{1}{N}\sum_{i=1}^{N}(V_{b,i} - y_{b,i})^2, \qquad \mathcal{L}_V = \frac{1}{B}\sum_{b=1}^{B}\text{mín}\{c_v, \text{máx}(0, \ell_b)\}. \tag{A.6}$$

Las muestras válidas son las que la máscara del lote no excluye del cálculo. Este recorte del error cuadrático no es el *value clipping* convencional de PPO, que compara predicciones de valor recortadas y no recortadas. Como el error se promedia entre agentes antes de aplicar la cota, una muestra saturada deja de contribuir al ajuste de todas las salidas del crítico, lo que puede afectar las estimaciones de valor y las ventajas que guían a los actores; la frecuencia de saturación no se cuantifica en los registros descritos en este trabajo.

IDQN reutiliza transiciones de un búfer de experiencia priorizado y aprende valores para las acciones; no tiene un actor PPO, y la selección se realiza entre acciones permiti-

das. Los métodos tipo PPO recolectan un lote de trayectorias. En IPPO y MAPPO, un *minibatch* es una subdivisión del lote que produce una actualización; HAPPO procesa el lote completo por actor. Los críticos y optimizadores son componentes de entrenamiento, mientras que en PPO la ejecución de cada actor depende de su observación y de la máscara vigente.

En MAPPO, los objetivos de valor y las ventajas se calculan por separado con las recompensas individuales y las estimaciones del crítico. Las ventajas se estandarizan por agente: se resta su media y se divide por su desviación estándar.

En el aprendiz de HAPPO de la versión congelada inspeccionada, el crítico se actualiza al comienzo de cada *epoch* con el lote completo. Se aplica un paso de gradiente por actor y época. Las ventajas individuales estandarizadas ya calculadas inicializan los factores de reponderación M; estos se reinician en cada *epoch*, y las ventajas no se recalculan durante la secuencia de actores. Antes de actualizar actores, el código construye un orden entero de agentes. En cada turno, `greedy_selection_prob` determina la probabilidad de elegir, entre los agentes pendientes, el que tiene la mayor media del valor absoluto de su factor inicial; de lo contrario, elige uno al azar. Esta regla ordena a los actores para su actualización, no selecciona la fase semafórica. El orden no se vuelve a sortear después de reponderar factores.

Tras actualizar un actor, el código calcula para la acción de la muestra la razón entre su probabilidad bajo el actor actualizado y su probabilidad bajo la política recolectora. Esa razón multiplica los factores de los agentes que aún esperan su turno. La reponderación no ejecuta una actualización de gradiente adicional para esos actores. Estas reglas describen la adaptación de HAPPO utilizada en la tesina.

Zhong et al. encuadran HAPPO como una instancia de HAML y demuestran mejora monótona del retorno conjunto y convergencia bajo las condiciones del marco, que incluyen una recompensa compartida en un juego cooperativo. Las ventajas por agente no constituyen por sí solas una diferencia con ese planteo: HAPPO utiliza una descomposición por agentes de la ventaja conjunta. En esta adaptación, los objetivos de valor y las ventajas se calculan a partir de recompensas específicas por agente, en un juego parcialmente observable. Al faltar el retorno común, las garantías teóricas no se aplican directamente a la formulación experimental [ZKF+24].

A.6.1. Desarrollo algebraico del codificador

Para una espera acumulada por carril $x \geq 0$ (en veh·s) y una cota superior de escala $M_{\text{lane}} > 0$, la entrada normalizada se define como $\hat{x} = \min(1, x/M_{\text{lane}}) \in [0, 1]$. Para $I \geq 2$ niveles discretos, sea $L = (I-1)/M_{\text{lane}}$ el factor de escala (con $0 < L < 1$). La función de mezcla entre las componentes logarítmica y lineal queda dada por:

$$g(\hat{x}) = (1 - L)\frac{\log_2\left(\frac{\hat{x}}{L} + 1\right)}{\log_2\left(\frac{1}{L} + 1\right)} + L\hat{x}, \tag{A.7}$$

y el nivel entero codificado $e(x) \in \{0, 1, \ldots, I-1\}$ se obtiene escalando por $I-1$, redondeando hacia arriba y acotando al nivel máximo:

$$e(x) = \text{mín}\{I-1, \text{ceil}((I-1)g(\hat{x}))\}. \tag{A.8}$$

Para la observación vecinal, se aplica la misma función sobre la espera media por carril W_j/m_j, empleando la cota de escala vecinal M_{vec} (en veh·s por carril).

Definiendo el coeficiente logarítmico ponderado $F = (1-L)(I-1)/\log_2(1/L+1)$, para cualquier valor en el rango $0 \le x < M_{\text{lane}}$ la transformación implementada en código se computa equivalentemente como:

$$e(x) = \text{ceil}\left(F \cdot \log_2\left(\frac{x}{M_{\text{lane}} \cdot L} + 1\right) + L^2 \cdot x\right), \tag{A.9}$$

mientras que para $x \ge M_{\text{lane}}$ se asigna directamente el nivel máximo $I-1$.

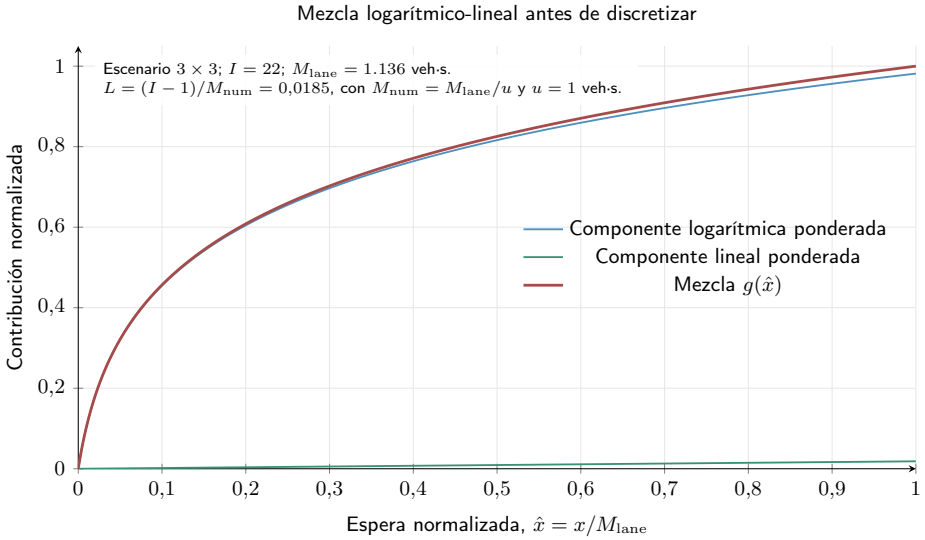


Figura A.3: Ejemplo ilustrativo con $I = 22$ y límite de escala $M_{\text{lane}} = 1.136$ veh·s calibrado para el escenario 3×3; $I = 22$ no corresponde a la configuración de la comparación A/C.

A.6.2. Aplicación de la máscara de acciones

En cada decisión, $\mathcal{A}_i^{\text{adm}}(t)$ es el subconjunto de acciones permitidas por la señal vigente. La máscara binaria $b_i(a,t)$ vale uno si a pertenece a ese conjunto y cero en caso contrario. Si $z_i(a,t)$ es la puntuación no normalizada (*logit*) asignada por el actor, la puntuación enmascarada $\widetilde{z}_i(a,t)$ se obtiene sumando el logaritmo natural de la máscara:

$$\widetilde{z}_i(a,t) = z_i(a,t) + \log b_i(a,t), \qquad b_i(a,t) = \begin{cases} 1 & \text{si } a \in \mathcal{A}_i^{\text{adm}}(t) \\ 0 & \text{si } a \notin \mathcal{A}_i^{\text{adm}}(t) \end{cases} \tag{A.10}$$

Con $\log(0) = -\infty$, las acciones no permitidas reciben probabilidad cero después de normalizar la política. La implementación sustituye el infinito por una puntuación finita muy baja y requiere al menos una acción admisible. Los algoritmos PPO aplican la máscara a las puntuaciones de política; IDQN filtra las acciones al seleccionarlas y evaluarlas.

La Tabla 3.4, en la metodología, distingue las señales por procedimiento. En el programa de fase 1 consultado, la normalización por carriles está desactivada por defecto; esa configuración del programa no verifica cada ensayo histórico. La comparación A/C fija la señal local en A y activa la contribución vecinal normalizada en C, según la ecuación (3.4).

A.6.3. Configuración fija de la comparación multiescenario

La Tabla A.24 conserva los hiperparámetros de MAPPO h128 seleccionados en un ensayo histórico de $4 \times 4_H$ y aplicados sin cambios entre las tres redes de tráfico. El ensayo `49cfbba0` está registrado en `rho_screening_recipes/mappo_4x4_H_h128.json` e incorporado a los manifiestos de campaña. Los valores continuos se muestran redondeados; la fuente conserva precisión completa.

Se entrenaron 24 parejas por red. La planificación usó una desviación de referencia cercana a dos segundos por vehículo y un intervalo del 95 % basado en t de Student, bajo supuestos de diferencias independientes y aproximadamente normales. Con 24 parejas, se esperaba una semiamplitud cercana a 0,85 segundos por vehículo.

Tabla A.24: Hiperparámetros fijos de aprendizaje de la comparación multiescenario.

Parámetro	Valor
Arquitectura	dos capas de 128 unidades
Tasa de aprendizaje (`lr`)	$4,95235885 \times 10^{-5}$
Recorte del actor (`clip_param`)	0,20148266
GAE (`lambda_gae`)	0,90059980
Coeficiente de entropía	0,00483113
Recorte del gradiente	1,48846291
Coeficiente KL	0,24180060
Objetivo KL	0,01053851
Cota de pérdida de valor	10,11299284
Recorridos de optimización	8
Episodios por lote	3
Tamaño del minilote	128
Niveles del codificador (I)	17
Descuento (γ)	0,99

La comparación fija el peso de la recompensa vecinal en cero para la variante local y en 0,1 para la vecinal; también asigna observación local o vecinal y normaliza por

carriles, según la condición. Los límites de escala del codificador corresponden a cada escenario. Los tres episodios de cada lote representan $3 \times (3600/5) = 2160$ pasos de entorno por componente de aprendizaje. El número de procesos de simulación no multiplica este tamaño de lote.

Los manifiestos de la comparación registran SUMO 1.26.0, Ray 2.54.1 y PyTorch 2.10.0 con CUDA 12.8. Estas versiones corresponden a esa campaña y no se atribuyen automáticamente a las búsquedas históricas. Los archivos `manifest.json` de cada par conservan su configuración y versiones efectivas.

La calibración vecinal específica se conserva junto con la configuración de la comparación; otros archivos de calibración pueden corresponder a diseños distintos.

A.6.4. Calibraciones y semillas de cooperación

El límite de escala local se calculó con las semillas 42–44; para 3×1, 3×3 y $4 \times 4_H$ se obtuvieron 1184, 1136 y 712, respectivamente. La calibración de la señal de espera vecinal incorporada en la observación C utilizó las semillas 51001–51003 y produjo límites de escala de 9660, 5329 y 4652 en esas redes. Los límites locales se expresan en veh·s y los vecinales en veh·s por carril. En ambos casos se empleó el percentil 99 y 300 segundos de calentamiento. Para la señal vecinal, el percentil se calculó sobre la medida media agrupada por intersección y paso. Estos valores corresponden a escalas de señales codificadas, no a tiempos máximos de viaje ni a valores de recompensa.

La comparación en las tres redes utiliza las semillas 61001–61024 para el entrenamiento y 71001–71005 para la evaluación. Las semillas de evaluación son comunes a las dos variantes y se mantienen separadas de las de entrenamiento.

A.6.5. Preparación de las figuras de escenarios

Las redes sintéticas se representan con TikZ a partir de los XML de SUMO mediante `metodologia_generar_redes.py`. El generador conserva los hashes de las fuentes, verifica los conteos de agentes, tramos y carriles y representa los sentidos de circulación. Las conexiones entre centros de nodos se representan mediante calzadas divididas en carriles, con anchos convencionales y escala propia por panel. El mapa urbano conserva la captura de SUMO y destaca el tramo de la TAZ 28. El encuadre conserva la escala de distancia. La Figura 3.3 muestra el dominio urbano modelado; el punto de conteo de la cámara 169 queda fuera de ese dominio y no aparece en la captura.

A.6.6. Vistas documentales de los escenarios

La Figura A.4 permite consultar la distribución temporal de los programas sintéticos de referencia. Los intervalos V_i, A_i y R corresponden a verde, amarillo y todo rojo, respectivamente. La Figura A.5 conserva la vista completa de las zonas urbanas: el celeste identifica sus tramos y los recuadros rojos facilitan localizarlos; no son límites territoriales.

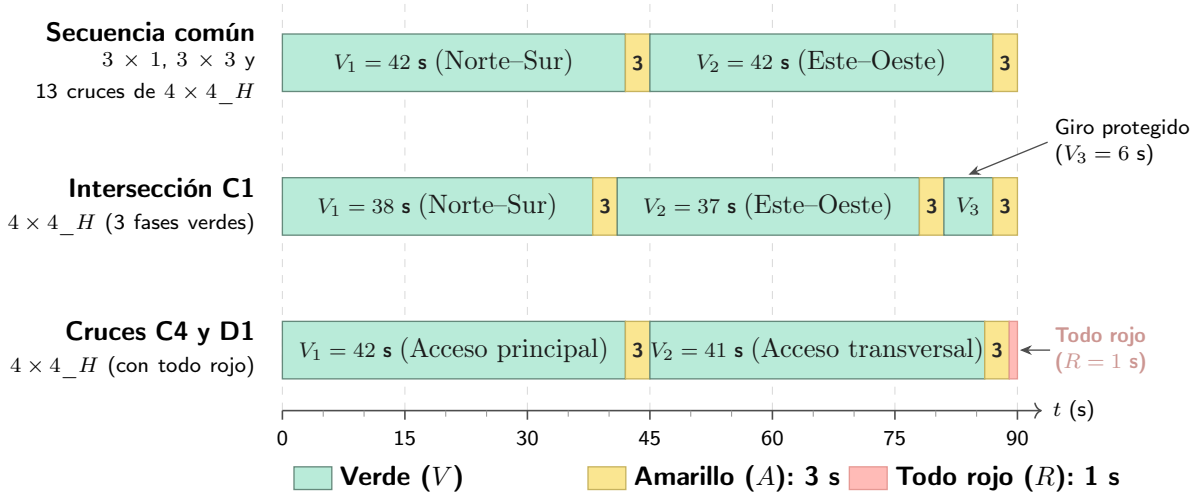


Figura A.4: Distribución de los intervalos de verde, amarillo y todo rojo en los ciclos de referencia de 90 segundos.

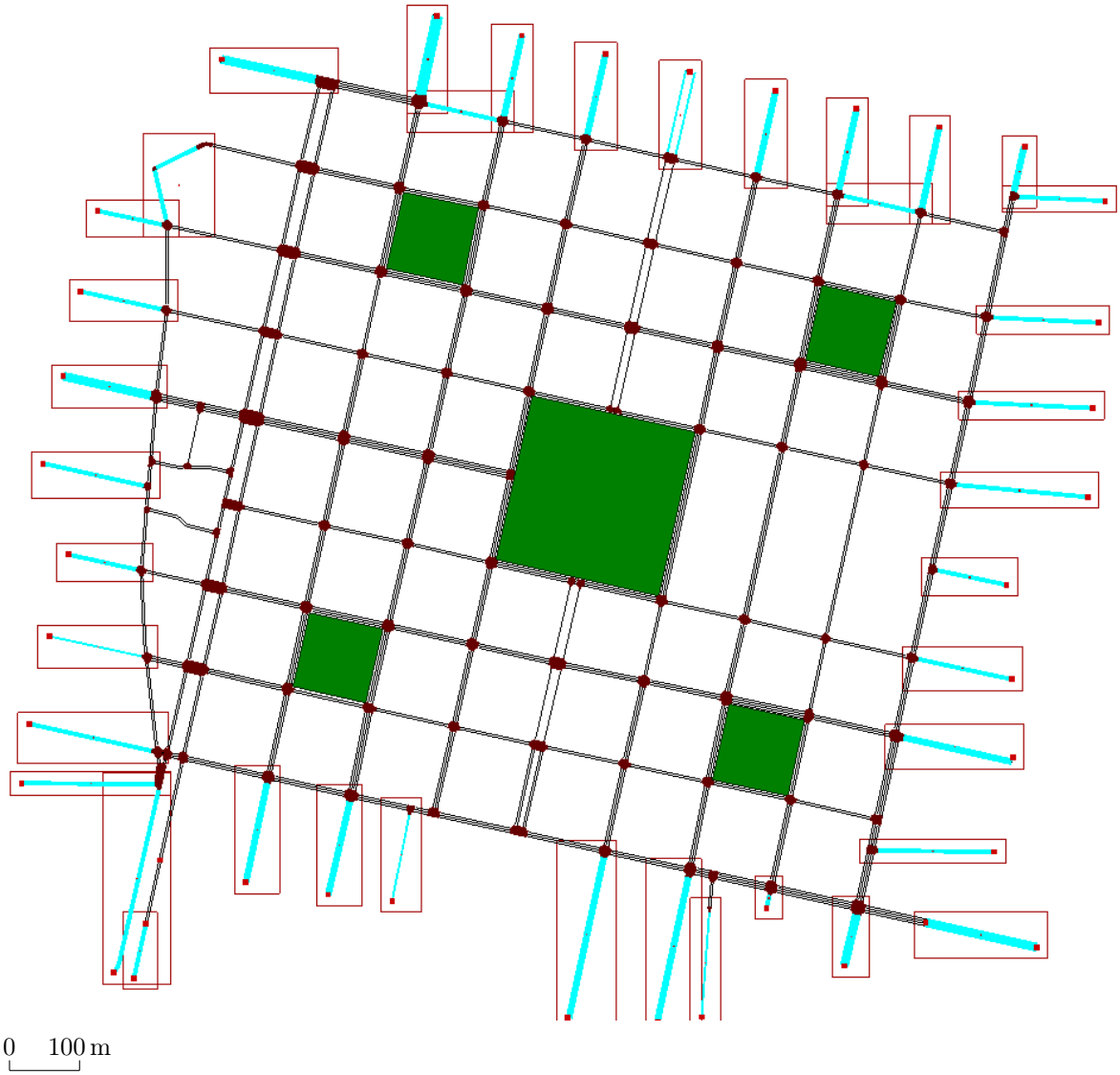


Figura A.5: Zonas de análisis de tránsito y sus tramos asociados en la red de Mendoza.

Bibliografía

[ACS24] Albrecht, Stefano V., Filippos Christianos y Lukas Schäfer: *Multi-Agent Reinforcement Learning: Foundations and Modern Approaches*. MIT Press, 2024. `https://www.marl-book.com/`, Consultado en 2025.

[AHL16] Abel, David, D. Ellis Hershkowitz y Michael L. Littman: *Near Optimal Behavior via Approximate State Abstraction*. En *Proceedings of the 33rd International Conference on Machine Learning*, volumen 48 de *Proceedings of Machine Learning Research*, páginas 2915–2923. PMLR, 2016.

[ALBBW+18] Alvarez Lopez, Pablo, Michael Behrisch, Laura Bieker-Walz, Jakob Erdmann, Yun Pang Flötteröd, Robert Hilbrich, Leonhard Lücken, Johannes Rummel, Peter Wagner y Evamarie Wießner: *Microscopic Traffic Simulation using SUMO*. En *2018 21st International Conference on Intelligent Transportation Systems (ITSC)*, páginas 2575–2582. IEEE, 2018.

[AOS20] Amato, Christopher, Frans A. Oliehoek y Matthijs T.J. Spaan: *A Survey of Decentralized Partially Observable Markov Decision Processes and Applications*. Journal of Artificial Intelligence Research, 69:1–72, 2020.

[AS22] Ault, James y Guni Sharon: *Cooperative Multi-agent Reinforcement Learning Applied to Multi-intersection Traffic Signal Control*. En *Proceedings of the Third Workshop on Data-driven Intelligent Transportation, co-located with CIKM '22*, 2022. `https://people.engr.tamu.edu/guni/papers/CIKM22-MATCS.pdf`, Fuente: [7, 8] Destaca el "éxito limitado"de SARL y los enfoques independientes en redes.

[ASC+21] Agarwal, Rishabh, Max Schwarzer, Pablo Samuel Castro, Aaron C. Courville y Marc G. Bellemare: *Deep Reinforcement Learning at the Edge of the Statistical Precipice*. En *Advances in Neural Information Processing Systems 34*, páginas 29304–29320, 2021. `https://proceedings.neurips.cc/paper_files/paper/2021/hash/f514cec81cb148559cf475e7426eed5e-Abstract.html`.

[Aso18] Asociación de Fábricas Argentinas de Componentes: *Flota circulante en Argentina 2017*. Informe técnico, Asociación de Fábricas Argentinas de

Componentes, mayo 2018. https://cdn.motor1.com/pdf-files/afac-informe-parque-circulante-2017pdf.pdf, Informe elaborado con datos de AFAC y Promotive.

[Aso26] Asociación de Fábricas Argentinas de Componentes: *Flota vehicular circulante en Argentina*. Informe técnico, Asociación de Fábricas Argentinas de Componentes, 2026. https://www.automotrix.com.ar/adm/enviosmasivos/Plantillas/PRENSA/2026/AFAC_PRENSA_FLOTACIRCULANTE_2025.pdf, Datos al cierre de 2025; informe elaborado en el marco del convenio entre AFAC y Promotive S.A.

[ASY+19] Akiba, Takuya, Shotaro Sano, Toshihiko Yanase, Takeru Ohta y Masanori Koyama: *Optuna: A next-generation hyperparameter optimization framework*. En *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, páginas 2623–2631, 2019.

[BB24] Bishop, Christopher M. y Hugh Bishop: *Deep Learning: Foundations and Concepts*. Springer, Cham, 2024.

[BBBK11] Bergstra, James S., Rémi Bardenet, Yoshua Bengio y Balázs Kégl: *Algorithms for Hyper-Parameter Optimization*. En *Advances in Neural Information Processing Systems 24*, 2011. https://proceedings.neurips.cc/paper/2011/hash/86e8f7ab32cfd12577bc2619bc635690-Abstract.html.

[BV02] Bowling, Michael y Manuela Veloso: *Multiagent learning using a variable learning rate*. Artificial Intelligence, 136(2):215–250, 2002.

[CWCL19] Chu, Tianshu, Jie Wang, Lara Codecà y Zhaojian Li: *Multi-agent deep reinforcement learning for large-scale traffic signal control*. IEEE Transactions on Intelligent Transportation Systems, 21(3):1086–1095, 2019.

[CYL+24] Chen, Wubing, Shangdong Yang, Wenbin Li, Yujing Hu, Xiao Liu y Yang Gao: *Learning Multi-Intersection Traffic Signal Control via Coevolutionary Multi-Agent Reinforcement Learning*. IEEE Transactions on Intelligent Transportation Systems, 25(11):15947–15963, 2024.

[CyMR18] Mayor R., Rafael Cal y: *Ingeniería de tránsito: Fundamentos y aplicaciones*. Alfaomega, 9na edición, 2018.

[DLR26] DLR – Institute of Transportation Systems: *SUMO User Documentation*, 2026. https://sumo.dlr.de/docs/, Documentación web consultada en 2026.

[DLSZ26] Deng, Maojiang, Shoufeng Lu, Jiazhao Shi y Wen Zhang: *Adaptive Traffic Signal Control Optimization Using a Novel Road Partition and Multi-Channel State Representation Method*. Urban Lifeline, 4(1):9, 2026.

[dWGM+20] Witt, Christian Schroeder de, Tarun Gupta, Denys Makoviichuk, Viktor Maksimov, Edward Ivanov, Michael Schmid, Gregory Yakovlev, Philip Torr *y cols.*: *Is independent learning all you need in the starcraft multi-agent challenge?* arXiv preprint arXiv:2011.09533, 2020.

[Ecl26a] Eclipse SUMO: *SUMO Documentation: Randomness*. `https://sumo.dlr.de/docs/Simulation/Randomness.html`, 2026. Documentación en línea, consultada el 8 de octubre de 2026.

[Ecl26b] Eclipse SUMO: *SUMO Documentation: Save and Load*. `https://sumo.dlr.de/docs/Simulation/SaveAndLoad.html`, 2026. Documentación en línea, consultada el 13 de octubre de 2026.

[Ecl26c] Eclipse SUMO: *SUMO Documentation: Summary*. `https://sumo.dlr.de/docs/Simulation/Output/Summary.html`, 2026. Documentación en línea, consultada el 15 de octubre de 2026.

[Ecl26d] Eclipse SUMO: *SUMO Documentation: TraCI: Vehicle Value Retrieval*. `https://sumo.dlr.de/docs/TraCI/Vehicle_Value_Retrieval.html`, 2026. Documentación en línea, consultada el 8 de octubre de 2026.

[Ecl26e] Eclipse SUMO: *SUMO Documentation: TripInfo*. `https://sumo.dlr.de/docs/Simulation/Output/TripInfo.html`, 2026. Documentación en línea, consultada el 8 de octubre de 2026.

[Ecl26f] Eclipse SUMO: *SUMO Documentation: Why Vehicles Are Teleporting*. `https://sumo.dlr.de/docs/Simulation/Why_Vehicles_are_teleporting.html`, 2026. Documentación en línea, consultada el 13 de octubre de 2026.

[ELR23] Eimer, Theresa, Marius Lindauer y Roberta Raileanu: *Hyperparameters in Reinforcement Learning and How To Tune Them*. En *Proceedings of the 40th International Conference on Machine Learning*, volumen 202 de *Proceedings of Machine Learning Research*, páginas 9104–9149. PMLR, 2023. `https://proceedings.mlr.press/v202/eimer23a.html`.

[FA11] Fernández A., Rodrigo: *Elementos de la teoría del tráfico vehicular*. Fondo Editorial de la Pontificia Universidad Católica del Perú, 2011.

[Fed08] Federal Highway Administration: *Traffic Signal Timing Manual*. Informe técnico FHWA-HOP-08-024, Federal Highway Administration, Office of Operations, 2008. Primera edición. Capítulos 4: Traffic Signal Design y 6: Coordination. Disponible en: `https://ops.fhwa.dot.gov/publications/fhwahop08024/`.

[FNF+17] Foerster, Jakob, Nantas Nardelli, Gregory Farquhar, Philip H.S. Torr, Pushmeet Kohli y Shimon Whiteson: *Stabilising Experience Replay for Deep Multi-Agent Reinforcement Learning*. En *Proceedings of the 34th International Conference on Machine Learning (ICML)*, páginas 1146–1155. PMLR, 2017.

[FYX+23] Fang, Jie, Ya You, Mengyun Xu, Juanmeizi Wang y Sibin Cai: *Multi-Objective Traffic Signal Control Using Network-Wide Agent Coordinated Reinforcement Learning*. Expert Systems with Applications, 229:120535, 2023.

[GBC16] Goodfellow, Ian, Yoshua Bengio y Aaron Courville: *Deep Learning*. MIT Press, Cambridge, MA, 2016. `https://www.deeplearningbook.org/`.

[GR18] Genders, Wade y Saiedeh Razavi: *Evaluating Reinforcement Learning State Representations for Adaptive Traffic Signal Control*. Procedia Computer Science, 130:26–33, 2018.

[HHA22] Haddad, Tarek Amine, Djalal Hedjazi y Sofiane Aouag: *A Deep Reinforcement Learning-Based Cooperative Approach for Multi-Intersection Traffic Signal Control*. Engineering Applications of Artificial Intelligence, 114:105019, 2022.

[JLW+24] Jiang, Haoyuan, Ziyue Li, Hua Wei, Xuantang Xiong, Jingqing Ruan, Jiaming Lu, Hangyu Mao y Rui Zhao: *X-Light: Cross-City Traffic Signal Control Using Transformer on Transformer as Meta Multi-Agent Reinforcement Learner*. En *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence (IJCAI)*, páginas 94–102, 2024.

[KCW+22] Kuba, Jakub Grudzien, Ruiqing Chen, Muning Wen, Ying Wen, Fanglei Sun, Jun Wang y Yaodong Yang: *Trust region policy optimisation in multi-agent reinforcement learning*. En *International Conference on Learning Representations (ICLR)*, 2022. `https://arxiv.org/abs/2109.11251`.

[KKBA23] Kolat, Máté, Bálint Kővári, Tamás Bécsi y Szilárd Aradi: *Multi-Agent Reinforcement Learning for Traffic Signal Control: A Cooperative Approach*. Sustainability, 15(4):3479, 2023.

[KL02] Kakade, Sham y John Langford: *Approximately optimal reinforcement learning.* En *International Conference on Machine Learning (ICML)*, páginas 267–274, 2002.

[Kra98] Krauß, Stefan: *Microscopic Modeling of Traffic Flow: Investigation of Collision Free Vehicle Dynamics.* DLR-Forschungsbericht 98-08, Deutsches Zentrum für Luft- und Raumfahrt, 1998. `https://elib.dlr.de/8380/`.

[KT03] Konda, Vijay R. y John N. Tsitsiklis: *On Actor-Critic Algorithms.* SIAM Journal on Control and Optimization, 42(4):1143–1166, 2003.

[Law15] Law, Averill M.: *Statistical Analysis of Simulation Output Data: The Practical State of the Art.* En *Proceedings of the 2015 Winter Simulation Conference*, páginas 1810–1819, 2015. `https://informs-sim.org/wsc15papers/188.pdf`.

[LFYZ25] Lu, Qiong, Haoda Fang, Zhangcheng Yin y Guliang Zhu: *HAPS-PPO: A Multi-Agent Reinforcement Learning Architecture for Coordinated Regional Control of Traffic Signals in Heterogeneous Road Networks.* Applied Sciences, 15(20):10945, 2025.

[LJR+20] Li, Liam, Kevin Jamieson, Afshin Rostamizadeh, Ekaterina Gonina, Jonathan Ben-Tzur, Moritz Hardt, Benjamin Recht y Ameet Talwalkar: *A system for massively parallel hyperparameter tuning.* En *Proceedings of Machine Learning and Systems (MLSys)*, volumen 2, páginas 230–246, 2020.

[LLN+18a] Liang, Eric, Richard Liaw, Robert Nishihara, Philipp Moritz, Roy Fox, Ken Goldberg, Joseph Gonzalez, Michael Jordan y Ion Stoica: *RLlib: Abstractions for distributed reinforcement learning.* En *International Conference on Machine Learning (ICML)*, páginas 3053–3062. PMLR, 2018.

[LLN+18b] Liaw, Richard, Eric Liang, Robert Nishihara, Philipp Moritz, Joseph E Gonzalez y Ion Stoica: *Tune: A research platform for distributed model selection and training.* arXiv preprint arXiv:1807.05118, 2018.

[LML+25] Liang, Jiaxin, Haotian Miao, Kai Li, Jianheng Tan, Xi Wang, Rui Luo y Yueqiu Jiang: *A Review of Multi-Agent Reinforcement Learning Algorithms.* Electronics, 14(4):820, 2025. `https://doi.org/10.3390/electronics14040820`.

[LWT+17] Lowe, Ryan, Yi Wu, Aviv Tamar, Jean Harb, Pieter Abbeel y Igor Mordatch: *Multi-Agent Actor-Critic for Mixed Cooperative-Competitive Environments.* En *Advances in Neural Information Processing Systems (NeurIPS)*, páginas 6379–6390, 2017.

[LZF+25] Liu, Wanting, Chengwei Zhang, Wanqing Fang, Kailing Zhou, Yihong Li, Furui Zhan, Qi Wang, Wanli Xue y Rong Chen: *Vehicle-Level Fairness-Oriented Constrained Multi-Agent Reinforcement Learning for Adaptive Traffic Signal Control*. IEEE Transactions on Intelligent Transportation Systems, 26(4):4878–4890, 2025.

[MKS+15] Mnih, V., K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg y D. Hassabis: *Human-level control through deep reinforcement learning*. Nature, 518(7540):529–533, 2015. https://doi.org/10.1038/nature14236.

[MMLK25] Michailidis, Panagiotis, Iakovos Michailidis, Charalampos Rafail Lazaridis y Elias Kosmatopoulos: *Traffic Signal Control via Reinforcement Learning: A Review on Applications and Innovations*. Infrastructures, 10(5):114, 2025. Fuente: [4, 5] Revisión que documenta la evolución de RL como framework para TSC.

[PGM+19] Paszke, Adam, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai y Soumith Chintala: *PyTorch: An Imperative Style, High-Performance Deep Learning Library*. En *Advances in Neural Information Processing Systems 32*, páginas 8026–8037, 2019. NeurIPS 2019. https://proceedings.neurips.cc/paper/2019/hash/bdbca288fee7f92f2bfa9f7012727740-Abstract.html.

[PW16] Pande, Anurag y Brian Wolshon: *Traffic Engineering Handbook*. John Wiley & Sons, Inc., 7th edición, 2016.

[RDZ+24] Ren, Fuyue, Wei Dong, Xiaodong Zhao, Fan Zhang, Yaguang Kong y Qiang Yang: *Two-Layer Coordinated Reinforcement Learning for Traffic Signal Control in Traffic Network*. Expert Systems with Applications, 235:121111, 2024.

[RSDW+18] Rashid, Tabish, Mikayel Samvelyan, Christian Schroeder De Witt, Gregory Farquhar, Jakob Foerster y Shimon Whiteson: *QMIX: Monotonic Value Function Factorisation for Deep Multi-Agent Reinforcement Learning*. En *Proceedings of the 35th International Conference on Machine Learning (ICML)*, páginas 4295–4304. PMLR, 2018.

[Sar13] Sargent, Robert G.: *Verification and validation of simulation models.* Journal of Simulation, 7(1):12–24, 2013.

[SB18] Sutton, R.S. y A.G. Barto: *Reinforcement learning: An introduction.* The MIT Press, Cambridge, MA, 2nd edición, 2018.

[Sha53] Shapley, Lloyd S: *Stochastic games.* Proceedings of the National Academy of Sciences, 39(10):1095–1100, 1953.

[SLA+15] Schulman, John, Sergey Levine, Pieter Abbeel, Michael Jordan y Philipp Moritz: *Trust region policy optimization.* En *International Conference on Machine Learning (ICML)*, páginas 1889–1897. PMLR, 2015.

[SLG+18] Sunehag, Peter, Guy Lever, Audrunas Gruslys, Wojciech M. Czarnecki, Vinicius Zambaldi, Max Jaderberg, Marc Lanctot, Nicolas Sonnerat, Joel Z. Leibo, Karl Tuyls y Thore Graepel: *Value-Decomposition Networks For Cooperative Multi-Agent Learning.* En *Proceedings of the 17th International Conference on Autonomous Agents and MultiAgent Systems (AAMAS)*, páginas 2085–2087. International Foundation for Autonomous Agents and Multiagent Systems, 2018.

[SML+16] Schulman, John, Philipp Moritz, Sergey Levine, Michael Jordan y Pieter Abbeel: *High-dimensional continuous control using generalized advantage estimation.* En *International Conference on Learning Representations (ICLR)*, 2016.

[SWD+17] Schulman, John, Filip Wolski, Prafulla Dhariwal, Alec Radford y Oleg Klimov: *Proximal policy optimization algorithms.* arXiv preprint arXiv:1707.06347, 2017.

[TMK+17] Tampuu, Ardi, Tambet Matiisen, Dorian Kodelja, Ilya Zauss, Kristjan Korjus, Juhan Aru, Jaan Birk y Raul Vicente: *Multiagent cooperation and competition with deep reinforcement learning.* PloS one, 12(4):e0172395, 2017.

[Tok24] Toko: *Manual de uso del clúster Toko.* `https://toko.uncu.edu.ar/files/Manual_Toko-20240401.pdf`, 2024.

[vZW80] Zuylen, Henk J. van y Luis G. Willumsen: *The most likely trip matrix estimated from traffic counts.* Transportation Research Part B: Methodological, 14(3):281–293, 1980.

[W+19] Wei, H. *y cols.*: *A survey on traffic signal control methods*, 2019. Available at: `https://arxiv.org/abs/1904.08117`.

[WCZ+19] Wei, Hua, Chacha Chen, Guanjie Zheng, Kan Wu, Vikash Gayah, Kai Xu y Zhenhui Li: *PressLight: Learning Max Pressure Control to Coordinate Traffic Signals in Arterial Network*. En *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, páginas 1290–1298, 2019.

[Wie00] Wiering, Marco A: *Multi-agent reinforcement learning for traffic light control*. Machine learning, 2000.

[WVW19] Wunderlich, Karl, Meenakshy Vasudevan y Peiwei Wang: *Traffic Analysis Toolbox Volume III: Guidelines for Applying Traffic Microsimulation Modeling Software. 2019 Update to the 2004 Version*. Informe técnico FHWA-HOP-18-036, Federal Highway Administration, Office of Operations, Abril 2019. `https://ops.fhwa.dot.gov/publications/fhwahop18036/index.htm`.

[WWS+22] Wu, Qiang, Jianqing Wu, Jun Shen, Bo Du, Akbar Telikani, Mahdi Fahmideh y Chao Liang: *Distributed Agent-Based Deep Reinforcement Learning for Large Scale Traffic Signal Control*. Knowledge-Based Systems, 241:108304, 2022.

[WXZ+19] Wei, Hua, Nan Xu, Huichu Zhang, Guanjie Zheng, Xinshi Zang, Chacha Chen, Weinan Zhang, Yanmin Zhu, Kai Xu y Zhenhui Li: *CoLight: Learning Network-level Cooperation for Traffic Signal Control*. En *Proceedings of the 28th ACM International Conference on Information and Knowledge Management (CIKM)*, páginas 1913–1922, 2019.

[WZK+23] Wang, Xihuai, Lingpan Zheng, Jakub Grudzien Kuba, Ruiqing Chen, Li Shen, Yidong Cao, Jun Wang y Yaodong Yang: *Order matters: Agent-by-agent policy optimization*. En *International Conference on Learning Representations (ICLR)*, 2023.

[Yan23] Yang, Shantian: *Hierarchical Graph Multi-Agent Reinforcement Learning for Traffic Signal Control*. Information Sciences, 634:55–72, 2023.

[YVV+22] Yu, Chao, Akash Velu, Eugene Vinitsky, Jiaxuan Gao, Yu Wang, Alexandre Bayen y Yi Wu: *The surprising effectiveness of ppo in cooperative multi-agent games*. Advances in Neural Information Processing Systems, 35:24611–24624, 2022.

[ZKF+24] Zhong, Yifan, Jakub Grudzien Kuba, Xidong Feng, Siyi Hu, Jiaming Ji y Yaodong Yang: *Heterogeneous-Agent Reinforcement Learning*. Journal of

Machine Learning Research, 25:1–67, 2024. https://www.jmlr.org/papers/v25/23-0488.html.

[ZLFS24] Zhang, Yifeng, Peizhuo Li, Mingfeng Fan y Guillaume Sartoretti: *HeteroLight: A General and Efficient Learning Approach for Heterogeneous Traffic Signal Control*. En *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, páginas 1010–1017, 2024.

[ZLYZ23] Zhou, Mengdi, Xiaoteng Li, Yaodong Yang y Weinan Zhang: *A Survey on Centralized Training for Decentralized Execution in Multi-Agent Reinforcement Learning*. Artificial Intelligence Review, 2023.

[ZWL24] Zhang, Haipeng, Zhiwen Wang y Na Li: *MATLight: Traffic Signal Coordinated Control Algorithm Based on Heterogeneous-Agent Mirror Learning With Transformer*. En *Proceedings of the 23rd International Conference on Autonomous Agents and Multiagent Systems (AAMAS)*, páginas 2582–2584, 2024.